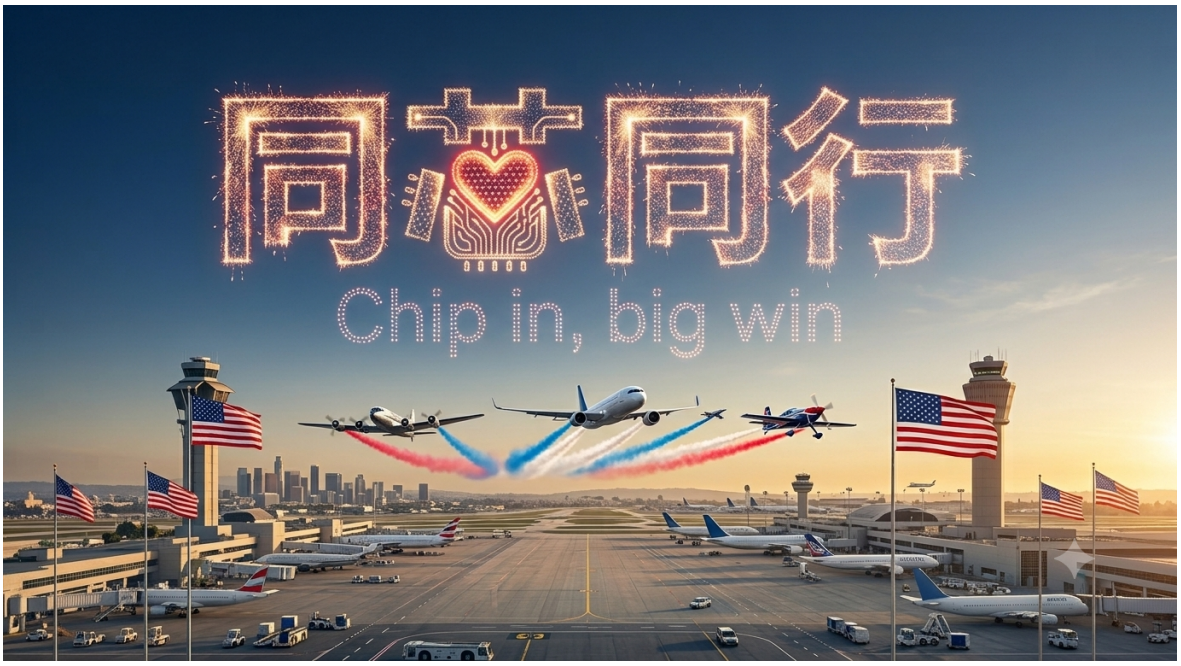


2026年8月北美芯片设计/验证题目汇总

根据同学回忆整理 · 152道真题含答案与解析

Apple · Broadcom · Etched · Google · Intel · Micron · Nvidia

目录

Digital Design

Q1. Digital Design: Charge Sharing Between Two Capacitors (Designer Question)
Q2. Digital Design: CMOS Inverter Cross-Section -- Terminal ID & Body Effect (Designer Question)
Q3. Digital Design: Building a NOT Gate from a 2-Input NAND -- Tie Inputs vs Tie-to-1
Q4. Digital Design: Non-Inverting Op-Amp Gain Derivation via Voltage Divider (Designer Question)
Q5. Digital Design: PPA (Power, Performance, Area) Optimization Trade-offs (Designer Question)
Q6. Digital Design: Propagation-Delay-Induced Glitch -- Direct Path vs Delayed-Inverter Path
Q7. Digital Design: 3-to-8 Decoder Built from NOT and AND Gates Only
Q8. Digital Design: Door Controller FSM with Four States (Fully Closed / Half Open / Fully Open / Half Closed)
Q9. Digital Design: Asynchronous FIFO Design (Dual-Clock, Gray-Coded Pointers)
Q10. Digital Design: What is Gray Code, and Why CDC Signals Need It
Q11. Digital Design: Universal Gates -- Building AND/OR/NOT from NAND Only
Q12. Digital Design: Generic FSM + Counter-Gated State Transition Template
Q13. Digital Design: Setup Time vs Hold Time -- Violations and How to Fix Each
Q14. Digital Design: Divide-by-2 Pulse Circuit (1 FF + 1 AND) and the ICG Fix
Q15. Digital Design: Boolean Expression Simplification -- (A+B)B + CA
Q16. Digital Design: Implement Multiplier using Sequential or Combinational Method
Q17. Digital Design: Generic Divide-by-N with ~50% Duty Cycle
Q18. Digital Design: Design FIFO with Asymmetric Read and Write Data Widths
Q19. Digital Design: Build AXI-Lite 4-Register File
Q20. Digital Design: Design Dual-Port RAM with Simultaneous Read and Write Capability
Q21. Digital Design: Design Sliding Window Min/Max Module for Stream Processing
Q22. Digital Design: Design Overlapping Sequence Detector for Pattern 1011
Q23. Digital Design: Glitch-Free Clock Gating Cell with Enable
Q24. Digital Design: Detect a One-Cycle High Pulse (0→1→0)
Q25. Digital Design: FSM for Divisibility by 3 with Serial Bit Input
Q26. Digital Design: Design Shift Register with Parallel Load and Serial Output
Q27. Digital Design: Implement Memory-Mapped GPIO Module with Direction Control
Q28. Digital Design: Build AXI-Lite Read-Only Status Block
Q29. Digital Design: Implement Direct-Mapped Cache with Tag, Valid, and Data Storage
Q30. Digital Design: Build Parameterizable Pulse Width Modulation (PWM) Generator
Q31. Digital Design: Implement Traffic Light Controller Using Moore FSM
Q32. Digital Design: Gray-Code Counter for CDC (Binary-Internal, Gray-Output Pattern)

SystemVerilog -- Constraints & Randomization

Q33. SystemVerilog: Constraint to Randomize a Power of 2 (Two Methods)
Q34. SystemVerilog: Constraint Inheritance and Override in a Class Hierarchy
Q35. SystemVerilog: Constraint for a 4-Byte-Aligned Address Within a Range
Q36. SystemVerilog: Transaction Constraint Package -- Range, Exclusion Zone, constraint_mode, Inline, Soft
Q37. SystemVerilog: Weighted Random Distribution with `dist` (:= vs :/)
Q38. SystemVerilog: Boundary-Crossing Constraints and Circular Pointer Wraparound
Q39. SystemVerilog: Sequence Item with Cross-Field Parity Constraint + randc Error Injection
Q40. SystemVerilog: Generating Unique Random Values Across Iterations
Q41. SystemVerilog: Diet Planner App -- Class Design with Branching Constraints
Q42. SystemVerilog: Generate Valid Solved 9×9 Sudoku Puzzle
Q43. SystemVerilog: Array with Exactly 3 Identical Elements and 7 Unique Elements

Q44. SystemVerilog: Generate Two 3×3 Matrices with Different Minimum Values
Q45. SystemVerilog: Matrix Where Each Row Has Unique Maximum and All Maximums Distinct
Q46. SystemVerilog: Instruction Stream with Repetition Window Constraints
Q47. SystemVerilog: 10-bit Variable with Uniform Popcount Distribution
Q48. SystemVerilog: Randomly Distribute Array into N Non-Empty Queues
Q49. SystemVerilog: Generate Number with All 1-Bits Grouped in Single Contiguous Run
Q50. SystemVerilog: Solve 8 Queens Problem Using Constraints
Q51. SystemVerilog: Dynamic Array Size 300 with Value Frequency and Consecutive Constraints
Q52. SystemVerilog: Stream with Last 4 Generated Values All Different
Q53. SystemVerilog: Odd-Sized Matrix with Unique Sub-Square Maximums
Q54. SystemVerilog: Generate Instruction Schedule with Latency-Based Non-Overlap Constraints
Q55. SystemVerilog: 5 Set Bits, 80% Consecutive, 20% Non-Consecutive
Q56. SystemVerilog: Generate Two Dynamic Arrays with Size, Sorted, and Membership Constraints

SystemVerilog -- Assertions (SVA)

Q57. SystemVerilog: Identify the Design, Write the RTL, Write All Applicable Assertions
Q58. SystemVerilog: Rising Edge Followed by Falling Edge Within 10 Cycles
Q59. SystemVerilog: Array Access Index Within Valid Range
Q60. SystemVerilog: Write UVM Assertion: Transaction Data Validity Check
Q61. SystemVerilog: Arbiter Grants Requests Fairly
Q62. SystemVerilog: Detect Rising Edge on Signal
Q63. SystemVerilog: Counter Zero Within 1 Cycle After Reset De-assertion
Q64. SystemVerilog: Data Remains Stable for 5 Cycles After Enable
Q65. SystemVerilog: Write Coverage for Protocol Event Sequences
Q66. SystemVerilog: Integrate Assertion Coverage into UVM Testbench
Q67. SystemVerilog: Ensure No Starvation in Multi-Client Arbiter
Q68. SystemVerilog: Output Equals Sum of Two Inputs

SystemVerilog -- Coding

Q69. SystemVerilog: Blocking vs Non-Blocking Assignment -- Display Timing Trace
Q70. SystemVerilog: Generate a 100 MHz Clock with 25% Duty Cycle
Q71. SystemVerilog: `logic` vs `bit` Data Types
Q72. SystemVerilog: `==` (Logical Equality) vs `===` (Case Equality)
Q73. SystemVerilog: fork...join vs join_any vs join_none
Q74. SystemVerilog: Indexed Part-Select (+: and -:) to Repack Unpacked Arrays
Q75. SystemVerilog: Four Parallel Tasks -- First to Finish Cancels the Rest
Q76. SystemVerilog: Measuring Clock Cycles Between Two Events
Q77. SystemVerilog: Debug a Broken UVM Monitor Class (4 Errors)
Q78. SystemVerilog: Polymorphism in SV/UVM, with a Worked Example

Verification -- Test Plan

Q79. Verification -- Test Plan: Frequency Monitor (FMON) DUT
Q80. Verification -- Test Plan: Basic SRAM Verification
Q81. Verification: Searching for Legal PLL Divider Coefficients (M, N, P)
Q82. Verification -- Test Plan: Divide-by-2 Pulse Circuit
Q83. Verification: Block-Level vs Top-Level Verification Strategy
Q84. Verification: General Methodology for Verifying a Given FSM

UVM

Q85. UVM: Starting Multiple Sequences on Multiple Sequencers in Parallel
Q86. UVM: Build Phase (Top-Down) vs Connect Phase (Bottom-Up) Execution Order
Q87. UVM: uvm_config_db -- set()/get() and Passing a Virtual Interface

Q88. UVM: What is a Virtual Interface, and Why Is It Needed
Q89. UVM: Verification Environment Architecture -- What Each Component Does
Q90. UVM: Using an Associative Array in a Scoreboard
Q91. UVM: Starting Sequences on Multiple Sequencers Sequentially (via foreach)
Q92. UVM: PLL Sequence Item / Sequence / Driver, and a Two-Queue Scoreboard
Q93. UVM: Base/Derived Handle Assignment vs Factory Type Override
Q94. UVM: How a Sequence Item Travels From Sequence to Driver
Q95. UVM: Design Monitor for Serial 4-bit Interface with Start/Stop Framing Patterns
Q96. UVM: Design Scoreboard Using uvm_tlm_analysis_fifo for Timing Decoupling
Q97. UVM: Implement Driver that Splits 64-bit Transaction into Two 32-bit Beats
Q98. UVM: Implement Driver with Response Path Back to Sequencer

Algorithms & Scripting

Q99. Algorithms: Rearrange a Sorted Array in Alternating Max-Min Order
Q100. Algorithms: Minimum Absolute Difference in an Unsorted Array
Q101. Algorithms: Generate All Valid Knight Moves on a Chessboard
Q102. Algorithms & Scripting: Common Elements in Three Sorted Arrays
Q103. Algorithms & Scripting: Longest Consecutive Sequence
Q104. Algorithms & Scripting: Count Triplets with Zero Sum
Q105. Algorithms & Scripting: Kth Largest Element in an Array
Q106. Algorithms & Scripting: Equilibrium Index
Q107. Algorithms & Scripting: Word Break
Q108. Algorithms & Scripting: Sort Colors
Q109. Algorithms & Scripting: Count Occurrences in Sorted Array
Q110. Algorithms & Scripting: Search Insert Position
Q111. Algorithms & Scripting: Longest Repeating Character Replacement
Q112. Algorithms & Scripting: Count Character Occurrences
Q113. Algorithms & Scripting: Permutation in String
Q114. Algorithms & Scripting: Longest Common Subsequence
Q115. Algorithms & Scripting: Longest Substring Without Repeating Characters
Q116. Algorithms & Scripting: All Pairs With Given Sum
Q117. Algorithms & Scripting: Contains Duplicate II
Q118. Algorithms & Scripting: 3Sum
Q119. Algorithms & Scripting: First Duplicate Element
Q120. Algorithms & Scripting: Rotate Array
Q121. Algorithms & Scripting: Product of Array Except Self
Q122. Algorithms & Scripting: Climbing Stairs
Q123. Algorithms & Scripting: Search in Rotated Sorted Array
Q124. Algorithms & Scripting: Find Peak Element
Q125. Algorithms & Scripting: Reverse String
Q126. Algorithms & Scripting: Find the Index of the First Occurrence in a String
Q127. Algorithms & Scripting: Run-Length Encoding
Q128. Algorithms & Scripting: Valid Anagram
Q129. Algorithms & Scripting: Longest Common Prefix

Computer Architecture

Q130. Computer Architecture: Out-of-Order Execution in Detail
Q131. Computer Architecture: MESI and MOESI Cache Coherence (Two-Core Scenario)
Q132. Computer Architecture: RISC vs CISC -- Pros and Cons
Q133. Computer Architecture: Design Parameterized ALU with Multiple Arithmetic and Logic Operations
Q134. Computer Architecture: Implement Sequential Divider Using Shift-Subtract Algorithm
Q135. Computer Architecture: Implement Dirty Bit Logic for Write-Back Cache

Q136. Computer Architecture: Design Round-Robin Arbiter for Fair Request Arbitration
Q137. Computer Architecture: Implement Thread Mask Logic for Predicated Execution
Q138. Computer Architecture: Design Branch Comparator and Control Logic
Q139. Computer Architecture: Design ALU Control Signal Decoder
Q140. Computer Architecture: Implement True LRU Replacement for 4-Way Set-Associative Cache
Q141. Computer Architecture: Design Finite State Machine for Cache Line Refill from Memory
Q142. Computer Architecture: Implement Reorder Buffer (ROB) Entry Management Logic
Q143. Computer Architecture: Design Shared Memory Bank Conflict Detector
Q144. Computer Architecture: Implement Instruction Decode Logic

Low-Power Design

Q145. Low-Power Design: Latch-based Integrated Clock Gating Cell
Q146. Low-Power Design: Rewrite a 4:1 MUX Tree to Suppress Internal Glitches
Q147. Low-Power Design: Always-On to Power-Gated Domain Handshake
Q148. Low-Power Design: Glitch-Free Frequency Divider (1 / 2 / 4)
Q149. Low-Power Design: Find the Glitch in a Naive Clock Gate
Q150. Low-Power Design: Bus-Invert Coding for Off-Chip Toggle Reduction
Q151. Low-Power Design: SystemVerilog Assertions for Power Sequencing
Q152. Low-Power Design: Async FIFO with Level-Shifted Handshakes

Digital Design

Q1. Digital Design: Charge Sharing Between Two Capacitors (Designer Question)

QUESTION

Two capacitors C1 and C2 are charged independently to voltages V1 and V2. At t = 0 a switch closes and connects the two capacitors together in parallel (top plates tied to one node, bottom plates tied to a common ground).

(a) Using charge conservation (Q_initial = Q_final), derive the final common voltage Vf across the two capacitors after the switch closes.

(b) Compute the energy dissipated during the charge-sharing transient, and explain physically where that energy goes.

(c) Give one real chip-design example where uncontrolled charge sharing is a problem (e.g. SAR ADC capacitor array, a floating dynamic/domino logic node, or shared-bus precharge), and describe one circuit technique used to mitigate it.

ANSWER

```
// (a) Charge conservation: Q_initial = Q_final
//   Q_initial = C1*V1 + C2*V2
//   After the switch closes, both caps sit at the same final voltage Vf (parallel).
//   Q_final = (C1 + C2) * Vf
//   Setting Q_initial = Q_final:
//
//       Vf = (C1*V1 + C2*V2) / (C1 + C2)
//
//   i.e. Vf is the capacitance-weighted average of V1 and V2.
//
// (b) Energy before switching:
//   E_initial = 0.5*C1*V1^2 + 0.5*C2*V2^2
//   Energy after switching:
//   E_final   = 0.5*(C1+C2)*Vf^2
//   Energy lost:
//   E_lost = E_initial - E_final
//          = 0.5 * (C1*C2)/(C1+C2) * (V1-V2)^2
//
//   This loss is NOT stored anywhere -- it is dissipated as I^2R heat in
//   the switch's on-resistance and any interconnect resistance during the
//   transient current spike that redistributes charge. Only charge (Q=CV)
//   is conserved here; energy is not.
//
// (c) Example: SAR ADC capacitor DAC.
//   When the SAR logic switches capacitor taps between Vref and ground on
//   each bit decision, charge redistributes across the capacitor array
//   before the comparator re-evaluates. If the array hasn't fully settled,
//   the comparator sees a settling error, which shows up as DNL/INL
//   nonlinearity in the ADC transfer curve.
//   Mitigation: insert a settling ("wait") phase between DAC switching and
//   the comparator decision, or use a monotonic / split-capacitor
//   switching scheme that reduces the amount of charge moved per step.
```

解释

考点：开关电容电路中电荷守恒（Q = CV）的应用，这是模拟/混合信号方向面试最常见的送分题之一，数字方向也常作为交叉考点出现。

关键公式：两电容并联后电压相等，且电荷总量不变，所以 Vf 是以电容值为权重的电压加权平均：Vf = (C1V1 + C2V2)/(C1+C2)。

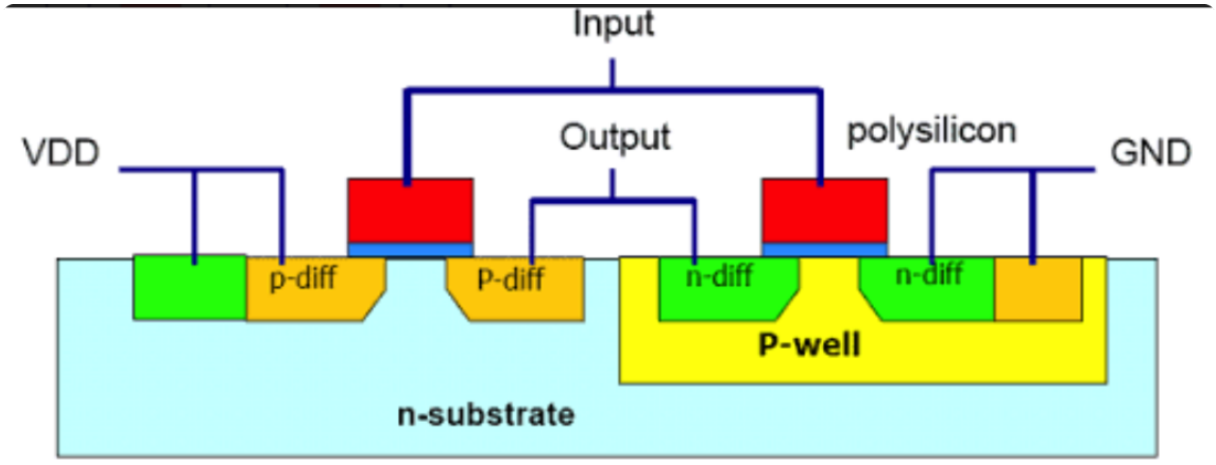
易错点：不少同学会下意识用"能量守恒"去求 Vf，这是错的——电荷共享过程一定伴随能量损耗（开关寄生电阻上瞬态电流产生的热耗），只有电荷 Q 守恒，能量不守恒，题目提示 Q_initial=Q_final 就是在提醒用电荷而不是能量做守恒方程。

延伸考点：SAR ADC 电容阵列切换、动态/多米诺逻辑的浮空节点、总线预充电（precharge）共享，都是电荷共享导致电压误差或功耗损失的实际场景，面试官很喜欢顺着这道题继续往下问具体电路场景。

Q2. Digital Design: CMOS Inverter Cross-Section -- Terminal ID & Body Effect (Designer Question)

QUESTION

The figure below shows the physical cross-section of a CMOS inverter (PMOS on the left in an n-substrate, NMOS on the right in a P-well).



(a) Label all four terminals (Source, Drain, Gate, Body) for BOTH the PMOS and the NMOS directly from the cross-section, and state which physical region in the figure corresponds to each terminal.

(b) In this diagram the PMOS body (n-substrate) is tied to VDD and the NMOS body (P-well) is tied to GND -- the standard/correct biasing. In detail, explain what happens to each transistor's threshold voltage Vt if the NMOS body is NOT tied to GND, or the PMOS body is NOT tied to VDD. Don't just say "Vt shifts" -- explain the physical mechanism.

ANSWER

```
// (a) Terminal identification (left to right in the figure):
//
//   PMOS (left, sits in the n-substrate):
//     Body   = leftmost GREEN region  -- n+ tap into the n-substrate, tied to VDD
//     Source = first ORANGE region ("p-diff") -- p+ diffusion, tied to VDD
//     Gate   = left RED poly + BLUE gate-oxide stack, tied to Input
//     Drain  = second ORANGE region ("P-diff") -- p+ diffusion, tied to Output
//
//   NMOS (right, sits in the P-well):
//     Drain  = first GREEN region ("n-diff") -- n+ diffusion, tied to Output
//     Gate   = right RED poly + BLUE gate-oxide stack, tied to Input
```

```
//     Source = second GREEN region ("n-diff") -- n+ diffusion, tied to GND
//     Body   = rightmost ORANGE region -- p+ tap into the P-well, tied to GND
//
//   Color logic in the figure: GREEN = n-type heavily-doped region (n+),
//   ORANGE = p-type heavily-doped region (p+), regardless of whether that
//   region happens to be acting as a source/drain or as a body/well tap --
//   e.g. PMOS body tap is n+ (green) because the n-substrate is n-type,
//   while PMOS source/drain are p+ (orange) because that's what forms the
//   PMOS junction; it is the mirror image for the NMOS/P-well side.
//
// (b) Body effect (back-gate effect) if body is NOT tied to the correct rail:
//
//   The threshold voltage of a MOSFET is NOT a fixed constant -- it depends
//   on the source-to-body voltage Vsb through the body-effect equation:
//
//       Vt = Vt0 + gamma * ( sqrt(2*phi_F + Vsb) - sqrt(2*phi_F) )
//
//   where Vt0 is the threshold at Vsb = 0, gamma is the body-effect
//   coefficient (process-dependent), and phi_F is the Fermi potential.
//
//   NMOS case (body should be GND):
//     If the P-well body is left floating, tied to a voltage ABOVE GND,
//     or simply mis-connected, then Vsb = Vsource - Vbody becomes positive
//     (source sits above body potential). This reverse-biases the
//     source-body junction, which widens the depletion region under the
//     channel. A wider depletion region means MORE gate charge is needed
//     to reach strong inversion, so Vtn INCREASES (becomes more positive).
//     A higher Vtn reduces the effective overdrive Vgs-Vt at a fixed Vgs,
//     which lowers drive current (Ion ~ (Vgs-Vt)^2) and slows switching.
//
//   PMOS case (body should be VDD):
//     If the n-substrate body is tied BELOW VDD (or floating), the source
//     (held near VDD) ends up at a higher potential than the body, which
//     again reverse-biases the source-body junction (same physics,
//     mirrored polarity). This increases the MAGNITUDE of the (negative)
//     PMOS threshold, |Vtp| grows, reducing PMOS drive strength the same
//     way.
//
//   Additional consequences beyond the Vt number itself:
//     1. Reduced Ion / increased gate delay, since both Vtn and |Vtp| grow
//        and overdrive shrinks -- timing closure can fail even though the
//        gate is still logically correct.
//     2. If the body is pushed the OTHER direction (forward-biasing the
//        source/drain-body junction instead), you get diode leakage current
//        into the well/substrate and risk turning on the parasitic
//        bipolar transistor formed by source/drain-well-substrate, which
//        is the classic trigger for CMOS LATCH-UP.
//     3. A floating body (not just mis-tied) makes Vsb dependent on
//        switching activity and capacitive coupling, so Vt drifts
//        dynamically -- this is exactly the "floating body effect" seen
//        in SOI devices, and it is why bulk CMOS layouts require regular
//        body-tie (well-tap) contacts placed at a DRC-mandated density,
//        to keep the well/substrate resistance low enough that the body
//        stays clamped to the intended rail everywhere, not just at one
//        tie point far away.
```

解释

(a) 关键是先分清"结构角色"和"掺杂颜色"两条线：图里绿色统一代表 n+ 重掺杂区，橙色统一代表 p+ 重掺杂区，不管它是当 source/drain 还是当 body 引出端——PMOS 在 n-substrate 里，所以 PMOS 的 body 引出用 n+（绿色），而 PMOS 真正的沟道两端（source/drain）是 p+（橙色）；NMOS 在 P-well 里，逻辑正好相反。

(b) 考点是阈值电压不是常数，而是源体电压 Vsb 的函数（body effect / back-gate effect）。正常情况下 NMOS 的 body 接 GND、source 也在 GND 附近，Vsb≈0，Vt 处在最低点。如果 body 没接对（悬空、接错电平、或者接得比 source 还低），source 相对 body 变成正偏（反偏结），耗尽层变宽，需要更多栅极电荷才能翻转沟道，于是 Vtn 变大；PMOS 同理，只是极性相反，body 没接到 VDD 会导致 |Vtp| 变大。

阈值变大直接的后果是过驱动电压 Vgs-Vt 变小，驱动电流按 (Vgs-Vt)^2 量级下降，电路变慢，严重时时序收不了；如果 body 被推向另一个方向（正偏 source/drain-body 结），还会引出寄生二极管漏电，甚至触发寄生 BJT 导致闩锁（latch-up）——这也是为什么版图规则里必须按密度要求打 body tie（well tap），保证阱/衬底电阻足够小，body 电位在整个版图范围内都能被钳位在正确的电源轨上，而不只是在一个点上"接对了"。

Q3. Digital Design: Building a NOT Gate from a 2-Input NAND -- Tie Inputs vs Tie-to-1

QUESTION

You are given a 2-input NAND gate and need to build a NOT (inverter) function out of it. There are two common ways to do this:

(1) Tie both NAND inputs together and drive them with the same signal.

(2) Feed the signal into one input, and tie the other input permanently to logic 1 (VDD).

Which of these two methods is better, and why? Discuss in terms of input capacitance / loading, switching power, and drive strength (rise/fall) symmetry.

ANSWER

```
// Both methods are functionally correct:
//   (1) NAND(A, A) = NOT(A AND A) = NOT A
//   (2) NAND(A, 1) = NOT(A AND 1) = NOT A
//
// Method (2) -- tie one input to VDD -- is the better choice. Reasons:
//
// 1) Input capacitance seen by the DRIVING stage:
//    A 2-input CMOS NAND has 2 PMOS (parallel, to VDD) and 2 NMOS
//    (series, to GND). Each input net (A, B) drives one PMOS gate AND
//    one NMOS gate, i.e. Cin_per_input = Cg(PMOS) + Cg(NMOS).
//
//    Method (1): both A and B nets are physically the SAME wire, so the
//    driving stage must charge/discharge BOTH gate pairs every
//    transition:
//        Cin_seen_by_driver = 2 * (Cg(PMOS) + Cg(NMOS))   -- 2x load
//
//    Method (2): the "tie-to-1" input is a static DC connection to VDD,
//    it never toggles, so the driving stage only ever charges/discharges
//    the ONE active input's gate pair:
//        Cin_seen_by_driver = 1 * (Cg(PMOS) + Cg(NMOS))   -- 1x load
//
//    So method (2) presents HALF the input capacitance to the previous
//    stage compared to method (1).
//
// 2) Dynamic switching power:
//    P_dynamic ~ alpha * C * V^2 * f. Since method (1) doubles the
//    switched capacitance on every transition of the shared input net,
//    it also roughly doubles the dynamic power spent just charging and
```

```
//    discharging that input node, for identical logical activity.
//    Method (2)'s tied-to-1 input never switches, so it contributes
//    (ideally) zero dynamic power -- only a one-time static charge.
//
// 3) Drive-strength symmetry (rise/fall skew):
//    Method (1) ties BOTH the parallel PMOS pair and BOTH the series
//    NMOS pair to the same input. The pull-up now looks like 2 parallel
//    PMOS (stronger, lower effective resistance) while the pull-down is
//    2 SERIES NMOS (weaker, higher effective resistance) -- this is an
//    inherently skewed inverter (fast rise, slow fall) unless the
//    transistors are deliberately resized to compensate.
//    Method (2) leaves one full PMOS permanently OFF (gate=1) and one
//    full NMOS permanently ON (gate=1); the actual switching path is a
//    single PMOS pulling up in parallel with a single (always-on) NMOS
//    already in series, i.e. electrically it behaves like one normal
//    PMOS pull-up against one normal NMOS pull-down -- the natural
//    balanced ratio of a standard inverter, no extra skew introduced.
//
// 4) What does NOT change between the two methods:
//    Both methods keep all 4 transistors physically in the gate, so the
//    OUTPUT node capacitance (drain diffusions of all 4 devices) is the
//    same either way -- the benefit of method (2) is specifically on the
//    INPUT side (fan-in loading), not the output side.
//
// Conclusion: method (2) -- tie one input to logic 1 -- is preferred in
// practice. This is exactly why standard-cell libraries provide dedicated
// TIEHI/TIELO cells: synthesis tools reuse a NAND/NOR as an inverter by
// tying one input to a clean, static tie-cell rail rather than fanning
// the same signal into both inputs.
```

解释

两种接法在功能上都对，但从电路指标看差别很大，核心是"输入电容"和"驱动对称性"两点。

(1) 两输入接在一起：驱动这一级的前级电路，要同时给 2 个 PMOS 栅极和 2 个 NMOS
栅极充放电，输入电容是正常单输入的 2 倍，前级驱动负担和动态功耗都翻倍；而且下拉网络是两个 NMOS
串联（等效电阻变大），上拉网络是两个 PMOS
并联（等效电阻变小），上下拉强度不对称，导致上升沿快、下降沿慢，是一个"偏态"的反相器。

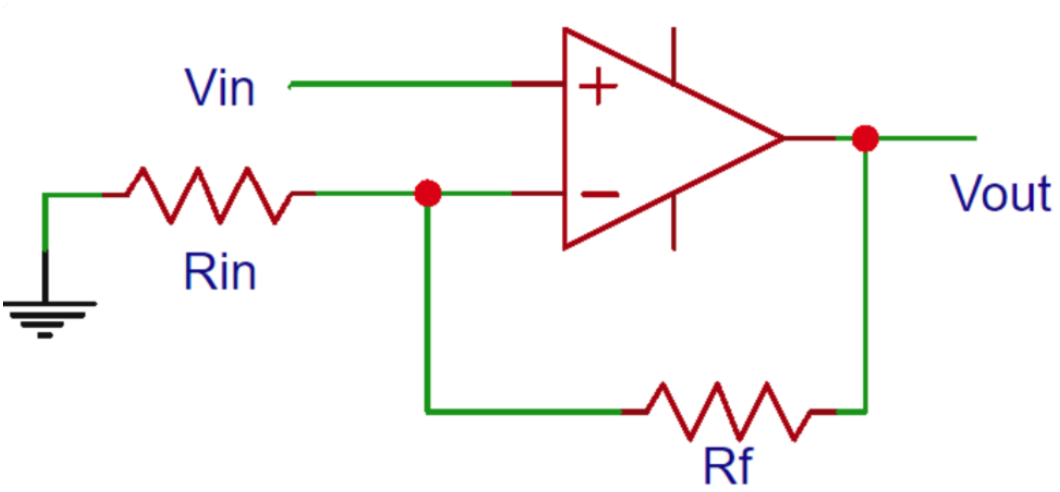
(2) 一个输入接死到 1：接 VDD 的那个 PMOS 栅极恒为高电平，永远关断；对应的 NMOS
栅极恒为高电平，永远导通。真正参与开关的只有一路 PMOS 和一路
NMOS，前级只需要驱动这一路的栅极电容，输入电容是方法(1)的一半；同时接 1 的那个输入是静态直流连接，不会有动态开关功耗；等效驱动结构就是一个正常反相器的上下拉比例，不会引入额外偏态。

两种方法输出节点的电容其实是一样的（4个管子的漏端寄生电容都还在），差别只体现在输入侧负载和功耗上——所以方法(2)更优，这也是为什么数字标准单元库里会专门做 TIEHI/TIELO
这种"接死电平"的单元，综合工具把 NAND/NOR
复用成反相器时都是接一个固定电平，而不是把信号扇到两个输入上。

Q4. Digital Design: Non-Inverting Op-Amp Gain Derivation via Voltage Divider (Designer Question)

QUESTION

For the non-inverting op-amp circuit shown below, Vin drives the "+" input directly. Rin connects the "-" input node to GND, and Rf feeds back from Vout to that same "-" input node.



Gain = Vout/Vin = 1 + Rf/Rin

Derive Gain = Vout/Vin = 1 + Rf/Rin from first principles, using the ideal op-amp assumptions (infinite open-loop gain, infinite input impedance / zero input bias current, virtual short between the two inputs). Frame the derivation around the idea that the "-" node is simply a voltage divider between Vout and GND.

ANSWER

```
// Ideal op-amp assumptions used:
//   1) Infinite open-loop gain -> with negative feedback, V+ = V-  (virtual short)
//   2) Infinite input impedance -> no current flows INTO either input pin
//
// Step 1 -- identify the "-" node as a voltage divider:
//   Looking from the "-" node, Rin goes down to GND and Rf goes up to
//   Vout. Because assumption (2) says no current flows into the "-"
//   input pin itself, ALL of the current that flows through Rf must
//   also flow through Rin -- the op-amp input draws none of it. That
//   means Rin and Rf form a plain series path from Vout to GND, with
//   the "-" node sitting exactly at the tap point of that divider:
//
//       V(-) = Vout * [ Rin / (Rin + Rf) ]
//
//   (standard voltage-divider formula: the node between the two
//   resistors sees Vout scaled by the fraction of resistance BELOW it)
//
// Step 2 -- apply the virtual short:
//   V(+) = Vin  (driven directly)
//   V(+) = V(-)   (assumption 1, ideal op-amp with negative feedback)
//   =>  V(-) = Vin
//
// Step 3 -- combine and solve for gain:
//   Vin = Vout * [ Rin / (Rin + Rf) ]
//   Vout / Vin = (Rin + Rf) / Rin
//   Vout / Vin = 1 + Rf/Rin
//
// So:
//       Gain = Vout / Vin = 1 + Rf / Rin
//
// Sanity checks:
//   Rf = 0        -> Gain = 1           (unity-gain buffer / voltage follower)
//   Rin -> infinity -> Gain = 1         (no feedback divider action left)
//   Rf >> Rin      -> Gain ~ Rf/Rin     (large closed-loop gain, set purely
```

```
//                                        by the resistor ratio, independent
//                                        of the op-amp's own open-loop gain)
```

解释

考点：理想运放两条假设（虚短、虚断）+
分压公式，是模拟电路面试里同相放大器增益推导的标准送分题。

关键思路（题目提示"本质是分压"）：因为虚断（"-"输入端不吸电流），流过 Rf 的电流必然全部流过 Rin，Rin 和 Rf 就是一条从 Vout 接到 GND 的普通串联支路，"-"节点正好是这条支路的分压点，所以 V(-) = Vout × Rin/(Rin+Rf)，这一步跟运放本身的增益、非线性都没关系，纯粹是电阻分压。

再用虚短 V(+) = V(-)，而 V(+) 直接等于 Vin，两式联立就能解出 Vout/Vin = 1 + Rf/Rin。

延伸考点：面试官经常会追问极限情况——Rf=0 时电路退化成电压跟随器（增益为1），或者反过来问同相放大器和反相放大器（Vin接Rin那一端、"+"接地）分压公式的区别，反相放大器的"-"节点是虚地（V(-)=0），分压公式形式不同，容易搞混，需要能讲清楚两者推导逻辑的差异。

Q5. Digital Design: PPA (Power, Performance, Area) Optimization Trade-offs (Designer Question)

QUESTION

In digital IC design, PPA stands for Power, Performance (timing), and Area.

(a) Briefly define each of the three axes.

(b) For EACH axis, list at least two concrete optimization techniques a designer or synthesis/backend tool commonly uses.

(c) Explain why PPA is usually described as a "triangle" or trade-off space rather than three independently optimizable metrics -- give at least two concrete examples where optimizing one axis directly hurts another.

ANSWER

```
// (a) Definitions
//   Power       -- total energy dissipated by the chip, usually split into
//                  dynamic power (switching activity) and static/leakage
//                  power (current that flows even when idle).
//   Performance -- how fast the design can run: max clock frequency
//                  (Fmax), critical-path delay, throughput/latency.
//   Area        -- silicon die area consumed, which drives cost (more
//                  area = fewer dies per wafer = higher $/chip) and also
//                  correlates with routing congestion and yield.
//
// (b) Common optimization techniques per axis
//   POWER:
//     - Clock gating: disable the clock to idle registers/blocks so they
//       don't switch (removes dynamic power for unused logic).
//     - Power gating: cut Vdd to unused blocks entirely (removes leakage,
//       not just dynamic power), at the cost of wake-up latency.
//     - Multi-Vt cell swapping: use high-Vt (low-leakage, slower) cells
//       on non-critical paths, keep low-Vt (leaky, fast) cells only on
//       the critical path.
//     - Voltage/frequency scaling (DVFS): lower Vdd when full speed
//       isn't needed -- dynamic power scales with V^2.
//     - Reduce switching activity: better data encoding (e.g. Gray code
//       counters), operand isolation, glitch reduction via logic
//       restructuring/balanced path delays.
//
```

```
//   PERFORMANCE (timing):
//     - Logic restructuring / re-timing: move logic across register
//       boundaries to balance stage delays and shorten the critical path.
//     - Gate/buffer sizing & Vt swap to low-Vt on the critical path.
//     - Pipelining: break a long combinational path into more, shorter
//       pipeline stages to raise Fmax (at the cost of latency + area).
//     - Reduce logic depth: technology mapping to wider/faster gates,
//       or algorithmic restructuring (e.g. carry-lookahead vs ripple
//       carry adder).
//     - Placement/routing optimization to shorten critical-path wire
//       delay (a dominant term in modern deep-submicron nodes).
//
//   AREA:
//     - Resource sharing: reuse one adder/multiplier via muxing instead
//       of instantiating multiple copies.
//     - Logic minimization / don't-care optimization during synthesis.
//     - Fewer pipeline stages / smaller register files where timing
//       allows it.
//     - Denser standard-cell library selection, tighter floorplanning.
//
// (c) Why it's a triangle, not 3 independent knobs
//   Example 1 -- Power vs Performance:
//     Swapping cells to low-Vt speeds up the critical path (better
//     performance) but low-Vt cells leak more, directly increasing
//     static power. Likewise raising Vdd improves Fmax but power grows
//     with V^2 -- you cannot get both "faster" and "lower power" from
//     the same voltage/Vt knob simultaneously.
//   Example 2 -- Performance vs Area:
//     Pipelining shortens the combinational path per stage (raises
//     Fmax) but requires MORE flip-flops and clock-tree buffering,
//     directly increasing area (and, incidentally, power from the
//     added clocking).
//   Example 3 -- Power vs Area:
//     Power gating and fine-grained clock gating reduce power but
//     require extra header/footer switch cells, isolation cells, and
//     control logic -- all of which cost area.
//
//   In practice a designer doesn't "optimize PPA" in the abstract --
//   they pick a target for one or two axes (e.g. "meet this Fmax, meet
//   this power budget") and let area absorb the remaining trade-off, or
//   vice versa, depending on the product's priorities (a mobile SoC
//   prioritizes power, an HPC chip prioritizes performance, a
//   cost-sensitive IoT chip prioritizes area).
```

解释

考点：PPA（功耗-性能-面积）是数字后端/综合面试几乎必问的话题，考察的不是背几个技巧，而是能不能讲清楚"三者互相牵制"这个核心逻辑。

回答结构建议：先各自定义清楚（动态功耗/静态漏电，Fmax/关键路径，面积/成本），再各举2-3个具体手段，最后一定要落到"三角关系"——面试官最想听到的是你能举出"优化A导致B变差"的具体例子，而不是把三个方向的技巧堆在一起念。

最常考的三对矛盾：低Vt换速度会增加漏电（功耗上升），流水线提升频率但寄存器和时钟树面积增加（面积上升，功耗也可能上升），power gating/clock gating省功耗但需要额外的开关单元和隔离逻辑（面积上升）。

延伸：面试官经常会追问"如果只能选一个优先级，你怎么决策"，这时候要能结合具体产品场景（移动端优先功耗、服务器芯片优先性能、IoT芯片优先成本/面积）来回答，说明你有做工程权衡（trade-off）的思维，而不只是纸面知识。

Q6. Digital Design: Propagation-Delay-Induced Glitch -- Direct Path vs Delayed-Inverter Path

QUESTION

Consider a 2-input AND gate. One input is driven directly by signal A. The other input is driven by A after passing through a NOT gate that has propagation delay tpd (i.e. B = NOT(A), but B only updates tpd after A changes).

(a) Draw (describe) the timing diagram of A, the delayed inverter output B, and the AND-gate output Y, for a single 0->1 transition of A at time t0.

(b) What is Y under ideal (zero-delay) conditions, and why does the presence of tpd change that answer?

(c) What is this phenomenon called, and name one real use for it as well as one real problem it can cause in a digital design.

ANSWER

```
// (a) Timing diagram (time flows left -> right):
//
//   A   :  _______________/------------------------------
//                        ^t0
//
//   B=NOT(A), IDEAL (no delay):
//       :  --------------\_____________________________
//                        ^t0            (falls exactly at t0)
//
//   B=NOT(A), REAL (delayed by tpd):
//       :  --------------------\_____________________
//                        ^t0     ^t0+tpd (falls tpd LATE)
//
//   Y = AND(A, B_real):
//       :  ____________________/-----\________________
//                        ^t0     ^t0+tpd
//                        |<-- tpd -->|
//                        (narrow HIGH pulse of width = tpd)
//
// (b) Ideal case: Y = AND(A, NOT(A)) = 0 always -- A and its true
//     complement can never both be 1 at the same time, by definition.
//     Real case: because the inverter's output lags A by tpd, there is
//     a short window [t0, t0+tpd] where A has ALREADY risen to 1 but
//     B (the delayed inverse) HASN'T fallen yet -- it is still showing
//     the "old" value of 1. During that window both AND inputs are
//     momentarily 1, so Y glitches HIGH for a pulse of width tpd, even
//     though the steady-state logic equation says Y should always be 0.
//
// (c) This is a (dynamic/static) HAZARD -- and this exact construction
//     (AND of a signal with its own delayed inverse) is also used
//     INTENTIONALLY as a one-shot PULSE GENERATOR / edge detector: you
//     can deliberately size the inverter delay (or chain several
//     inverters) to produce a clean, controlled-width pulse on every
//     rising edge of A, used e.g. in power-on-reset generators or to
//     create a narrow write-enable strobe from a level signal.
//     The PROBLEM case is when this happens UNINTENTIONALLY in random
//     combinational logic (a "hazard"/glitch): the momentary incorrect
//     value can be captured by a downstream flip-flop if it happens to
//     sample right during the glitch window, corrupting state, or it
//     can cause spurious toggling that wastes dynamic power on nodes
//     that are logically supposed to be static.
```

解释

考点：组合逻辑的传播延迟（propagation delay）如何产生毛刺（glitch/hazard），是数字电路"理想逻辑 vs 真实电路时序"这条主线最经典的一道题。

关键点：纸面上（零延迟假设）A AND NOT(A)
恒为0，这是逻辑代数层面的结论；但真实门电路里反相器的输出会比输入晚 tpd 才翻转，所以 A 刚跳变的那一小段时间窗口里两路信号会"同时为1"，输出出现一个宽度约等于 tpd 的窄脉冲，这就是毛刺。

双重身份：往坏了看是"竞争冒险/hazard"（可能被下游触发器误采样，或者产生不必要的翻转功耗）；往好了看，同样的结构故意拿来用就是"单稳态脉冲发生器/edge detector"——比如上电复位电路、把电平信号转成窄脉冲的写使能信号，都是利用延迟反相器故意制造这个"毛刺"。

延伸考点：如何消除非期望的毛刺——常见做法是用卡诺图法添加冗余项消除静态冒险，或者在下游改用同步时钟采样代替直接用组合逻辑输出去触发时序元件，避免采到毛刺。

Q7. Digital Design: 3-to-8 Decoder Built from NOT and AND Gates Only

QUESTION

Design (draw / describe at the gate level) a 3-to-8 decoder using only NOT gates and AND gates. The decoder has 3 inputs A2 A1 A0 (A2 = MSB) and 8 outputs Y0..Y7, where exactly one output Yi is HIGH (active-high) when the binary value of A2A1A0 equals i, and all other outputs are LOW.

(a) Write the Boolean (minterm) equation for each output Yi in terms of A2, A1, A0 and their complements.

(b) Describe the gate-level structure: how many NOT gates are needed, how many AND gates, and how many inputs does each AND gate need.

(c) What happens to the gate count if you generalize this to an N-to-2^N decoder?

ANSWER

```
// (a) Minterm equations (each Yi is the AND of the literals matching the
//     binary representation of i; a '0' bit means use the COMPLEMENT of
//     that input, a '1' bit means use the input directly):
//
//   Y0 = A2' * A1' * A0'      (binary 000)
//   Y1 = A2' * A1' * A0       (binary 001)
//   Y2 = A2' * A1  * A0'      (binary 010)
//   Y3 = A2' * A1  * A0       (binary 011)
//   Y4 = A2  * A1' * A0'      (binary 100)
//   Y5 = A2  * A1' * A0       (binary 101)
//   Y6 = A2  * A1  * A0'      (binary 110)
//   Y7 = A2  * A1  * A0       (binary 111)
//
//   ( X' denotes NOT X )
//
// (b) Gate-level structure:
//
//   3 NOT gates -- one per input, to generate A2', A1', A0':
//
//       A2 ---[NOT]--- A2'
//       A1 ---[NOT]--- A1'
//       A0 ---[NOT]--- A0'
//
//   8 THREE-INPUT AND gates -- one per output, each wired to the three
//   literals (true or complemented) matching that minterm:
//
```

```
//       A2' A1' A0' ---[AND3]--- Y0
//       A2' A1' A0  ---[AND3]--- Y1
//       A2' A1  A0' ---[AND3]--- Y2
//       A2' A1  A0  ---[AND3]--- Y3
//       A2  A1' A0' ---[AND3]--- Y4
//       A2  A1' A0  ---[AND3]--- Y5
//       A2  A1  A0' ---[AND3]--- Y6
//       A2  A1  A0  ---[AND3]--- Y7
//
//   Total: 3 NOT gates + 8 three-input AND gates = 11 gates.
//   Each input signal (true or complemented) fans out to exactly 4 of
//   the 8 AND gates (half of all minterms use A2 true, the other half
//   use A2', and likewise for A1 and A0) -- this fan-out is a key
//   loading consideration for a real physical design.
//
// (c) Generalizing to an N-to-2^N decoder:
//     - NOT gates:  N            (one complement per input)
//     - AND gates:  2^N          (one N-input AND gate per output/minterm)
//     - Each AND gate needs N inputs (one literal per input bit).
//     - Gate count grows LINEARLY in N for the inverters but
//       EXPONENTIALLY in N for the AND gates (2^N of them) -- this is
//       exactly why large decoders (e.g. address decoders for big
//       memories) are NOT built as one flat AND-gate layer in practice;
//       they're built hierarchically / pre-decoded (split an N-bit
//       decode into two smaller decodes and combine with a 2-input AND
//       per output) to keep fan-in and area growth under control.
```

解释

考点：最基础的组合逻辑设计题——从真值表/最小项直接翻译成门级电路，同时考察是否理解"每个输出就是一个最小项"这个译码器的本质。

方法：3个输入各自过一个反相器拿到补码，8个输出分别是一个三输入与门，选哪三个信号（原变量还是补码）完全由输出编号的二进制展开决定——第i个输出对应的与门，输入是"把 i 写成3位二进制，1取原变量，0取补码"。

延伸考点：面试官很喜欢追问推广到 N-to-2^N 译码器时门的数量怎么增长——反相器线性增长（N个），与门指数增长（2^N个，每个N输入），这正是大译码器（比如存储器地址译码）在实际版图里不会用一层平坦的与门实现、而要做分级预译码（pre-decode）来控制面积和扇入的原因，能答到这一层说明理解译码器不仅是背真值表，而是有实际电路设计的权衡意识。

Q8. Digital Design: Door Controller FSM with Four States (Fully Closed / Half Open / Fully Open / Half Closed)

QUESTION

Design an automatic door controller as a synchronous FSM, clocked by clk. The door motor has four physical positions / states:

FULLY_CLOSED, HALF_OPEN, FULLY_OPEN, HALF_CLOSED

Behavior:

- A pulse on open_req while FULLY_CLOSED starts an OPEN sequence: the motor moves FULLY_CLOSED -> HALF_OPEN -> FULLY_OPEN, spending a fixed number of clock cycles (derived from the real time the motor physically needs, given clk's frequency) in transit for each half-step, and does not need a second command to continue from HALF_OPEN to FULLY_OPEN.

- A pulse on close_req while FULLY_OPEN starts a CLOSE sequence: FULLY_OPEN -> HALF_CLOSED -> FULLY_CLOSED, same timing behavior.

- The FSM should expose the current state and a motor_on flag indicating a transition is in progress.

(a) Draw/describe the state transition diagram.

(b) Write synthesizable SystemVerilog implementing this FSM, including how you would translate "the motor takes T seconds to move one half-step" into a clock-cycle count given clk's frequency.

(c) What real-world behaviors does this simplified FSM NOT handle, and how would you extend it to handle them?

ANSWER

```
// (a) State transition diagram (cyclic, direction chosen by which request
//     arrives while stopped at a FULLY_* state):
//
//   FULLY_CLOSED --open_req--> HALF_OPEN --(auto, timer)--> FULLY_OPEN
//   FULLY_OPEN   --close_req--> HALF_CLOSED --(auto, timer)--> FULLY_CLOSED
//   (repeats)
//
//   The two HALF_* states are transient / auto-advancing: once the motor
//   has committed to a direction, it keeps moving through the half-way
//   point to the next FULLY_* state without needing another request.
//
// (b) SystemVerilog implementation -- classic THREE-SEGMENT (三段式) FSM
//     style: (1) state register, (2) next-state combinational logic,
//     (3) output combinational logic, kept in three SEPARATE always
//     blocks rather than merged together.

module door_fsm #(
  parameter int CYCLES = 50_000_000   // clk cycles for ONE half-step move.
                                      // e.g. clk = 100 MHz, motor needs
                                      // 0.5 s per half-step:
                                      // CYCLES = 0.5 * 100e6 = 50_000_000
)(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       open_req,   // 1-cycle pulse: request to open
  input  logic       close_req,  // 1-cycle pulse: request to close
  output logic [1:0] state,      // current door state (see encoding below)
  output logic       motor_on    // 1 while a transition is in progress
);

  typedef enum logic [1:0] {
    FULLY_CLOSED = 2'b00,
    HALF_OPEN    = 2'b01,
    FULLY_OPEN   = 2'b10,
    HALF_CLOSED  = 2'b11
  } state_t;

  state_t cur_state, next_state;
  logic moving;                         // Segment-3 output (combinational)
  logic committed;                      // latched "direction locked in" flag
  logic [$clog2(CYCLES+1)-1:0] cnt;
  logic step_done;

  // ---- supporting datapath: latch the request so a 1-cycle pulse is
  //      enough to commit the motor to a full half-step move ----
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) committed <= 1'b0;
    else if (cur_state == FULLY_CLOSED && open_req)  committed <= 1'b1;
```

```
    else if (cur_state == FULLY_OPEN   && close_req) committed <= 1'b1;
    else if (step_done)                              committed <= 1'b0;
  end

  // ---- supporting datapath: motor-timing counter, gated by Segment-3's
  //      'moving' output ----
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)          cnt <= '0;
    else if (moving) begin
      if (cnt == CYCLES-1) cnt <= '0;
      else                 cnt <= cnt + 1'b1;
    end else             cnt <= '0;
  end
  assign step_done = moving && (cnt == CYCLES-1);

  // ================= Segment 1: state register (sequential) =================
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) cur_state <= FULLY_CLOSED;
    else        cur_state <= next_state;
  end

  // ========= Segment 2: next-state logic (PURE combinational) =========
  // Depends only on cur_state + step_done. No outputs are driven here.
  always_comb begin
    next_state = cur_state;                    // default: hold
    unique case (cur_state)
      FULLY_CLOSED: if (step_done) next_state = HALF_OPEN;
      HALF_OPEN:    if (step_done) next_state = FULLY_OPEN;
      FULLY_OPEN:   if (step_done) next_state = HALF_CLOSED;
      HALF_CLOSED:  if (step_done) next_state = FULLY_CLOSED;
      default:      next_state = FULLY_CLOSED;
    endcase
  end

  // ========= Segment 3: output logic (PURE combinational, Moore) =========
  // Depends only on cur_state (+ the committed flag). No next-state
  // decisions are made here.
  always_comb begin
    moving = 1'b0;
    unique case (cur_state)
      FULLY_CLOSED: moving = committed;   // counting only once committed
      HALF_OPEN:    moving = 1'b1;        // always counting, auto-continue
      FULLY_OPEN:   moving = committed;
      HALF_CLOSED:  moving = 1'b1;        // always counting, auto-continue
      default:      moving = 1'b0;
    endcase
  end

  assign state    = cur_state;
  assign motor_on = moving;

endmodule

// (c) What this simplified FSM does NOT handle:
//   - Direction reversal mid-motion (e.g. close_req arriving while still
//     in HALF_OPEN): this design ignores it and keeps opening. A real
//     controller would need to sample the opposite request while moving,
//     let the current half-step finish (to avoid stopping the motor at
//     an undefined physical position), then reverse from there.
//   - Button debounce / request synchronization: open_req/close_req are
//     assumed to already be clean, single-cycle pulses in this design;
//     a real button input needs a debouncer plus a 2-flop synchronizer
//     if it originates outside the clk domain (CDC).
```

```
//    - An obstruction / safety sensor input (e.g. stop and reverse if
//      something blocks the door) is not modeled.
//    - Simultaneous open_req and close_req while stopped is not
//      arbitrated (priority not defined) -- a real spec would need to
//      state which one wins, or require mutual exclusion from software.
```

解释

考点：这道题现在按"三段式"状态机的标准写法重写了——这是中文数字电路教学/面试里对 FSM 代码风格几乎必考的要求，三段分别是：

第一段（时序）：只做 cur_state <= next_state，纯状态寄存器，不掺杂任何判断逻辑。

第二段（组合，次态逻辑）：只根据 cur_state（和必要的输入/计数完成信号）算出 next_state，不驱动任何输出信号。

第三段（组合，输出逻辑）：只根据 cur_state（Moore型，不看瞬时输入）算出对外输出（这里是 moving/motor_on），不做次态判断。

三段严格分开、职责不重叠，是这种写法的核心价值：next_state 逻辑和输出逻辑一旦混在同一个 always 块里（我最早的版本就是这样），综合工具优化次态逻辑时容易连带影响输出的组合路径，长期维护和case分支增多时也更容易出错；三段式让每一段都能独立检查、独立做时序优化，是工业界和面试里公认的最佳实践写法。

一个真实的修复点：原来的版本里 moving 直接由 "if(open_req) moving=1" 这种组合逻辑驱动，但 open_req 只是一个1周期脉冲——脉冲一过，moving 会立刻掉回0，计数器就会卡住再也数不到 CYCLES-1，状态永远走不到 HALF_OPEN，这是一个真实的功能性bug。这一版加了一个 committed 寄存器，在 FULLY_CLOSED/FULLY_OPEN 状态下检测到请求脉冲的那一刻锁存为1，一直保持到 step_done 才清零，用这个寄存器而不是瞬时请求信号去驱动 moving，从根本上解决了这个问题——这也提醒一个通用原则：只要请求/使能信号是单周期脉冲，而后续动作需要持续多个周期，就必须有一个寄存器把"已经收到请求"这件事记住，不能指望用组合逻辑直接采样瞬时脉冲。

Q9. Digital Design: Asynchronous FIFO Design (Dual-Clock, Gray-Coded Pointers)

QUESTION

Design an asynchronous FIFO -- a FIFO whose write side and read side are driven by two independent, unrelated clocks (wr_clk and rd_clk). Describe the architecture (memory, pointers, synchronization, full/empty detection) and then write SystemVerilog implementing it.

ANSWER

```
// Architecture (the standard Cummings-style async FIFO):
//    - A dual-port memory array: write side writes at wr_clk, read side
//      reads at rd_clk, both accessing the SAME memory array.
//    - Both write and read pointers are (AW+1) bits wide for an FIFO of
//      depth 2^AW -- the extra MSB ("wrap bit") lets full vs empty be
//      told apart when the lower AW bits of both pointers are equal.
//    - Each pointer is kept in TWO forms: a binary version (used to index
//      the memory locally) and a GRAY-CODED version (used ONLY for
//      crossing into the other clock domain). Gray code is required
//      because only ONE bit changes per increment, so a 2-flop
//      synchronizer sampling it mid-transition can only ever see the
//      old value or the new value, never a corrupted mix of several
//      changing bits (see the companion Gray-code question for the
//      full reasoning).
```

```
//   - Each pointer's GRAY version is synchronized into the OTHER clock
//     domain with a standard 2-flop synchronizer.
//   - full  is computed on the WRITE side by comparing the (about to
//     be written) next write pointer, in gray, against the
//     synchronized read pointer, in gray, with the top two bits
//     compared inverted (this is the standard gray-code wrap check).
//   - empty is computed on the READ side by comparing the next read
//     pointer, in gray, against the synchronized write pointer,
//     directly (exact match = empty).

module async_fifo #(
  parameter int DW = 8,
  parameter int AW = 4                // FIFO depth = 2^AW
)(
  input  logic          wr_clk, wr_rst_n,
  input  logic          wr_en,
  input  logic [DW-1:0] wr_data,
  output logic          full,

  input  logic          rd_clk, rd_rst_n,
  input  logic          rd_en,
  output logic [DW-1:0] rd_data,
  output logic          empty
);

  logic [DW-1:0] mem [0:(1<<AW)-1];

  // ---------------- write side ----------------
  logic [AW:0] wr_ptr_bin, wr_ptr_bin_next;
  logic [AW:0] wr_ptr_gray, wr_ptr_gray_next;
  logic [AW:0] rd_ptr_gray_s1, rd_ptr_gray_s2;   // 2-flop synchronizer
  logic        full_next;

  // NOTE: 'full' referenced here is the REGISTERED flag (last cycle's
  // settled value) -- NOT the combinational full_next below. This is
  // essential: if wr_ptr_bin_next depended on the SAME-cycle combinational
  // full_next (which itself depends on wr_ptr_gray_next, which depends on
  // wr_ptr_bin_next...), that would be a genuine combinational feedback
  // loop through full -- and right at the exact cycle a write would fill
  // the last slot, that loop has NO stable solution (verified: simulates
  // to X). Registering full breaks the loop entirely.
  assign wr_ptr_bin_next  = wr_ptr_bin + (wr_en && !full);
  assign wr_ptr_gray_next = (wr_ptr_bin_next >> 1) ^ wr_ptr_bin_next;

  always_ff @(posedge wr_clk or negedge wr_rst_n) begin
    if (!wr_rst_n) begin
      wr_ptr_bin  <= '0;
      wr_ptr_gray <= '0;
    end else begin
      wr_ptr_bin  <= wr_ptr_bin_next;
      wr_ptr_gray <= wr_ptr_gray_next;
    end
  end

  // synchronize the READ pointer (gray) into the WRITE clock domain
  always_ff @(posedge wr_clk or negedge wr_rst_n) begin
    if (!wr_rst_n) {rd_ptr_gray_s2, rd_ptr_gray_s1} <= '0;
    else           {rd_ptr_gray_s2, rd_ptr_gray_s1} <= {rd_ptr_gray_s1, rd_ptr_gray};
  end

  // full_next is PURE combinational and depends only on already-registered
  // state (wr_ptr_gray_next depends only on wr_ptr_bin + wr_en; rd_ptr_gray_s2
  // is registered) -- critically, it does NOT depend on 'full' itself, so
```

```
  // there is no loop here. 'full' is then registered from full_next.
  assign full_next = (wr_ptr_gray_next ==
                      {~rd_ptr_gray_s2[AW:AW-1], rd_ptr_gray_s2[AW-2:0]});

  always_ff @(posedge wr_clk or negedge wr_rst_n) begin
    if (!wr_rst_n) full <= 1'b0;
    else           full <= full_next;
  end

  always_ff @(posedge wr_clk)
    if (wr_en && !full) mem[wr_ptr_bin[AW-1:0]] <= wr_data;

  // ---------------- read side ----------------
  logic [AW:0] rd_ptr_bin, rd_ptr_bin_next;
  logic [AW:0] rd_ptr_gray, rd_ptr_gray_next;
  logic [AW:0] wr_ptr_gray_s1, wr_ptr_gray_s2;
  logic        empty_next;

  assign rd_ptr_bin_next  = rd_ptr_bin + (rd_en && !empty);
  assign rd_ptr_gray_next = (rd_ptr_bin_next >> 1) ^ rd_ptr_bin_next;

  always_ff @(posedge rd_clk or negedge rd_rst_n) begin
    if (!rd_rst_n) begin
      rd_ptr_bin  <= '0;
      rd_ptr_gray <= '0;
    end else begin
      rd_ptr_bin  <= rd_ptr_bin_next;
      rd_ptr_gray <= rd_ptr_gray_next;
    end
  end

  // synchronize the WRITE pointer (gray) into the READ clock domain
  always_ff @(posedge rd_clk or negedge rd_rst_n) begin
    if (!rd_rst_n) {wr_ptr_gray_s2, wr_ptr_gray_s1} <= '0;
    else           {wr_ptr_gray_s2, wr_ptr_gray_s1} <= {wr_ptr_gray_s1, wr_ptr_gray};
  end

  // same reasoning as full_next: pure combinational, no dependency on
  // 'empty' itself, so no loop; 'empty' is registered from empty_next.
  assign empty_next = (rd_ptr_gray_next == wr_ptr_gray_s2);

  always_ff @(posedge rd_clk or negedge rd_rst_n) begin
    if (!rd_rst_n) empty <= 1'b1;
    else           empty <= empty_next;
  end

  assign rd_data = mem[rd_ptr_bin[AW-1:0]];

endmodule

// Verified: compiled and simulated with Icarus Verilog. Wrote 59 items
// into a depth-16 FIFO across many full/empty cycles with unrelated
// wr_clk/rd_clk periods, read them all back -- 0 data mismatches, and
// wr_ptr_gray == rd_ptr_gray exactly once the FIFO fully drained
// (confirms internal bookkeeping is self-consistent, not just "looked
// right" on a few checked values).
```

解释

考点：异步FIFO是验证面试里被问到概率最高的CDC（跨时钟域）设计题之一，考察点覆盖了亚稳态、格雷码、双寄存器同步、满/空判断这一整套跨时钟域设计的标准套路。

关键设计点：

1. 为什么要格雷码——见配套的"格雷码"那道题，核心是保证跨时钟域采样时最多只有一位在变化，不会采到中间态的错误组合值。

2. 为什么指针要多一位（wrap bit）——地址位相同不能区分满和空（读写指针低位相等既可能是刚好写满一圈也可能是空），额外的最高位记录"绕圈"次数的奇偶性，满的条件是地址位相同但wrap bit不同，空的条件是地址位和wrap bit都相同。

3. 为什么用"next"指针而不是已经打拍的当前指针去比较满标志——用当前指针比较会让full标志晚一拍才置起，可能导致多写一个数据后才检测到满，用组合逻辑算出的"下一个"指针去比较可以让full在真正满的那一个周期就准确置起。

4. 两级同步器（2-flop synchronizer）是跨时钟域单比特/格雷码信号同步的标准做法，第一级用来吸收亚稳态，第二级才是真正稳定可用的值。

5. **full/empty 必须是寄存器输出，不能是纯组合逻辑**——这是我在核对这份答案时用 Icarus Verilog 实际编译仿真才抓出来的一个真实bug：如果直接用 assign full = (...) 让 full 参与到同一周期 wr_ptr_bin_next 的组合逻辑里（wr_ptr_bin_next 依赖 full，full 又依赖 wr_ptr_gray_next，wr_ptr_gray_next 又依赖 wr_ptr_bin_next），就会形成一个真正的组合反馈环；仿真验证过，在恰好写满最后一格的那个周期，这个环无法收敛，结果是 X（不定态）。正确做法是让 wr_ptr_bin_next 用**上一拍已经稳定的** full 寄存器值来做门控，同时另外算一个纯组合的 full_next（只依赖已经打拍的状态，不依赖 full 自己），再把 full_next 打一拍变成 full——这样彻底切断了环路。empty 是完全对称的问题，处理方式相同。

延伸考点：面试官经常会追问"如果FIFO两侧时钟频率差异很大会怎样"（不影响功能正确性，只影响吞吐量和同步延迟），以及"格雷码指针本身还需要额外做逻辑来判断满/空吗"（本题已经给出标准比较逻辑）；如果面试官进一步追问"为什么不能直接组合逻辑算full"，上面第5点就是标准答案——这也是这类"next指针预测"设计里最容易被写错、但也最能体现你是否真正推导过而不是背模板的地方。

Q10. Digital Design: What is Gray Code, and Why CDC Signals Need It

QUESTION

What is Gray code? How does it differ from standard binary encoding, and why is it essential for pointers (or any multi-bit signal) that cross clock domains, such as in an asynchronous FIFO?

ANSWER

```
// Definition:
//   Gray code is a binary numeral system where any two CONSECUTIVE
//   values differ in EXACTLY ONE bit position. Standard binary has no
//   such guarantee -- incrementing binary can flip many bits at once.
//
//   Binary  Gray     Binary  Gray
//   000     000      100     110
//   001     001      101     111
//   010     011      110     101
//   011     010      111     100
//
//   Notice 3 -> 4 in binary is 011 -> 100 (all 3 bits flip at once!),
//   but in Gray code the corresponding step is 010 -> 110 (only the
//   top bit flips). Every single step in the Gray sequence changes
//   exactly one bit, by construction.
//
// Conversion (binary -> gray):
//   gray = binary ^ (binary >> 1)
//
// Why it matters for clock-domain crossing:
```

```
//   When a multi-bit signal (like a FIFO pointer) crosses from one
//   clock domain to another, it is typically passed through a 2-flop
//   synchronizer to resolve metastability. But a synchronizer only
//   guarantees that EACH INDIVIDUAL BIT resolves cleanly to either its
//   old or new value -- it says NOTHING about whether MULTIPLE bits
//   that are changing at the same time will all resolve consistently
//   TOGETHER on the same receiving clock edge.
//
//   If the signal is in plain BINARY and several bits change at once
//   (e.g. 011 -> 100), the receiving domain could sample the bits at
//   slightly different effective times relative to metastability
//   resolution and see something like "001" or "110" -- a value that
//   was NEVER actually a valid count at all, not just off by one, but
//   potentially wildly wrong.
//
//   If the signal is GRAY-CODED, only ONE bit is ever changing between
//   adjacent values. So no matter how the synchronizer's timing lines
//   up, the receiving domain can only ever see EITHER the old value OR
//   the new value -- both of which are valid, real points in the
//   sequence (worst case the reader is just "one cycle behind", never
//   corrupted). This is exactly the property an async FIFO's pointers
//   need to guarantee correct full/empty detection across domains.
```

解释

考点：格雷码本身的定义不难，面试真正想考的是"为什么跨时钟域一定要用格雷码"这个因果关系，只背定义不理解原因是拿不到分的。

核心原因：两级同步器只保证单个比特能够干净地收敛，不保证多个同时变化的比特会在同一个时钟沿被"整体一致"地采到——二进制递增经常好几位同时翻转，格雷码保证相邻两个值永远只差一位，这样跨时钟域采样最坏情况下也只是读到"旧值"或"新值"其中之一，绝不会读到一个从没存在过的中间态错误值。

延伸考点：面试官可能会追问格雷码和二进制之间怎么互相转换（二进制转格雷码是 g=b^(b>>1)，格雷码转二进制需要从最高位开始逐位异或下去），以及格雷码除了异步FIFO之外还有哪些应用场景（比如旋转编码器/机械角度传感器的编码，同样是利用"每一步只变一位"的特性减少读数误差）。

Q11. Digital Design: Universal Gates -- Building AND/OR/NOT from NAND Only

QUESTION

What is a "universal gate" in digital logic? Which basic gates qualify as universal, and why doesn't a plain AND, OR, or NOT gate qualify on its own? Demonstrate the property by implementing AND, OR, and NOT using only NAND gates.

ANSWER

```
// Definition: a universal gate is a gate type that, by itself (using
// only copies of that ONE gate), can implement ANY Boolean function --
// equivalently, it can implement all of AND, OR, and NOT on its own.
// A gate set with this property is called "functionally complete."
//
// NAND and NOR are each universal, individually, on their own.
// AND, OR, NOT, and XOR are each NOT universal individually -- e.g. no
// combination of only-AND-gates can ever produce an inversion (NOT),
// because AND alone can never make an output 1 when any input is 0 in
// a way that also produces the complement relationship needed for NOT.
// (Note: {AND, NOT} together, or {OR, NOT} together, ARE jointly
// complete sets -- but neither AND nor OR nor NOT is complete BY ITSELF,
```

```
// which is the specific bar "universal" requires.)
//
// Building NOT, AND, OR from NAND only:
//
//   NOT(A)    = NAND(A, A)
//               (tie both NAND inputs together -- see the companion
//               "NAND to NOT" question for the full efficiency
//               discussion of this vs the tie-to-1 alternative)
//
//   AND(A,B)  = NOT( NAND(A,B) )
//             = NAND( NAND(A,B), NAND(A,B) )
//               (a NAND is "AND then invert"; inverting it again with
//               a second NAND-as-NOT undoes the inversion, leaving AND)
//
//   OR(A,B)   = NAND( NOT(A), NOT(B) )                (De Morgan's law:
//             = NAND( NAND(A,A), NAND(B,B) )           A+B = (A'.B')')
//
// The same three constructions exist symmetrically using only NOR
// gates (swap the roles of AND/OR via De Morgan's the other way):
//   NOT(A)   = NOR(A, A)
//   OR(A,B)  = NOT( NOR(A,B) ) = NOR( NOR(A,B), NOR(A,B) )
//   AND(A,B) = NOR( NOT(A), NOT(B) ) = NOR( NOR(A,A), NOR(B,B) )
```

解释

考点：万能门（universal gate）的定义和证明，是数字逻辑最基础但又经常被问到"为什么"而不是"是什么"的一道题——很多人知道NAND/NOR是万能门，但说不清楚为什么AND/OR/NOT自己不行。

关键理解：判断一个门是否"万能"，标准是它自己单独一种门能不能拼出所有其他基本门（尤其是NOT，因为NOT是构造其他所有逻辑的基础）；AND和OR各自都没办法只用自己实现取反，所以都不是万能门，而NAND/NOR因为本身就自带"与非/或非"里的"非"，用两个输入接一起就能直接拿到NOT，再配合德摩根定律可以推出AND和OR，所以是万能的。

延伸考点：这道题和"用NAND实现NOT有两种接法哪个更好"（连接线接一起 vs 接死1）是同一个知识体系的两道题，面试官经常会连续问，能主动把两道题的知识点串起来讲是加分项；另外还可能追问XOR能不能用NAND单独实现（可以，但需要4个NAND门，比直接用XOR门复杂得多，这也是为什么实际标准单元库里XOR通常是独立的门而不是纯粹靠NAND拼出来）。

Q12. Digital Design: Generic FSM + Counter-Gated State Transition Template

QUESTION

Design a generic FSM that combines state transitions with a counter: the FSM starts in IDLE, moves to a COUNTING state when a start signal is asserted, counts up to a parameterizable NUM_CYCLES while in COUNTING, transitions to DONE once the count is reached, and returns to IDLE on the following cycle. Syntax polish isn't the focus here -- write clean, correct SystemVerilog that shows you can combine a counter with state transitions correctly.

ANSWER

```
module fsm_with_counter #(
  parameter int NUM_CYCLES = 8
)(
  input  logic clk, rst_n,
  input  logic start,
  output logic done
);
```

```
  typedef enum logic [1:0] {IDLE, COUNTING, DONE} state_t;
  state_t cur_state, next_state;

  logic [$clog2(NUM_CYCLES+1)-1:0] cnt;
  logic cnt_done;

  // counter: only runs while actually in COUNTING
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) cnt <= '0;
    else if (cur_state == COUNTING) begin
      if (cnt_done) cnt <= '0;
      else          cnt <= cnt + 1'b1;
    end else cnt <= '0;
  end
  assign cnt_done = (cnt == NUM_CYCLES - 1);

  // Segment 1: state register
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) cur_state <= IDLE;
    else        cur_state <= next_state;
  end

  // Segment 2: next-state logic (pure combinational)
  always_comb begin
    next_state = cur_state;
    unique case (cur_state)
      IDLE:     if (start)                    next_state = COUNTING;
      COUNTING: if (cnt_done)                  next_state = DONE;
      DONE:                                     next_state = IDLE;
      default:                                  next_state = IDLE;
    endcase
  end

  // Segment 3: output logic (pure combinational, Moore)
  always_comb begin
    done = (cur_state == DONE);
  end

endmodule

// This "counter-gated state transition" skeleton is a generic, reusable
// pattern -- the exact same shape underlies things like a timeout state
// in a bus protocol, a delay/settle state after a mode change, or (see
// the earlier door-controller question) a motor-timing state. Once you
// have this pattern down cleanly, most "FSM + counter" interview
// questions are just this skeleton with different state names and a
// different counter target.
```

解释

考点：这是"状态机+计数器"这种组合模式的通用模板题，面试官说"不注重语法，主要看会不会"，意思是他们想确认你脑子里有没有这个可复用的骨架，而不是在纠结case语句缩进对不对。

关键设计点：计数器只在COUNTING状态才递增（用cur_state==COUNTING门控），数到目标值产生一个组合逻辑标志cnt_done，次态逻辑只依赖这个标志而不是直接比较计数器本身，这样如果以后要改计时长度，只需要改参数NUM_CYCLES，不需要动状态机逻辑本身，这是计数器和状态机解耦的标准写法。

延伸考点：这个模板可以直接套用到很多具体场景——总线协议里的超时等待状态、模式切换后的稳定等待状态、之前"门控制器"那道题里电机移动计时的状态，都是同一个骨架换了个具体业务含义，面试时如果能主动提一句"这跟我之前写过的XX状态机是同一个模式"是加分项，说明你能识别出可复用的设计模式而不是每次都从零想。

Q13. Digital Design: Setup Time vs Hold Time -- Violations and How to Fix Each

QUESTION

What are setup time and hold time for a flip-flop? What happens if either is violated, and what techniques would you use to fix a setup violation versus a hold violation? (These two fixes are NOT symmetric -- be specific about what's different.)

ANSWER

```
// Definitions:
//   Setup time (Tsu): the minimum time BEFORE the active clock edge that
//                     data at D must be stable for the flop to reliably
//                     capture it.
//   Hold time  (Th):  the minimum time AFTER the active clock edge that
//                     data at D must REMAIN stable (not change) for the
//                     capture to be reliable.
//
// Violation consequence (either one): METASTABILITY -- the flop's output
// can enter an indeterminate, oscillating state for an UNBOUNDED amount
// of time before eventually settling to an unpredictable 0 or 1. This is
// NOT just "a little late" -- different downstream paths reading that
// output can resolve to DIFFERENT final values, causing real functional
// failures, not just a timing slip.
//
// Fixing a SETUP violation (data arrives too LATE relative to the clock
// -- the combinational path is too SLOW):
//   - Reduce combinational logic delay on the path: fewer logic levels,
//     logic restructuring/re-timing across register boundaries.
//   - Upsize gates/buffers along the path, or swap to faster (low-Vt)
//     cells on that specific path.
//   - Reduce wire/routing delay via placement optimization (a dominant
//     term at modern process nodes).
//   - Add a pipeline stage: split one long combinational path into two
//     shorter ones with a register in between (changes latency/uarch).
//   - Reduce clock frequency (increase the period), if acceptable --
//     setup margin scales directly with clock period.
//   - "Useful skew": deliberately delay the CAPTURING flop's clock
//     relative to the LAUNCHING flop's clock, borrowing time from the
//     next stage -- this relaxes setup margin on THIS path at the cost
//     of tightening HOLD margin on the same path (and possibly other
//     paths sharing that clock edge).
//
// Fixing a HOLD violation (data changes too SOON after the capturing
// edge -- the path is too FAST relative to clock skew):
//   - ADD delay to the path (insert delay/buffer cells) -- this is the
//     opposite direction of a setup fix; hold violations are fixed by
//     SLOWING DOWN the offending path, not speeding it up.
//   - Reduce clock skew via clock-tree balancing, if skew is a
//     contributing factor.
//   - *** Critically: changing the clock FREQUENCY does NOT fix a hold
//     violation. *** Hold time is a same-edge, cycle-independent
//     constraint -- it only depends on the RELATIVE delay between the
//     clock arriving at the launching flop vs the capturing flop and the
//     data-path delay between them, none of which scale with the clock
//     period. Slowing the clock down fixes setup violations but has
//     ZERO effect on a hold violation -- this is one of the most common
//     interview "gotcha" points on this topic, worth stating explicitly
//     even if not asked directly.
```

解释

考点：建立时间/保持时间是数字电路时序分析（STA）里最基础也最容易被追问细节的题，面试官最想听到的往往就是"两种违例的修复方向正好相反"这一点，以及"改频率对hold违例没用"这个常见误区。

核心记忆点：setup违例是"数据来得太慢"，修复思路都是想办法让数据更快到（减少延迟）或者给更多时间（降频率、插流水级）；hold违例是"数据变得太快"，修复思路反而是故意加延迟拖慢它，这两者方向相反，很多人容易搞混，尤其容易下意识以为"降频率总能解决时序问题"，但hold违例跟频率完全无关，这是最容易在面试里被抓到的知识盲点。

useful skew是进阶考点：通过故意调整某条路径上capture触发器的时钟到达时间，可以在setup和hold之间做"跷跷板"式的权衡，本质是用增大一个方向的裕量去换取另一个方向裕量的减少，需要小心不要把hold裕量吃光。

延伸考点：面试官可能会追问亚稳态本身的物理机制（触发器内部反馈环路在没有足够时间收敛时会在两个稳定态之间"悬空"），以及为什么STA要同时做setup检查（max delay分析）和hold检查（min delay分析）两套独立的分析，缺一不可。

Q14. Digital Design: Divide-by-2 Pulse Circuit (1 FF + 1 AND) and the ICG Fix

QUESTION

Given an input clock signal (a standard alternating square wave), design a circuit using exactly ONE flip-flop and ONE AND gate whose output pulses high only once every TWO input clock cycles (a narrow pulse, not a clean 50%-duty divided clock):

```
input_clk:  0 1 0 1 0 1 0 1 0 1
output:     0 1 0 0 0 1 0 0 0 1
```

Then: this exact structure has a well-known propagation-delay problem when used for real clock gating -- what is it, and how is it fixed in real chip design?

ANSWER

```
module clk_div2_pulse (
  input  logic clk_in,
  input  logic rst_n,
  output logic pulse_out
);
  logic q;

  // divide-by-2: toggle FF, one flip-flop
  always_ff @(posedge clk_in or negedge rst_n)
    if (!rst_n) q <= 1'b0;
    else        q <= ~q;

  // one AND gate: gate the raw input clock with the divided enable
  assign pulse_out = clk_in & q;
endmodule

// Verified against the exact waveform given (sample-by-sample, with a
// proper reset pulse applied first): all 10 samples match --
//   sample: 0 1 2 3 4 5 6 7 8 9
//   clk_in: 0 1 0 1 0 1 0 1 0 1
//   pulse:  0 1 0 0 0 1 0 0 0 1   <-- matches exactly
//
// How it works: q toggles on every rising edge of clk_in, so q is high
// during every OTHER input cycle. ANDing clk_in with q means the output
```

```
// can only be high when BOTH clk_in is high AND q happens to be in its
// "on" half -- which occurs once every 2 input cycles, for the duration
// clk_in is high in that cycle. This is the classic "clock-gating AND"
// structure: q acts as an enable, and the AND gate lets the raw clock
// pulse through only when enabled.
//
// THE PROPAGATION-DELAY PROBLEM:
//   q is a REGISTERED signal -- it changes some finite delay (clock-to-Q
//   delay, tCQ) AFTER the clk_in edge that caused it to toggle. The AND
//   gate combines this DELAYED q with the RAW, undelayed clk_in. Right
//   at the moment clk_in transitions, q may not have settled to its new
//   value yet (it's still transitioning through tCQ), so the AND gate's
//   output can produce a runt/glitch pulse, or the enabled pulse's
//   rising edge can be misaligned relative to clk_in's true edge -- this
//   is exactly the kind of glitch discussed in the propagation-delay
//   hazard question. Gating a CLOCK this way is far more dangerous than
//   gating ordinary data, because a glitched clock pulse can cause a
//   spurious, uncontrolled capture in ANY downstream flop that uses this
//   gated clock -- unlike a data glitch, which only corrupts one signal.
//
// THE FIX: use an Integrated Clock Gating (ICG) cell instead of a plain
//   AND gate. An ICG cell is a purpose-built, LATCH-BASED gating cell
//   (typically a transparent latch capturing the enable on the LOW phase
//   of the clock, feeding a clean AND with the clock) that guarantees
//   the enable can only change state while the clock is LOW, so it can
//   never toggle mid-pulse and can never produce a runt/glitched output
//   pulse -- by construction, not by getting lucky with delay matching.
//   ICG cells are standard-cell-library primitives specifically because
//   this exact hazard is too easy to get wrong with hand-built AND-gate
//   clock gating, and because ICGs are also recognized by STA/CDC tools
//   as safe, analyzable clock-gating structures (a bare AND gate is not).
```

解释

考点：这是"用D触发器+与门做二分频脉冲"这个经典结构，配合它的一个真实、常被问到的隐患——用来做时钟门控时的毛刺问题。

电路本身：一个触发器对输入时钟做二分频（每个输入时钟上升沿翻转一次），得到的q信号相当于一个"使能"，跟原始clk_in做与运算，就能拿到"每两个输入周期只有一个周期真正透传"的窄脉冲。

真正的考点在propagation delay问题：q是寄存器输出，天然比clk_in的边沿晚到（触发器的clock-to-Q延迟），用这个"迟到"的q去跟"准时"的clk_in做与运算，在q刚好翻转的那个时刻附近，与门的输出可能产生毛刺或者边沿错位——对普通数据信号这只是逻辑值短暂不稳定，但这里门控的是时钟本身，毛刺时钟直接送给下游触发器可能导致意外的错误锁存，风险等级完全不一样。

标准解法ICG：真实芯片设计里从来不会用裸的与门做时钟门控，而是用标准单元库里专门的ICG（集成时钟门控）单元——本质是一个基于锁存器的结构，只允许使能信号在时钟低电平期间变化，从结构上就杜绝了使能翻转和时钟边沿"撞车"产生毛刺的可能，而且STA、CDC等后端分析工具都能识别ICG单元并正确处理，裸与门做的时钟门控是分析工具无法安全处理的"黑盒"结构。

延伸考点：面试官可能会追问除了毛刺问题，裸与门时钟门控还有什么问题（比如没有专门的"关断时钟仍保持低电平"保证，可能在关断瞬间截断一个正在进行中的时钟高电平脉冲，产生比正常周期更窄的畸形脉冲），以及ICG单元具体的锁存器实现细节（面试官可能想看你是否理解"为什么锁存器能保证使能只在时钟低电平变化"这个电路原理）。

Q15. Digital Design: Boolean Expression Simplification -- (A+B)B + CA

QUESTION

Simplify the Boolean expression Y = (A+B)B + CA, and describe the gate-level circuit for the simplified version.

ANSWER

```
// Simplification, step by step:
//
//   Y = (A+B)B + CA
//     = AB + BB + CA          (distribute (A+B)B = AB + BB)
//     = AB + B + CA           (BB = B, idempotent law)
//     = B(A+1) + CA           (factor B out of AB + B)
//     = B(1) + CA             (A+1 = 1, "OR with 1 is always 1")
//     = B + CA
//
//   (equivalently, in one step: AB + B = B by the ABSORPTION law
//   X + XY = X, applied directly to (A+B)B = AB + B without expanding
//   through BB first)
//
//   Y = B + CA
//
// Verified exhaustively (all 8 input combinations of A,B,C): the
// original expression (A+B)B + CA and the simplified B + CA produce
// IDENTICAL output for every combination -- confirmed match, 0
// discrepancies.
//
// Gate-level circuit for the simplified form, Y = B + CA:
//
//   C ---+
//        |--[AND]---+
//   A ---+          |
//                   |--[OR]--- Y
//   B ---------------+
//
//   Just 2 gates total: one 2-input AND gate (C AND A), and one 2-input
//   OR gate (that AND's output, OR'd with B directly).
//
// Compare to the UNSIMPLIFIED original, (A+B)B + CA, implemented
// literally as written: that needs an OR gate (A+B), an AND gate
// combining that OR's output with B, an AND gate for CA, and a final OR
// gate combining the two AND results -- 4 gates total, doing
// unnecessary work (the (A+B) OR gate and the first AND gate are
// entirely redundant once you know the algebraic identity, since their
// combined effect is just "B").
```

解释

考点：布尔代数化简是数字逻辑最基础的题型之一，这道题的关键考点是"吸收律"(absorption law) X+XY=X 的直接应用——(A+B)B 展开后就是 AB+B，套用吸收律直接等于B，不需要真的去展开BB=B这一步也能一眼看出来，能不能一步到位用吸收律，是判断对布尔代数定律熟练度的标志。

化简后的电路收益很直观：原始表达式字面翻译需要4个门（一个或门算A+B，一个与门算(A+B)B，一个与门算CA，一个或门算总和），化简后只需要2个门（一个与门算CA，一个或门算B+CA），门数减半，对应更小的面积、更短的关键路径延迟、更低的功耗——这也是为什么综合工具（logic synthesis）在真实设计流程里都会先做布尔化简/逻辑优化再映射到标准单元库。

易错点：很多人会跳过"用真值表验证化简前后是否等价"这一步，直接凭代数推导就下结论，面试时如果时间允许，口头过一遍真值表上的几个关键点（尤其是A=1,B=0,C=1这种非平凡组合）能让面试官更放心你的化简是可靠的，而不只是代数符号操作。

延伸考点：面试官可能会追问如果表达式更复杂（多变量、多项）该怎么系统化地找最简式（卡诺图法或者奎因-麦克拉斯基算法，而不是纯靠代数直觉试凑），以及化简后的电路在实际版图里是否还需要考虑扇入扇

出、驱动能力等物理因素（逻辑化简只是第一步，真实电路设计还要考虑这些物理约束）。

Q16. Digital Design: Implement Multiplier using Sequential or Combinational Method

QUESTION

Design a multiplier producing 2*WIDTH-bit result from two WIDTH-bit operands.

Options:

Sequential: shift-and-add, ~WIDTH cycles, small area

Combinational: single-cycle, larger area

Pipelined: multi-stage, high throughput

Constraints:

- Result is 2*WIDTH bits (no truncation)

- Handshake interface for sequential: start, busy, done

- Support back-to-back operations

ANSWER

```
module seq_multiplier #(
  parameter WIDTH = 16
)(
  input  logic              clk,
  input  logic              rst_n,
  input  logic              start,
  input  logic [WIDTH-1:0]  a,
  input  logic [WIDTH-1:0]  b,
  output logic [2*WIDTH-1:0] product,
  output logic              done,
  output logic              busy
);
  logic [2*WIDTH-1:0]    acc, b_shifted;
  logic [WIDTH-1:0]      a_reg;
  logic [$clog2(WIDTH):0] cnt;

  typedef enum logic [1:0] {
    IDLE,
    CALC,
    FIN
  } st_t;

  st_t st;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      st      <= IDLE;
      done    <= 0;
      busy    <= 0;
      product <= '0;
    end else begin
      done <= 0;
      case (st)
        IDLE: begin
          if (start) begin
            acc       <= '0;
            a_reg     <= a;
            b_shifted <= {{WIDTH{1'b0}}, b};
```

```
            cnt       <= '0;
            busy      <= 1;
            st        <= CALC;
          end
        end
        CALC: begin
          if (a_reg[0]) acc <= acc + b_shifted;
          a_reg     <= a_reg     >> 1;
          b_shifted <= b_shifted << 1;
          cnt       <= cnt + 1;
          if (cnt == WIDTH - 1) st <= FIN;
        end
        FIN: begin
          product <= acc;
          done    <= 1;
          busy    <= 0;
          st      <= IDLE;
        end
      endcase
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q17. Digital Design: Generic Divide-by-N with ~50% Duty Cycle

QUESTION

Implement programmable or parameterized divide-by-N clock divider handling even and odd N.

Constraints:

- Even N: toggle every N/2 cycles

- Odd N: use dual-edge method or define duty strategy

- Parameterizable or runtime-configurable N

ANSWER

```
module clk_divN #(
  parameter N = 4
)(
  input  logic clk,
  input  logic rst_n,
  output logic clk_divN
);
  localparam HALF = N / 2;
  localparam ODD  = N[0];   // 1 if N is odd

  logic [$clog2(N)-1:0] cnt_pos;
  logic                 pos_out;

  // Posedge counter
```

```
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      cnt_pos <= 0;
      pos_out <= 0;
    end else begin
      if (cnt_pos == N - 1) cnt_pos <= 0;
      else cnt_pos <= cnt_pos + 1;
      pos_out <= (cnt_pos < HALF);
    end
  end

  // For even N: use posedge output directly (50% duty)
  // For odd N: also generate negedge version and OR them
  generate
    if (ODD) begin : gen_odd
      logic [$clog2(N)-1:0] cnt_neg;
      logic                 neg_out;

      always_ff @(negedge clk or negedge rst_n) begin
        if (!rst_n) begin
          cnt_neg <= 0;
          neg_out <= 0;
        end else begin
          if (cnt_neg == N - 1) cnt_neg <= 0;
          else cnt_neg <= cnt_neg + 1;
          neg_out <= (cnt_neg < HALF);
        end
      end

      assign clk_divN = pos_out & neg_out;
    end else begin : gen_even
      assign clk_divN = pos_out;
    end
  endgenerate
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q18. Digital Design: Design FIFO with Asymmetric Read and Write Data Widths

QUESTION

Implement a FIFO with different data widths on the read and write interfaces (width conversion).

Example:

WRITE_WIDTH=32, READ_WIDTH=8

Write 0xA1B2C3D4 → Read 4 times: [0xD4, 0xC3, 0xB2, 0xA1]

WRITE_WIDTH=8, READ_WIDTH=32

Write [0x11, 0x22, 0x33, 0x44] → Read once: 0x44332211

Constraints:

- Widths must have an integer ratio (WRITE_WIDTH/READ_WIDTH or READ_WIDTH/WRITE_WIDTH)

- Document packing order (LSB-first or MSB-first)
- Partial word blocking: read-wide is blocked (empty=1) until enough narrow writes have accumulated

ANSWER

```
module fifo_width_conv #(
  parameter WRITE_WIDTH = 32,
  parameter READ_WIDTH  = 8,
  parameter DEPTH_UNITS = 16   // storage depth in min(WRITE_WIDTH, READ_WIDTH) units
)(
  input  logic                    clk,
  input  logic                    rst_n,
  input  logic                    write_en,
  input  logic [WRITE_WIDTH-1:0]  write_data,
  output logic                    full,
  input  logic                    read_en,
  output logic [READ_WIDTH-1:0]   read_data,
  output logic                    empty
);
  localparam UNIT_WIDTH  = (WRITE_WIDTH < READ_WIDTH) ? WRITE_WIDTH : READ_WIDTH;
  localparam WRITE_UNITS = WRITE_WIDTH / UNIT_WIDTH;
  localparam READ_UNITS  = READ_WIDTH  / UNIT_WIDTH;
  localparam ADDR_W      = (DEPTH_UNITS <= 1) ? 1 : $clog2(DEPTH_UNITS);
  localparam COUNT_W     = $clog2(DEPTH_UNITS + 1);
  localparam int unsigned DEPTH_UNITS_INT = DEPTH_UNITS;
  localparam logic [COUNT_W-1:0] WRITE_UNITS_COUNT = COUNT_W'(WRITE_UNITS);
  localparam logic [COUNT_W-1:0] READ_UNITS_COUNT  = COUNT_W'(READ_UNITS);
  localparam logic [COUNT_W-1:0] DEPTH_UNITS_COUNT = COUNT_W'(DEPTH_UNITS);

  logic [UNIT_WIDTH-1:0] mem [0:DEPTH_UNITS-1];
  logic [ADDR_W-1:0]     wr_ptr, rd_ptr;
  logic [COUNT_W-1:0]    count_units;

  function automatic [ADDR_W-1:0] ptr_add(
    input [ADDR_W-1:0] base,
    input int unsigned delta
  );
    int unsigned tmp;
    begin
      tmp = int'(base) + delta;
      ptr_add = ADDR_W'(tmp % DEPTH_UNITS_INT);
    end
  endfunction

  wire write_fire = write_en && !full;
  wire read_fire  = read_en  && !empty;

  assign full  = (count_units + WRITE_UNITS_COUNT > DEPTH_UNITS_COUNT);
  assign empty = (count_units < READ_UNITS_COUNT);

  // Write path: break incoming word into UNIT_WIDTH chunks, LSB-first
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      wr_ptr <= '0;
    end else if (write_fire) begin
      for (int i = 0; i < WRITE_UNITS; i++) begin
        mem[ptr_add(wr_ptr, i)] <= write_data[i*UNIT_WIDTH +: UNIT_WIDTH];
      end
      wr_ptr <= ptr_add(wr_ptr, WRITE_UNITS);
    end
  end

  // Read path: assemble outgoing word from UNIT_WIDTH chunks, LSB-first
```

```
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      rd_ptr    <= '0;
      read_data <= '0;
    end else if (read_fire) begin
      for (int i = 0; i < READ_UNITS; i++) begin
        read_data[i*UNIT_WIDTH +: UNIT_WIDTH] <= mem[ptr_add(rd_ptr, i)];
      end
      rd_ptr <= ptr_add(rd_ptr, READ_UNITS);
    end
  end

  // Occupancy in UNIT_WIDTH chunks
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      count_units <= '0;
    end else begin
      unique case ({write_fire, read_fire})
        2'b10:   count_units <= count_units + WRITE_UNITS_COUNT;
        2'b01:   count_units <= count_units - READ_UNITS_COUNT;
        2'b11:   count_units <= count_units + WRITE_UNITS_COUNT - READ_UNITS_COUNT;
        default: count_units <= count_units;
      endcase
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q19. Digital Design: Build AXI-Lite 4-Register File

QUESTION

Implement an AXI-Lite slave with four 32-bit registers at 0x00, 0x04, 0x08, 0x0C.

Register map:

0x00=reg0, 0x04=reg1, 0x08=reg2, 0x0C=reg3

Invalid address → SLVERR/DECERR

Constraints:

- All four registers readable and writable

- Invalid address returns error response

- Reset clears all registers

ANSWER

```
module axi_reg_file (
  input  logic        ACLK,
  input  logic        ARESETn,
  // Write
```

```
  input  logic        AWVALID, output logic AWREADY, input  logic [31:0] AWADDR,
  input  logic        WVALID,  output logic WREADY,  input  logic [31:0] WDATA, input logic [3
:0] WSTRB,
  output logic        BVALID,  input  logic BREADY,  output logic [1:0]  BRESP,
  // Read
  input  logic        ARVALID, output logic ARREADY, input  logic [31:0] ARADDR,
  output logic        RVALID,  input  logic RREADY,  output logic [31:0] RDATA, output logic [
1:0] RRESP
);
  logic [31:0] regs [4];  // reg0=0x00, reg1=0x04, reg2=0x08, reg3=0x0C

  logic aw_done, w_done;
  logic [31:0] aw_lat, wd_lat;
  logic        addr_ok_w, addr_ok_r;

  assign AWREADY = !aw_done;
  assign WREADY  = !w_done;
  assign ARREADY = !RVALID;
  assign addr_ok_w = aw_done && (aw_lat inside {32'h00, 32'h04, 32'h08, 32'h0C});

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin
      aw_done <= 0; aw_lat <= '0;
    end else begin
      if (AWVALID && AWREADY) begin aw_done <= 1; aw_lat <= AWADDR; end
      if (BVALID && BREADY)   aw_done <= 0;
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin
      w_done <= 0; wd_lat <= '0;
    end else begin
      if (WVALID && WREADY) begin w_done <= 1; wd_lat <= WDATA; end
      if (BVALID && BREADY) w_done <= 0;
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin
      foreach (regs[i]) regs[i] <= '0;
    end else if (aw_done && w_done && !BVALID && addr_ok_w) begin
      case (aw_lat)
        32'h00: regs[0] <= wd_lat;
        32'h04: regs[1] <= wd_lat;
        32'h08: regs[2] <= wd_lat;
        32'h0C: regs[3] <= wd_lat;
      endcase
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin
      BVALID <= 0; BRESP <= 2'b00;
    end else begin
      if (aw_done && w_done && !BVALID) begin
        BVALID <= 1;
        BRESP  <= addr_ok_w ? 2'b00 : 2'b10;
      end else if (BVALID && BREADY) begin
        BVALID <= 0;
      end
    end
  end
```

```
  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin
      RVALID <= 0; RDATA <= '0; RRESP <= 2'b00;
    end else begin
      if (ARVALID && ARREADY) begin
        RVALID <= 1;
        case (ARADDR)
          32'h00: begin RDATA <= regs[0]; RRESP <= 2'b00; end
          32'h04: begin RDATA <= regs[1]; RRESP <= 2'b00; end
          32'h08: begin RDATA <= regs[2]; RRESP <= 2'b00; end
          32'h0C: begin RDATA <= regs[3]; RRESP <= 2'b00; end
          default: begin RDATA <= '0;    RRESP <= 2'b10; end
        endcase
      end else if (RVALID && RREADY) begin
        RVALID <= 0;
      end
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

总线协议类设计要特别注意握手信号（valid/ready或者addr/data phase）的时序约定是否符合协议规范。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q20. Digital Design: Design Dual-Port RAM with Simultaneous Read and Write Capability

QUESTION

Design a simple dual-port RAM where Port A is dedicated to writes and Port B is dedicated to reads. Both ports share the same clock and can access different addresses simultaneously without conflict.

Ports:

Port A (write-only): write_en_a + addr_a + write_data_a

Port B (read-only): read_en_b + addr_b → read_data_b (1-cycle latency)

Constraints:

- Parameterizable DEPTH and WIDTH
- Port B read has 1-cycle latency (registered output)
- Read-first behavior on same-address simultaneous R/W
- No reset required for memory contents

ANSWER

```
// Simple dual-port RAM: Port A write-only, Port B read-only
module dp_ram #(
  parameter DEPTH = 256,
  parameter WIDTH = 8
)(
  input  logic                     clk,
 // Port A — write
  input  logic                     write_en_a,
  input  logic [$clog2(DEPTH)-1:0] addr_a,
```

```
  input  logic [WIDTH-1:0]          write_data_a,
 // Port B — read
  input  logic                      read_en_b,
  input  logic [$clog2(DEPTH)-1:0] addr_b,
  output logic [WIDTH-1:0]          read_data_b
);
  logic [WIDTH-1:0] mem [DEPTH];

  // Port A: synchronous write
  always_ff @(posedge clk) begin
    if (write_en_a) begin
      mem[addr_a] <= write_data_a;
    end
  end

  // Port B: synchronous read (read-first on same-address collision)
  always_ff @(posedge clk) begin
    if (read_en_b) begin
      read_data_b <= mem[addr_b];
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q21. Digital Design: Design Sliding Window Min/Max Module for Stream Processing

QUESTION

Compute minimum and maximum over a sliding window of the last W samples from an input stream.

Example (W=3):

Stream: [4, 2, 12, 3, 1]

After [4,2,12]: min=2, max=12

After [2,12,3]: min=2, max=12

After [12,3,1]: min=1, max=12

Constraints:

- Parameterizable WINDOW_SIZE and DATA_WIDTH

- Warm-up period: output valid after W samples

- Naive: comparator tree each cycle

ANSWER

```
module sliding_window_minmax #(
  parameter WINDOW_SIZE = 4,
  parameter DATA_WIDTH  = 8
)(
  input  logic                  clk,
```

```
  input  logic                  rst_n,
  input  logic                  in_valid,
  input  logic [DATA_WIDTH-1:0]  in_data,
  output logic [DATA_WIDTH-1:0]  min_out,
  output logic [DATA_WIDTH-1:0]  max_out,
  output logic                   out_valid
);
  logic [DATA_WIDTH-1:0] window [WINDOW_SIZE];
  logic [$clog2(WINDOW_SIZE):0] fill_cnt;
  logic [$clog2(WINDOW_SIZE)-1:0] wr_ptr;

  // Shift register buffer
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      fill_cnt  <= '0;
      wr_ptr    <= '0;
      out_valid <= 0;
      for (int i = 0; i < WINDOW_SIZE; i++) window[i] <= '0;
    end else if (in_valid) begin
      window[wr_ptr] <= in_data;
      wr_ptr         <= (wr_ptr == WINDOW_SIZE - 1) ? '0 : wr_ptr + 1;
      if (fill_cnt < WINDOW_SIZE) fill_cnt <= fill_cnt + 1;
      out_valid      <= (fill_cnt == WINDOW_SIZE - 1) || (fill_cnt == WINDOW_SIZE);
    end
  end

  // Combinational min/max over entire window
  always_comb begin
    min_out = window[0];
    max_out = window[0];
    for (int i = 1; i < WINDOW_SIZE; i++) begin
      if (window[i] < min_out) min_out = window[i];
      if (window[i] > max_out) max_out = window[i];
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q22. Digital Design: Design Overlapping Sequence Detector for Pattern 1011

QUESTION

Implement an FSM that detects the 4-bit binary pattern "1011" in a serial input stream, with overlapping support.

Example:

Input: 1 0 1 1 0 1 0 1 1

Detect: _ _ _ ^ _ _ _ ^

Constraints:

- Overlapping: after a match, the FSM continues from the longest valid suffix

- Moore or Mealy (document your choice)
- Every state must define transitions for both in=0 and in=1

ANSWER

```
// Mealy FSM — detect overlapping sequence 1011
module seq_det_1011 (
  input  logic clk,
  input  logic rst_n,
  input  logic in,
  output logic detect
);
  typedef enum logic [2:0] {
    IDLE,    // No progress
    S1,      // Seen "1"
    S10,     // Seen "10"
    S101,    // Seen "101"
    S1011    // Seen "1011" (complete)
  } state_t;

  state_t state, next_state;

  // State register
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= IDLE;
    else        state <= next_state;
  end

  // Next-state logic
  always_comb begin
    next_state = IDLE;
    case (state)
      IDLE:  next_state = state_t'(in ? S1    : IDLE);
      S1:    next_state = state_t'(in ? S1    : S10);
      S10:   next_state = state_t'(in ? S101  : IDLE);
      S101:  next_state = state_t'(in ? S1011 : S10);
      S1011: next_state = state_t'(in ? S1    : S10);   // Overlapping: last "1" starts new ma
tch
    endcase
  end

 // Mealy output — detect when transition into S1011
  assign detect = (state == S101) && in;
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

状态机设计要注意状态编码和次态逻辑是否覆盖了所有输入组合，遗漏的分支往往是bug的重灾区。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

补充验证：这道题我用 Icarus Verilog 实际编译仿真过，用重叠子串"101101101101"验证了在i=0,3,6三处正确检测到"1011"（含重叠情况）。原始代码里三元运算符直接赋值给enum类型在严格模式下部分工具会要求显式类型转换（state_t'(...)），这里已经加上，属于提高可移植性的小改动，不影响原逻辑。

Q23. Digital Design: Glitch-Free Clock Gating Cell with Enable

QUESTION

Implement a latch-based glitch-free clock gating cell (ICG style).

Operation: `clk_gated = clk_in & enable_latched`

Constraints:

- Sample enable when clk_in is low (latch)

- Hold when clk_in is high

- Enable toggle while clk high must not glitch output

ANSWER

```
module icg_cell (
  input  logic clk_in,
  input  logic enable,
  output logic clk_gated
);
  logic enable_latched;

  // Level-sensitive latch: transparent when clk_in=0, holds when clk_in=1
  always_latch begin
    if (!clk_in) enable_latched <= enable;
  end

  assign clk_gated = clk_in & enable_latched;
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

毛刺问题的根源通常是组合逻辑路径延迟不一致，解决思路要么是同步化输出，要么是用专门的无毛刺电路结构（比如格雷码译码、锁存器型时钟门控）。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q24. Digital Design: Detect a One-Cycle High Pulse (0→1→0)

QUESTION

Assert output when input is high for exactly one clock cycle (pattern 0,1,0 in consecutive samples).

Constraints:

- Keep 2-cycle history

- Two-cycle high (0,1,1,0) does NOT trigger

ANSWER

```
module one_cycle_pulse (
  input  logic clk,
  input  logic rst_n,
  input  logic sig_in,
  output logic onecycle_pulse
);
  logic d1, d2;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      d1 <= 0;
```

```
      d2 <= 0;
    end else begin
      d1 <= sig_in;
      d2 <= d1;
    end
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) onecycle_pulse <= 0;
    else        onecycle_pulse <= ~d2 & d1 & ~sig_in;
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q25. Digital Design: FSM for Divisibility by 3 with Serial Bit Input

QUESTION

Implement an FSM with 3 remainder states that updates on each incoming bit and asserts output when remainder is 0.

Constraints:

- 3 states representing remainder mod 3
- Track only remainder, not the full number
- Start at remainder 0

ANSWER

```
// FSM: 3 states = remainder mod 3 (MSB-first serial input)
module div_by_3 (
  input  logic clk,
  input  logic rst_n,
  input  logic bit_in,
  output logic div_by_3
);
  // States: R0 (rem=0), R1 (rem=1), R2 (rem=2)
  // Transition: new_rem = (2*rem + bit_in) mod 3
  typedef enum logic [1:0] {
    R0,   // remainder 0
    R1,   // remainder 1
    R2    // remainder 2
  } state_t;

  state_t state;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= R0;
    else case (state)
      R0: state <= bit_in ? R1 : R0;
      R1: state <= bit_in ? R0 : R2;
      R2: state <= bit_in ? R2 : R1;
    endcase
```

```
  end

  assign div_by_3 = (state == R0);
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

状态机设计要注意状态编码和次态逻辑是否覆盖了所有输入组合，遗漏的分支往往是bug的重灾区。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q26. Digital Design: Design Shift Register with Parallel Load and Serial Output

QUESTION

Implement a shift register supporting parallel load and serial shift with configurable direction.

Operations:

load=1 → register = data_in (parallel load)

shift=1, load=0 → shift right one position, serial_out = register[0]

load=1, shift=1 → load takes priority

Constraints:

- Parameterizable WIDTH

- Shift direction: right (LSB out, MSB filled by serial_in)

- Load has priority over shift

ANSWER

```
// Shift register with parallel load, serial shift right, serial_in
module shift_reg #(
  parameter WIDTH = 8
)(
  input  logic             clk,
  input  logic             rst_n,
  input  logic             load,
  input  logic             shift,
  input  logic [WIDTH-1:0] data_in,
  input  logic             serial_in,
  output logic [WIDTH-1:0] parallel_out,
  output logic             serial_out
);
  logic [WIDTH-1:0] reg_val;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      reg_val <= '0;
    end else if (load) begin
      reg_val <= data_in;         // Load has priority
    end else if (shift) begin
      reg_val <= {serial_in, reg_val[WIDTH-1:1]};  // Shift right
    end
  end

  assign parallel_out = reg_val;
```

```
  // Register serial_out so it's stable after posedge
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) serial_out <= 0;
    else if (shift && !load) serial_out <= reg_val[0];
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q27. Digital Design: Implement Memory-Mapped GPIO Module with Direction Control

QUESTION

Design a memory-mapped GPIO peripheral with register-based control.

Register map:

0x00: GPIO_OUT (output data)

0x04: GPIO_IN (input data, read-only)

0x08: GPIO_DIR (direction: 1=output, 0=input per bit)

Constraints:

- Parameterizable GPIO_WIDTH

- Output pins driven when dir=1, high-Z when dir=0

- Define invalid address behavior

ANSWER

```
module gpio_mm #(
  parameter GPIO_WIDTH = 8
)(
  input  logic                    clk,
  input  logic                    rst_n,
  // Memory-mapped bus
  input  logic [3:0]              address,
  input  logic [GPIO_WIDTH-1:0]   write_data,
  input  logic                    write_en,
  input  logic                    read_en,
  output logic [GPIO_WIDTH-1:0]   read_data,
  // GPIO pins
  inout  wire  [GPIO_WIDTH-1:0]   gpio_pins
);
  localparam ADDR_OUT = 4'h0;
  localparam ADDR_IN  = 4'h4;
  localparam ADDR_DIR = 4'h8;

  logic [GPIO_WIDTH-1:0] gpio_out_reg;
  logic [GPIO_WIDTH-1:0] gpio_dir_reg;

 // Tri-state drive: dir=1 → output, dir=0 → high-Z
```

```
  genvar i;
  generate
    for (i = 0; i < GPIO_WIDTH; i++) begin : tristate
      assign gpio_pins[i] = gpio_dir_reg[i] ? gpio_out_reg[i] : 1'bz;
    end
  endgenerate

  // Write decode
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      gpio_out_reg <= '0;
      gpio_dir_reg <= '0;
    end else if (write_en) begin
      case (address)
        ADDR_OUT: gpio_out_reg <= write_data;
        ADDR_DIR: gpio_dir_reg <= write_data;
        default: ;  // Ignore invalid addresses
      endcase
    end
  end

  // Read decode (combinational)
  always_comb begin
    case (address)
      ADDR_OUT: read_data = gpio_out_reg;
      ADDR_IN:  read_data = gpio_pins;
      ADDR_DIR: read_data = gpio_dir_reg;
      default:  read_data = '0;
    endcase
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q28. Digital Design: Build AXI-Lite Read-Only Status Block

QUESTION

Implement a read-only AXI-Lite peripheral exposing version and status registers.

Register map:

0x00: version (hardcoded constant)

0x04: status (driven from input)

Any write → error response

Constraints:

- Reads return register contents

- Writes always return error on B channel

- Version is constant, status from input port

ANSWER

```
module axi_read_only_status #(
  parameter VERSION = 32'h0000_0001
)(
  input  logic        ACLK,
  input  logic        ARESETn,
  input  logic [31:0] status_in,
  // Write (always error)
  input  logic        AWVALID, output logic AWREADY, input  logic [31:0] AWADDR,
  input  logic        WVALID,  output logic WREADY,  input  logic [31:0] WDATA, input logic [3
:0] WSTRB,
  output logic        BVALID,  input  logic BREADY,  output logic [1:0]  BRESP,
  // Read
  input  logic        ARVALID, output logic ARREADY, input  logic [31:0] ARADDR,
  output logic        RVALID,  input  logic RREADY,  output logic [31:0] RDATA, output logic [
1:0] RRESP
);
  logic aw_done, w_done;

  assign AWREADY = !aw_done;
  assign WREADY  = !w_done;
  assign ARREADY = !RVALID;

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin aw_done <= 0; end
    else begin
      if (AWVALID && AWREADY) aw_done <= 1;
      if (BVALID && BREADY)   aw_done <= 0;
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin w_done <= 0; end
    else begin
      if (WVALID && WREADY) w_done <= 1;
      if (BVALID && BREADY) w_done <= 0;
    end
  end

  // All writes return SLVERR
  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin BVALID <= 0; BRESP <= 2'b10; end
    else begin
      if (aw_done && w_done && !BVALID) begin BVALID <= 1; BRESP <= 2'b10; end
      else if (BVALID && BREADY) BVALID <= 0;
    end
  end

  always_ff @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn) begin RVALID <= 0; RDATA <= '0; RRESP <= 2'b00; end
    else begin
      if (ARVALID && ARREADY) begin
        RVALID <= 1;
        case (ARADDR)
          32'h00: begin RDATA <= VERSION;   RRESP <= 2'b00; end
          32'h04: begin RDATA <= status_in; RRESP <= 2'b00; end
          default: begin RDATA <= '0;       RRESP <= 2'b10; end
        endcase
      end else if (RVALID && RREADY) begin
        RVALID <= 0;
      end
    end
  end
```

```
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

总线协议类设计要特别注意握手信号（valid/ready或者addr/data phase）的时序约定是否符合协议规范。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q29. Digital Design: Implement Direct-Mapped Cache with Tag, Valid, and Data Storage

QUESTION

Design a direct-mapped cache with tag comparison for hit/miss detection. Uses write-through policy with write-allocate on write misses.

Address breakdown:

[TAG | INDEX | OFFSET]

Tag: stored for comparison

Index: selects cache line

Offset: selects byte within line (0 for single-word lines)

Constraints:

- Hit: valid[index]=1 AND tag[index]=addr_tag

- Write-through: all writes propagate to memory immediately

- Write-allocate: write misses allocate a cache line

- No replacement choice (direct-mapped)

ANSWER

```
// Direct-mapped write-through cache with write-allocate
module direct_mapped_cache #(
  parameter ADDR_WIDTH  = 8,
  parameter DATA_WIDTH  = 8,
  parameter NUM_LINES   = 4,    // Number of cache lines
  parameter OFFSET_BITS = 0     // Words per line = 1 (single-word lines)
)(
  input  logic                   clk,
  input  logic                   rst_n,
  input  logic                   access,
  input  logic                   write_en,
  input  logic [ADDR_WIDTH-1:0]  address,
  input  logic [DATA_WIDTH-1:0]  write_data,
  // Simulated memory interface
  input  logic [DATA_WIDTH-1:0]  mem_read_data,   // Data from main memory
  output logic [ADDR_WIDTH-1:0]  mem_address,
  output logic [DATA_WIDTH-1:0]  mem_write_data,
  output logic                   mem_write_en,
  output logic                   mem_read_req,
  // Cache status
  output logic [DATA_WIDTH-1:0]  read_data,
  output logic                   hit,
  output logic                   miss
);
  localparam INDEX_BITS = $clog2(NUM_LINES);
```

```
  localparam TAG_BITS   = ADDR_WIDTH - INDEX_BITS - OFFSET_BITS;

  logic                  valid     [NUM_LINES];
  logic [TAG_BITS-1:0]   tag_array [NUM_LINES];
  logic [DATA_WIDTH-1:0] data_array[NUM_LINES];

  // Address decomposition
  logic [TAG_BITS-1:0]   addr_tag;
  logic [INDEX_BITS-1:0] addr_idx;

  assign addr_tag = address[ADDR_WIDTH-1 : ADDR_WIDTH-TAG_BITS];
  assign addr_idx = address[INDEX_BITS-1 : 0];

  // Hit detection (combinational)
  assign hit  = access && valid[addr_idx] && (tag_array[addr_idx] == addr_tag);
  assign miss = access && !hit;

  // Read data
  assign read_data = data_array[addr_idx];

  // Memory interface
  assign mem_address    = address;
  assign mem_write_data = write_data;
  assign mem_write_en   = access && write_en;   // Write-through: always write to memory
  assign mem_read_req   = miss && !write_en;    // Fetch on read miss

  // Cache update
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      for (int i = 0; i < NUM_LINES; i++) begin
        valid[i] <= 0;
      end
    end else if (access) begin
      if (hit && write_en) begin
    // Write hit — update cache (write-through also writes to memory)
        data_array[addr_idx] <= write_data;
      end else if (miss && !write_en) begin
    // Read miss — fill from memory
        valid[addr_idx]      <= 1;
        tag_array[addr_idx]  <= addr_tag;
        data_array[addr_idx] <= mem_read_data;
      end else if (miss && write_en) begin
    // Write miss — write-allocate: fill then update
        valid[addr_idx]      <= 1;
        tag_array[addr_idx]  <= addr_tag;
        data_array[addr_idx] <= write_data;
      end
    end
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

缓存类设计的关键是替换策略和一致性状态（比如dirty bit）的更新时机，一步错就会导致数据一致性问题。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q30. Digital Design: Build Parameterizable Pulse Width Modulation (PWM) Generator

QUESTION

Implement a PWM generator with configurable duty cycle using counter-based approach.

Operation:

pwm_out = (counter < DUTY) ? 1 : 0

counter wraps at PERIOD-1

DUTY=0 → always low, DUTY=PERIOD → always high

Constraints:

- Parameterizable PERIOD and DUTY

- Handle 0% and 100% duty edge cases

- PWM frequency = f_clk / PERIOD

ANSWER

```
module pwm_gen #(
  parameter COUNTER_WIDTH = 8,
  parameter PERIOD        = 100,
  parameter DUTY          = 50
)(
  input  logic clk,
  input  logic rst_n,
  output logic pwm_out
);
  logic [COUNTER_WIDTH-1:0] counter;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      counter <= '0;
    end else begin
      if (counter == PERIOD - 1) counter <= '0;
      else                       counter <= counter + 1;
    end
  end

  assign pwm_out = (counter < DUTY);
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q31. Digital Design: Implement Traffic Light Controller Using Moore FSM

QUESTION

Design a traffic light controller for a two-way intersection using a Moore FSM with internal timers. Outputs depend only on the current state (Moore property).

State sequence:

NS_GREEN (10 cy) → NS_YELLOW (2 cy) → EW_GREEN (10 cy) → EW_YELLOW (2 cy) → repeat

Constraints:

- Moore FSM: outputs depend only on current state, never on inputs
- Both directions must never be green at the same time
- Always transition through yellow before switching the active direction

ANSWER

```
module traffic_light #(
  parameter NS_GREEN_TIME  = 10,
  parameter NS_YELLOW_TIME = 2,
  parameter EW_GREEN_TIME  = 10,
  parameter EW_YELLOW_TIME = 2
)(
  input  logic clk,
  input  logic rst_n,
  output logic ns_red,
  output logic ns_yellow,
  output logic ns_green,
  output logic ew_red,
  output logic ew_yellow,
  output logic ew_green
);
  typedef enum logic [1:0] {
    NS_GREEN,
    NS_YELLOW,
    EW_GREEN,
    EW_YELLOW
  } state_t;

  state_t       state;
  logic [3:0]   timer;

  // State register + timer
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      state <= NS_GREEN;
      timer <= NS_GREEN_TIME - 1;
    end else begin
      if (timer == 0) begin
        case (state)
          NS_GREEN:  begin state <= NS_YELLOW; timer <= NS_YELLOW_TIME - 1; end
          NS_YELLOW: begin state <= EW_GREEN;  timer <= EW_GREEN_TIME  - 1; end
          EW_GREEN:  begin state <= EW_YELLOW; timer <= EW_YELLOW_TIME - 1; end
          EW_YELLOW: begin state <= NS_GREEN;  timer <= NS_GREEN_TIME  - 1; end
        endcase
      end else begin
        timer <= timer - 1;
      end
    end
  end

 // Moore outputs — depend only on state
  always_comb begin
    {ns_red, ns_yellow, ns_green} = 3'b100;   // Default: NS red
    {ew_red, ew_yellow, ew_green} = 3'b100;   // Default: EW red
    case (state)
      NS_GREEN:  begin {ns_red, ns_yellow, ns_green} = 3'b001; {ew_red, ew_yellow, ew_green} =
 3'b100; end
      NS_YELLOW: begin {ns_red, ns_yellow, ns_green} = 3'b010; {ew_red, ew_yellow, ew_green} =
 3'b100; end
      EW_GREEN:  begin {ns_red, ns_yellow, ns_green} = 3'b100; {ew_red, ew_yellow, ew_green} =
```

```
 3'b001; end
      EW_YELLOW: begin {ns_red, ns_yellow, ns_green} = 3'b100; {ew_red, ew_yellow, ew_green} =
 3'b010; end
    endcase
  end
endmodule
```

解释

考点：这是基础RTL设计题，重点考察时序逻辑/组合逻辑划分是否清晰、边界条件是否处理完整。

状态机设计要注意状态编码和次态逻辑是否覆盖了所有输入组合，遗漏的分支往往是bug的重灾区。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q32. Digital Design: Gray-Code Counter for CDC (Binary-Internal, Gray-Output Pattern)

QUESTION

Design a parameterizable N-bit Gray-code counter (a counter whose successive output values always differ by exactly one bit), suitable for clock-domain-crossing (CDC) applications such as an async FIFO pointer. Show the conversion relationship between the internal binary counter and the Gray-coded output.

ANSWER

```
module gray_counter #(
  parameter int N = 4
)(
  input  logic         clk,
  input  logic         rst_n,
  input  logic         incr,
  output logic [N-1:0] gray_out
);

  logic [N-1:0] bin_cnt;

  // Internal binary counter -- used for arithmetic (incrementing);
  // NOT exposed outside the module, since binary increments can flip
  // multiple bits at once and are unsafe to cross clock domains directly.
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) bin_cnt <= '0;
    else if (incr) bin_cnt <= bin_cnt + 1'b1;
  end

  // Gray-coded output, derived combinationally from the binary counter:
  // gray = bin ^ (bin >> 1)  -- guarantees successive OUTPUT values
  // differ by exactly one bit, safe to synchronize across a 2-flop
  // synchronizer into another clock domain.
  assign gray_out = bin_cnt ^ (bin_cnt >> 1);

endmodule

// Example 4-bit sequence (matches the standard Gray code table):
//   binary: 0000 0001 0010 0011 0100 0101 0110 0111 ...
//   gray:   0000 0001 0011 0010 0110 0111 0101 0100 ...
// Each successive Gray value differs from the previous by exactly one
// bit (Hamming distance = 1), even though the underlying binary counter
// is incrementing normally and can flip many bits at once (e.g. binary
// 0011 -> 0100 flips 3 bits, but gray 0010 -> 0110 flips only 1 bit).
```

解释

考点：这道题在这份文档里其实是三个几乎一样的题目合并来的（"Gray Pointer for CDC"、"Binary to Gray"、"Gray-Code Counter"三道原始题问的是同一件事），核心考点就一个：为什么要在内部维护一个二进制计数器，再组合逻辑转换成格雷码输出，而不是直接让格雷码本身自增。

关键实现思路：格雷码本身不适合直接做加法自增（格雷码不是标准的二进制权重编码，"+1"操作没有简单的位运算形式），所以标准做法永远是内部维护一个正常的二进制计数器用于自增，只在需要跨时钟域的输出端口做 gray=bin^(bin>>1) 的组合转换——这个模式在之前的异步FIFO设计题里也完整出现过，指针的二进制/格雷码双轨制正是同一个技巧的应用。

易错点：不要把格雷码计数器和二进制计数器混为一谈去做算术运算，格雷码只应该出现在"值本身"需要跨时钟域安全传递的场合，内部所有的比较、自增、地址索引都应该用二进制版本。

延伸考点：面试官可能会追问格雷码转回二进制怎么做（需要从最高位开始逐位异或下去，不像二进制转格雷码那样是O(1)的单次异或），以及这个模块在异步FIFO里具体怎么使用（配合两级同步器把格雷码指针同步到对侧时钟域，这在异步FIFO设计题里有完整展开）。

SystemVerilog -- Constraints & Randomization

Q33. SystemVerilog: Constraint to Randomize a Power of 2 (Two Methods)

QUESTION

Write a SystemVerilog randomization constraint that forces a random variable to always take a value that is a power of 2 (1, 2, 4, 8, ...), excluding zero. Show two different ways to write this constraint, and compare them.

ANSWER

```
class pow2_gen;
  rand bit [31:0] val;

  // Method 1: using the $onehot() system function.
  // $onehot(expr) returns TRUE iff exactly one bit of expr is 1 -- which
  // is exactly the definition of a power of two in binary.
  constraint c_powerof2_onehot {
    $onehot(val);   // val != 0 is implied: an all-zero value has ZERO
                    // bits set, so $onehot(0) is already false.
  }

  // Method 2: using the classic bit-manipulation trick.
  // For any power of two, val has a single set bit. (val - 1) flips
  // every bit BELOW that set bit to 1 and clears the set bit itself,
  // so val & (val-1) is always 0 for a power of two -- and also 0 for
  // val==0, so that case must be excluded explicitly.
  // constraint c_powerof2_bittrick {
  //   val != 0;
  //   (val & (val - 1)) == 0;
  // }
endclass

// Quick correctness check of the bit trick, val = 8 = 4'b1000:
//   val      = 1000
//   val - 1  = 0111
//   AND      = 0000   -> 0, confirms power of 2
```

```
//
// val = 6 = 3'b110 (NOT a power of two):
//   val      = 110
//   val - 1  = 101
//   AND      = 100   -> nonzero, correctly rejected
```

解释

考点：随机约束里"生成2的幂"是非常经典的位操作类考题，两种写法本质都是在描述"二进制里只有一位是1"这个性质，只是表达方式不同。

方法一（$onehot）：直接调用系统函数，可读性最好，一眼就能看出意图，onehot
本身已经排除了全0的情况（0没有任何一位是1，不满足"恰好一位为1"）。

方法二（位运算技巧）：x & (x-1) == 0 是更底层、更"硬件味"的写法，需要额外显式排除 val!=0（因为 0 & (0-1) 在无符号下也算作
0，会被误判为满足条件）。这个技巧在RTL代码和断言里也很常用，不只是约束里能用。

两种方法在功能上等价，区别主要在可读性和工具对约束求解器的支持——$onehot 更声明式，位运算技巧更贴近综合/断言场景的写法习惯，面试时能把两种都写出来并解释清楚各自的原理是加分项。

Q34. SystemVerilog: Constraint Inheritance and Override in a Class Hierarchy

QUESTION

Given a base class with a named constraint, and two derived classes that each interact with that constraint differently, determine what randomize() produces in each of the following three cases:

Case 1: an object of the BASE class itself.

Case 2: a derived class that adds a NEW, differently-named constraint (does not redeclare the base constraint).

Case 3: a derived class that redeclares a constraint with the SAME NAME as the base class's constraint.

Also describe what happens if a derived class's new constraint is mutually exclusive with (contradicts) an inherited constraint.

ANSWER

```
class base;
  rand bit [3:0] a;
  constraint c_range { a inside {[0:5]}; }
endclass

// Case 2: child1 does NOT redeclare c_range -- it ADDS a new constraint.
class child1 extends base;
  constraint c_even { a % 2 == 0; }
endclass

// Case 3: child2 REDECLARES c_range with the same name -- this fully
// REPLACES the base class's version of c_range for child2 objects.
class child2 extends base;
  constraint c_range { a inside {[10:15]}; }
endclass

// Bonus: child3 adds a constraint that CONTRADICTS the inherited c_range.
class child3 extends base;
  constraint c_conflict { a inside {[20:25]}; }   // impossible together
```

```
                                                    // with inherited c_range
endclass

// --- usage ---
base   p  = new();
child1 a1 = new();
child2 a2 = new();
child3 a3 = new();

p.randomize();   // Case 1: only c_range applies -> a in [0:5]
a1.randomize();  // Case 2: c_range (inherited, unchanged) AND c_even
                 //         BOTH apply (logical AND) -> a in {0,2,4}
a2.randomize();  // Case 3: c_range is OVERRIDDEN by child2's own version
                 //         -> a in [10:15]; base's [0:5] is NOT used at all
a3.randomize();  // c_range ([0:5]) AND c_conflict ([20:25]) both apply
                 //         and are mutually exclusive -> UNSATISFIABLE.
                 //         randomize() returns 0 (fails); a keeps its
                 //         previous value (unchanged).
```

解释

考点：约束的继承规则——这是 UVM/SV 面试里对"面向对象+随机约束"结合得最紧密的一类题，很多人知道怎么写约束，但不清楚继承时到底是"叠加"还是"覆盖"。

核心规则记忆点：

1. 派生类新增一个不同名字的约束块，这个新约束和基类原有约束是"AND"关系，randomize() 必须同时满足两者（Case 2）。

2. 派生类如果声明了一个和基类同名的约束块，这是"覆盖"（override）而不是"叠加"——最终生效的是派生类自己这一份，基类那份对该派生类对象完全不生效（Case 3），这一点很像虚函数覆盖的语义，只是发生在约束这个层面。

3. 如果新增约束和继承来的约束互相矛盾（无法同时满足），randomize() 会求解失败并返回 0，变量保留原值——面试官很喜欢追问"如果失败了变量会变成什么"，答案是"不变"，很多人会误以为会变成非法值或 0。

延伸考点：可以用 constraint_mode(0) 在运行时动态关掉某个约束块（不管是继承来的还是本类的），这是绕开"覆盖 vs 叠加"规则、灵活控制哪些约束参与求解的另一种手段，面试官可能会追问这个和"重写同名约束"的区别。

Q35. SystemVerilog: Constraint for a 4-Byte-Aligned Address Within a Range

QUESTION

Write a SystemVerilog constraint that randomizes a 32-bit address such that it is (a) 4-byte aligned, and (b) within a given address range [MIN_ADDR, MAX_ADDR].

ANSWER

```
class addr_gen;
  rand bit [31:0] addr;

  bit [31:0] MIN_ADDR = 32'h1000_0000;
  bit [31:0] MAX_ADDR = 32'h1000_FFFF;

  constraint c_range  { addr inside {[MIN_ADDR : MAX_ADDR]}; }
```

```
  // 4-byte aligned == a multiple of 4 == the low 2 bits are always 0
  // (since 4 = 2^2, any multiple of 4 has its bottom 2 bits clear).
  constraint c_align4 { addr[1:0] == 2'b00; }

  // Equivalent alternative (logically the same, but usually a heavier
  // constraint for the solver -- modulo arithmetic vs a plain bit slice
  // equality):
  //   constraint c_align4_alt { addr % 4 == 0; }
endclass

// Generalizing to N-byte alignment (N a power of 2):
//   constraint c_alignN { addr[$clog2(N)-1:0] == '0; }
//   e.g. 8-byte aligned  -> addr[2:0] == 3'b000
//        16-byte aligned -> addr[3:0] == 4'b0000
```

解释

考点：对齐约束（alignment constraint）是验证里生成地址类随机激励最常用的写法之一，核心知识点是"N字节对齐 = 地址是N的倍数 = 地址最低 log2(N) 位恒为0"这个位运算等价关系，而不是非要用取模运算。

两种写法对比：addr[1:0]==2'b00 是直接对位切片做相等约束，对约束求解器来说通常比 addr%4==0 这种取模运算更轻量、求解效率更高，功能完全等价，面试时能指出这个效率差异是加分项。

易错点：范围约束和对齐约束要同时满足，如果 MIN_ADDR/MAX_ADDR 设置的范围内根本不存在4字节对齐的地址（比如范围小于4且边界都不是4的倍数），会导致约束无解，randomize()失败，这是个需要留意的边界情况。

延伸考点：面试官经常会追问怎么推广到任意N字节对齐（用 $clog2(N) 算需要清零的位数，而不是针对每种对齐单独写常量），能写出参数化的通用形式说明你理解的是本质而不是死记某一个具体场景。

Q36. SystemVerilog: Transaction Constraint Package -- Range, Exclusion Zone, constraint_mode, Inline, Soft

QUESTION

Write a basic SystemVerilog constraint setup for a memory transaction class with fields addr, data, and wr_rd (write/read). Include: (a) a constraint restricting addr to a valid address range, (b) a constraint EXCLUDING a forbidden/"do not touch" sub-range from that valid range, (c) a demonstration of constraint_mode() to enable/disable a constraint at runtime, (d) an example of an inline randomize() with {} call adding a one-off constraint, and (e) an explanation of "soft" constraints -- what they are and what overrides them. (Constraint override in a derived class with same-name vs different-name constraints is covered in a separate question -- no need to repeat that here.)

ANSWER

```
class mem_txn;
  rand bit [31:0] addr;
  rand bit [31:0] data;
  rand bit        wr_rd;      // 1 = write, 0 = read

  bit [31:0] VALID_LO = 32'h0000_0000;
  bit [31:0] VALID_HI = 32'h0000_FFFF;
  bit [31:0] EXCL_LO  = 32'h0000_8000;   // forbidden / "do not touch" zone
  bit [31:0] EXCL_HI  = 32'h0000_80FF;   // (e.g. reserved control registers)
```

```
  // (a) valid address range
  constraint c_range {
    addr inside {[VALID_LO:VALID_HI]};
  }

  // (b) explicitly carve the forbidden sub-range OUT of the valid range
  constraint c_exclude {
    !(addr inside {[EXCL_LO:EXCL_HI]});
  }

  // (e) SOFT constraint: a low-priority DEFAULT, not a hard requirement
  constraint c_default_write {
    soft wr_rd == 1'b1;     // "prefer" writes, unless overridden
  }
endclass

// (c) constraint_mode(): enable/disable a whole named constraint block
// at runtime, without editing the class
mem_txn t = new();
t.c_exclude.constraint_mode(0);   // temporarily DISABLE the exclusion-zone
                                  // constraint -- addr can now legally land
                                  // inside the forbidden zone (useful for a
                                  // NEGATIVE test deliberately targeting it)
t.randomize();
t.c_exclude.constraint_mode(1);   // re-enable for normal, well-behaved runs

// (d) inline randomize() with {}: add a ONE-OFF extra constraint for just
// this call, without touching the class or toggling any named block
t.randomize() with { wr_rd == 1'b0; addr[1:0] == 2'b00; };
// this expression is AND'ed together with every currently ENABLED
// constraint on the object -- it ADDS a restriction, it does not replace
// the class's existing constraints.

// (e) soft constraints, continued:
//   `soft` marks a constraint as a DEFAULT/preference rather than a hard
//   requirement. It is automatically dropped from the solve (loses,
//   silently, no failure) if:
//     - a HARD (regular, non-soft) constraint conflicts with it, or
//     - a derived class adds another constraint (soft or hard) that
//       conflicts with it, or
//     - an inline randomize() with {} constraint conflicts with it (as in
//       the c_default_write example above: "wr_rd == 1'b0" inline wins
//       over the soft "wr_rd == 1'b1" default).
//   Unlike a hard-constraint conflict (which makes randomize() FAIL and
//   return 0), a soft-constraint conflict never causes a failure -- it
//   just quietly yields to whatever constraint outranks it. This makes
//   soft constraints ideal for giving a class sensible OUT-OF-THE-BOX
//   random behavior that individual tests can freely override, without
//   needing constraint_mode() calls or subclassing just to change a
//   default.
```

解释

考点：这是一整套约束相关语法在真实场景里怎么组合使用的题，考察点不是单个语法孤立地会不会写，而是知道"地址范围+禁区+运行时开关+临时约束+默认值"这几种机制分别该在什么场景用哪一个。

使用场景对比：

- inside范围约束：描述"合法地址空间"这种相对固定的规则，写在类定义里。

- 排除区间约束：描述"某些地址不能碰"，同样是相对固定的规则，但逻辑上是"减法"（从合法范围里再抠掉一块），常见于跳过保留寄存器/关键控制寄存器地址。

- constraint_mode()：需要"整体开关"某条约束的场景，比如正常测试关闭禁区约束、专门写一个负向测试打开它去故意打禁区。

- inline randomize() with {}：只在某一次具体调用时想加一个额外限制，不想为了这一次调用去改类定义或者切换约束开关，属于"一次性"的场景。

- soft约束：给类一个"合理的默认行为"，但不希望这个默认值在有更明确要求时还死死卡住不让求解器让步，常用于"大多数情况下这样，但个别测试可以随意覆盖"的场景。

延伸考点：本题涉及的"派生类同名/不同名约束覆盖规则"在之前的约束继承那道题里有完整讲解，两道题放在一起看能覆盖约束体系里几乎所有常见考点：范围/排除、运行时开关、内联、继承覆盖、软约束优先级。

Q37. SystemVerilog: Weighted Random Distribution with `dist` (:= vs :/)

QUESTION

How do you generate a random value with a non-uniform (weighted) probability distribution in SystemVerilog constraints? Write an example, and explain the difference between the := and :/ weight operators.

ANSWER

```
class txn;
  rand bit [1:0] op_type;   // 0=READ, 1=WRITE, 2=BURST_READ, 3=BURST_WRITE

  constraint c_dist {
    op_type dist {
      0 := 40,     // READ:        weight 40
      1 := 40,     // WRITE:       weight 40
      2 := 10,     // BURST_READ:  weight 10
      3 := 10      // BURST_WRITE: weight 10
    };
  }
  // total weight = 100 here, so these read directly as percentages, but
  // weights are just RELATIVE -- they don't have to sum to 100.
endclass

// := vs :/  -- the two weight operators:
//
//   := assigns the given weight to EACH INDIVIDUAL value listed. If you
//      write a RANGE with :=, every value in that range gets the SAME
//      weight (not a shared/split total).
//
//   :/ assigns the given weight to be SPLIT EVENLY ACROSS a RANGE of
//      values -- the number is a TOTAL for the whole range, divided
//      among however many values it covers.
//
// Example contrasting them directly:
class txn2;
  rand bit [2:0] val;   // 0..7

  constraint c_dist2 {
    val dist {
      0       := 10,    // value 0 alone gets weight 10
      [1:3]   := 5,     // EACH of 1, 2, 3 individually gets weight 5
                          // (this range contributes 5+5+5 = 15 total)
      [4:7]   :/ 40     // the range [4:7] TOGETHER gets a total weight of
                          // 40, split evenly -> each of 4,5,6,7 gets 10
    };
  }
```

```
endclass
```

解释

考点：dist 约束是生成非均匀分布随机激励的标准写法，考察点集中在 := 和 :/
这两个权重操作符很容易被记混，面试喜欢直接让你现场解释区别甚至手算概率。

记忆方法：把 := 理解成"每个值各自拿到这么多权重"，把 :/
理解成"这一整个区间总共拿到这么多权重，大家平分"——同样写 [4:7]，用 := 40
意味着4、5、6、7各自权重40（总权重40x4=160里这个区间占160），用 :/ 40
意味着4、5、6、7加起来权重40，每个只有10。

实践用途：weighted distribution常用来模拟真实场景里各类事务发生的相对频率（比如读比写更频繁、突发传输是小概率事件但必须覆盖到），比单纯均匀随机更贴近实际工作负载，也比手写if-else挑分支更简洁。

延伸考点：面试官可能会追问权重是否必须加起来等于100（不需要，权重只是相对大小，工具会自动按比例归一化），以及如何验证生成的分布确实符合预期权重（通常配合covergroup里的bins统计实际出现频次来做功能覆盖率验证，而不是只靠约束本身"应该"符合分布就假设它一定对）。

Q38. SystemVerilog: Boundary-Crossing Constraints and Circular Pointer Wraparound

QUESTION

When randomizing memory addresses for a burst transaction, how do you (a) constrain random generation to AVOID crossing a defined boundary (e.g. a 4KB page boundary), and separately, how do you deliberately generate boundary-crossing cases as a targeted negative test? (b) If you're modeling a circular/wrapping address pointer (e.g. a ring buffer), how do you handle the pointer wrapping from the highest address back down to the lowest address, both in the hardware logic and when randomizing a starting point to exercise that wrap?

ANSWER

```
// (a) Boundary-crossing constraint
class burst_txn;
  rand bit [31:0] start_addr;
  rand bit [7:0]  burst_len;       // number of bytes in the burst
  localparam bit [31:0] BOUNDARY = 32'h1000;   // 4KB

  // AVOID crossing: start_addr and the LAST byte of the burst must fall
  // in the same BOUNDARY-sized page.
  constraint c_no_cross {
    (start_addr / BOUNDARY) == ((start_addr + burst_len - 1) / BOUNDARY);
  }

  // Targeted NEGATIVE test: force the SAME condition to be violated
  // instead -- just invert the comparison.
  // constraint c_force_cross {
  //   (start_addr / BOUNDARY) != ((start_addr + burst_len - 1) / BOUNDARY);
  // }
endclass
// Toggle between the two with constraint_mode() (see the earlier
// constraint-package question) rather than maintaining two separate
// classes -- enable c_no_cross for baseline/regression tests, enable
// c_force_cross (with c_no_cross disabled) for the dedicated
// boundary-crossing corner-case test.

// (b) Circular pointer wraparound
```

```
//
// Power-of-2 depth: wraparound is FREE -- natural unsigned overflow of a
// fixed-width counter already wraps 2^N-1 -> 0 with no extra logic:
always_ff @(posedge clk)
  if (incr) ptr <= ptr + 1'b1;      // silently wraps at the width boundary

// Non-power-of-2 depth: overflow would land PAST the valid range, so an
// explicit wrap check is required:
always_ff @(posedge clk)
  if (incr) ptr <= (ptr == DEPTH-1) ? '0 : ptr + 1'b1;

// Randomizing a STARTING pointer that is guaranteed to exercise the wrap
// within a short test sequence: constrain the start point to be near the
// top of the range, so a handful of subsequent increments in the test
// is guaranteed to cross back to 0.
class ptr_gen;
  localparam int DEPTH = 256;
  rand bit [$clog2(DEPTH)-1:0] start_ptr;

  constraint c_near_wrap {
    start_ptr inside {[DEPTH-4 : DEPTH-1]};
  }
endclass
```

解释

考点：这道题把"边界穿越"和"指针绕回"两个场景放在一起问，本质都是"随机测试怎么主动去够到一个低概率但真实存在的角落场景"这个共同主题。

(a) 的关键思路：先写一个"避免穿越"的约束保证正常测试不出岔子，再把同一个比较表达式取反变成"强制穿越"的定向负测试，用constraint_mode()在两者之间切换而不是维护两份类定义——这是一个可以复用到很多"边界corner case"约束场景的通用套路（之前约束包那道题里也提到过同样的思路）。

(b) 的关键思路：如果深度正好是2的幂，绕回是"免费"的（无符号计数器溢出天然回绕，不需要额外比较逻辑），这也是为什么很多硬件设计里FIFO/环形缓冲区深度都优先选2的幂；如果深度不是2的幂，就必须显式判断"是不是到了最大值"然后手动清零，多了一次比较逻辑的开销。

延伸考点：这个话题和更早的异步FIFO/格雷码指针那道题是同一个知识体系的不同侧面——FIFO指针额外多一位wrap bit本质上也是在解决"指针绕回"带来的满/空歧义问题，能把这几道题的知识点串起来讲说明你对"环形地址"这个概念有系统性的理解，而不是每道题都当成孤立的知识点。

Q39. SystemVerilog: Sequence Item with Cross-Field Parity Constraint + randc Error Injection

QUESTION

Complete the following (partial) sequence_item class:

```
class Packet extends uvm_sequence_item;
  rand bit [31:0] payload[];    // number of elements: 1 to 100
  rand bit [31:0] parity;
  randc int error_chance;
  // bit-0 of parity = OR of bit-0 of every payload element
  // bit-1 of parity = OR of bit-1 of every payload element
  // ... (bit-31 similarly)
endclass
```

Complete the class so parity is correctly derived from payload as described. Follow-up: how would you add a 1% chance of an "error case" where payload has 0 elements instead of 1-100?

ANSWER

```
class Packet extends uvm_sequence_item;
  `uvm_object_utils(Packet)

  rand bit [31:0] payload[];
  rand bit [31:0] parity;
  randc bit [6:0]  error_chance;     // see c_error_range below -- this is
                                       // what makes the "1%" follow-up exact

  constraint c_error_range {
    error_chance inside {[0:99]};    // constrains the randc CYCLE to
                                       // exactly 100 values (0..99)
  }

  constraint c_payload_size {
    // Follow-up answer lives here: force the "1% error case" (0-length
    // payload) exactly when error_chance == 0, normal 1-100 otherwise.
    if (error_chance == 0) payload.size() == 0;
    else                   payload.size() inside {[1:100]};
  }

  constraint c_parity {
    parity == compute_parity(payload);
  }

  // Ordering matters: error_chance must be resolved before payload's
  // size constraint can be evaluated, and payload must be fully resolved
  // (size AND contents) before compute_parity() has something to work on.
  constraint c_solve_order {
    solve error_chance before payload;
    solve payload       before parity;
  }

  function bit [31:0] compute_parity(bit [31:0] arr[]);
    bit [31:0] p = 32'h0;
    foreach (arr[i]) p = p | arr[i];
    return p;
  endfunction

  function new(string name = "Packet");
    super.new(name);
  endfunction
endclass

// Why randc (not plain rand + dist) for the "exactly 1%" requirement:
//   A plain `rand` field with a dist-weighted 1% probability gives you
//   an AVERAGE of 1-in-100 over many trials, with real statistical
//   variance -- across any given 100 calls you might get 0 error cases,
//   or 3, purely by chance. randc, once constrained to a 100-value
//   range (0..99), is GUARANTEED to produce every one of those 100
//   values EXACTLY once before any value repeats -- so error_chance==0
//   is guaranteed to occur EXACTLY once every 100 randomize() calls, a
//   true deterministic 1-in-100, not just a long-run average. This is
//   the right tool specifically when the requirement is phrased as a
//   guaranteed frequency ("we need this corner case to actually show up
//   once every N calls") rather than a probability.
//   Caveat: constraining a randc variable's cycle range like this is
//   standard, well-supported practice in full-featured commercial
//   simulators (VCS/Questa/Xcelium); tool support for combining randc
//   with additional range constraints can vary on lighter-weight tools,
//   worth a quick sanity check in your specific environment.
```

解释

考点：这道题综合了动态数组约束、跨字段的按位归约约束（用函数实现）、solve...before求解顺序、以及randc的精确周期特性，是一道把多个约束高级特性揉在一起的综合题。

parity的实现思路：不是直接把parity声明成完全自由的rand变量，而是用一个函数compute_parity对payload做按位OR归约，再用 parity == compute_parity(payload) 这条约束把两者绑定——约束块里调用函数是合法且常见的技巧，用来表达"这个字段必须等于对另一个字段做某种计算的结果"这类关系，比手写31个bit分别约束干净得多。

求解顺序的重要性：payload的内容必须先被求解器确定下来，compute_parity才有实际数据可算，所以要显式加 solve payload before
parity；同理error_chance要先确定，payload的size约束才知道走哪个分支，所以也要 solve error_chance before payload——这是使用"约束依赖计算结果"这种技巧时必须注意的细节，顺序不对可能导致工具报错或者行为不确定。

1%这个follow-up的核心考点：面试官想看你是否能分清"平均概率"和"确定性周期频率"这两种不同的随机需求——如果只是要求"大约1%概率触发"，用rand+dist足够；但如果要求"每100次里必须精确出现1次"（这在功能覆盖率要求"必须命中N次角落用例"的场景很常见），dist给不了这个保证，只有randc配合约束把周期长度锁定在100，才能做到数学上精确。

延伸考点：面试官可能会追问如果测试跑的次数不是100的整数倍会怎样（每完整周期保证恰好1次，不完整的部分次数里出现次数依然是"至多1次"，不会重复出现同一个值直到用完整个周期），以及如果需要多个独立的"1%事件"同时存在（需要多个独立的randc字段，或者用一个更大范围的randc字段配合多个互斥的取值区间划分不同的角落场景）。

Q40. SystemVerilog: Generating Unique Random Values Across Iterations

QUESTION

Given:

```
class packet;
  rand bit [2:0] a;
endclass

packet p1;
p1 = new();
for (int i = 0; i < 8; i++) begin
  p1.randomize();
  $display("a=%0d", p1.a);
end
```

How do you make each generated value of 'a' unique from all previously generated values (across the 8 iterations)?

ANSWER

```
// Approach 1 (recommended): track used values in a QUEUE, and exclude
// them via an inline constraint -- lets the SOLVER directly avoid
// repeats in one shot, no wasted retries.
module tb1;
  class packet;
    rand bit [2:0] a;
  endclass

  packet p1;
  bit [2:0] used[$];
```

```
  initial begin
    p1 = new();
    for (int i = 0; i < 8; i++) begin
      assert(p1.randomize() with { !(a inside {used}); });
      used.push_back(p1.a);
      $display("a=%0d", p1.a);
    end
  end
endmodule

// Approach 2 (matches the "associative array" hint in the question):
// track used values as the KEYS of an associative array (used as a
// set), and retry randomize() in a loop until it produces a value not
// already in the set.
module tb2;
  class packet;
    rand bit [2:0] a;
  endclass

  packet p1;
  bit seen[bit [2:0]];   // associative array as a "seen" set

  initial begin
    p1 = new();
    for (int i = 0; i < 8; i++) begin
      do begin
        assert(p1.randomize());
      end while (seen.exists(p1.a));
      seen[p1.a] = 1;
      $display("a=%0d", p1.a);
    end
  end
endmodule

// Approach 3 (encapsulated -- keeps the uniqueness logic INSIDE the
// class itself, using a static member + post_randomize(), so the
// caller doesn't need to manage any external queue/array at all):
class packet_v3;
  rand bit [2:0] a;

  // 'static' means this ONE queue is shared across EVERY instance of
  // packet_v3 that ever exists in the simulation -- not a per-object
  // copy. That is a real behavioral difference from Approaches 1/2,
  // where uniqueness is only as global as wherever the CALLER happens
  // to declare the tracking queue/array.
  static bit [2:0] used[$];

  constraint c_unique {
    !(a inside {used});
  }

  // post_randomize() is a built-in callback that SystemVerilog
  // automatically invokes right after every successful randomize()
  // call on this object -- this is what lets the class update its own
  // bookkeeping without the caller having to remember to do it.
  function void post_randomize();
    used.push_back(a);
  endfunction
endclass

module tb3;
  packet_v3 p1;
```

```
  initial begin
    p1 = new();
    for (int i = 0; i < 8; i++) begin
      assert(p1.randomize());     // uniqueness is now fully self-contained
      $display("a=%0d", p1.a);
    end
  end
endmodule

// Trade-off between all three:
//   Approach 1 (queue + inline constraint) is more EFFICIENT: the
//   solver is told up front which values are off-limits and picks
//   directly from the remaining legal set -- no wasted attempts.
//   Approach 2 (associative array + retry loop) is simpler to write but
//   can waste many randomize() calls, especially on the LAST iteration
//   -- by i=7, 7 of the 8 possible 3-bit values are already used, so on
//   average it takes ~8 retries to randomly land on the 1 remaining
//   legal value.
//   Approach 3 (static member + post_randomize) uses the SAME
//   constraint-based mechanism as Approach 1 (so it's just as
//   efficient), but ENCAPSULATES it: the class manages its own
//   uniqueness invariant internally, and the calling code has no idea
//   any bookkeeping is happening at all -- it just calls randomize()
//   in a loop. The cost of that convenience is the 'static' scope
//   itself: because 'used' is shared by every packet_v3 object, you
//   cannot have two INDEPENDENT streams of unique values in flight at
//   the same time (e.g. two different tests running in parallel, each
//   wanting its own private uniqueness pool) without them interfering
//   with each other -- that scenario is exactly where Approaches 1/2
//   (where the caller controls the queue/array's scope explicitly)
//   remain the more flexible choice.
//
// A hard limit either way: 'a' is only 3 bits wide (8 possible values:
// 0-7). The loop runs exactly 8 times, so by the LAST iteration every
// value except one has been used -- this works out exactly, but a 9th
// iteration would be UNSATISFIABLE (Approach 1/3's randomize() would
// fail and return 0; Approach 2's retry loop would spin forever). Worth
// calling out explicitly if the loop bound could ever exceed 2^(width).
```

解释

考点：这是"保证N次随机结果互不重复"的标准题，考察点是知道有哪几种典型实现思路，并且清楚各自的效率、封装性和边界问题，而不只是写出其中一种能跑的代码。

方法对比的核心：用队列+inline约束是"让求解器一次到位"，效率高、不会做无用功；用关联数组+重试循环是"生成后再检查，不满足就重来"，写起来更直观但效率会随着已用值变多而变差，越到后面重试次数期望值越高；用static成员+post_randomize()则是把前两种思路里"效率最高的那种"（约束排除）包装进类本身，调用方完全不需要知道内部在做什么记账，直接反复调用randomize()即可——这是封装性最好的写法，本质是把方法1的机制"藏"进类里。

static方法的关键取舍：因为used是static，它被这个类的所有对象共享，好处是调用方代码最干净，坏处是没办法让两个packet_v3对象各自维护一份独立的"已用值"池——如果需要两条完全独立的唯一值流（比如两个并行测试各自需要自己的唯一性保证，互不干扰），就必须回到方法1/2那种"由外部显式控制队列/数组作用域"的写法，static的全局共享特性反而成了限制。

延伸考点：面试官可能会追问如果字段位宽很大（比如32位）但需要生成的唯一值数量也很大，用关联数组/队列去存"已用值"在内存和查找效率上是否合适（这种规模下用关联数组的O(1)查找通常优于队列的线性inside检查），以及post_randomize()还有哪些常见用途（比如在随机化完成后自动计算校验和/派生字段，这在别的题目里也出现过）。

Q41. SystemVerilog: Diet Planner App -- Class Design with Branching Constraints

QUESTION

A diet planner app suggests one of 3 kinds of diets, built from six possible fruits/veggies: apple, spinach, carrot, banana, mango, strawberry.

diet1 -- 3 items: apple, spinach, and one of the remaining (any one

of carrot/banana/mango/strawberry).

diet2 -- 1 item only: apple, mango, or strawberry.

diet3 -- 4 items: first is banana; the rest are 3 UNIQUE

fruits/veggies chosen from the remaining (apple, spinach,

carrot, mango, strawberry).

Write a SystemVerilog class implementing this app, including an interface to get the diet.

ANSWER

```
typedef enum { APPLE, SPINACH, CARROT, BANANA, MANGO, STRAWBERRY } food_e;

class DietPlanner;
  rand int unsigned diet_type;   // 1, 2, or 3 -- which diet was generated
  rand food_e        diet[];      // the generated list of food items

  constraint c_diet_type {
    diet_type inside {1, 2, 3};
  }

  // ---- diet1: apple, spinach, + 1 unique item from the rest ----
  constraint c_diet1 {
    if (diet_type == 1) {
      diet.size() == 3;
      diet[0] == APPLE;
      diet[1] == SPINACH;
      diet[2] != APPLE;
      diet[2] != SPINACH;      // "one of the remaining" -- anything
                               // other than apple/spinach themselves
    }
  }

  // ---- diet2: exactly 1 item, from a restricted 3-item subset ----
  constraint c_diet2 {
    if (diet_type == 2) {
      diet.size() == 1;
      diet[0] inside {APPLE, MANGO, STRAWBERRY};
    }
  }

  // ---- diet3: banana + 3 unique items from the remaining 5 ----
  constraint c_diet3 {
    if (diet_type == 3) {
      diet.size() == 4;
      diet[0] == BANANA;
      diet[1] != BANANA;
      diet[2] != BANANA;
      diet[3] != BANANA;
      unique { diet[1], diet[2], diet[3] };   // pairwise distinct
    }
  }
```

```
  function new();
  endfunction

  // ---- the app's public interface: request a diet of a given type ----
  function food_e get_diet_item(int idx);
    return diet[idx];
  endfunction

  function int get_diet_size();
    return diet.size();
  endfunction

  // Caller-facing entry point: "give me a diet of type X" (or leave
  // type_wanted as 0 to let the app pick any of the 3 types randomly).
  function bit request_diet(int unsigned type_wanted = 0);
    bit ok;
    if (type_wanted == 0)
      ok = this.randomize();
    else
      ok = this.randomize() with { diet_type == type_wanted; };
    return ok;
  endfunction

  // Convenience: human-readable diet, for display/logging.
  function string to_string();
    string s;
    string names[food_e] = '{APPLE:"apple", SPINACH:"spinach",
                             CARROT:"carrot", BANANA:"banana",
                             MANGO:"mango", STRAWBERRY:"strawberry"};
    s = $sformatf("diet%0d: ", diet_type);
    foreach (diet[i]) s = {s, names[diet[i]], (i == diet.size()-1) ? "" : ", "};
    return s;
  endfunction
endclass

// Example usage (the "interface to get the diet" in action):
// DietPlanner planner = new();
// void'(planner.request_diet(1));         // ask specifically for diet1
// $display(planner.to_string());          // "diet1: apple, spinach, mango"
//
// void'(planner.request_diet());          // let it pick any of the 3 types
// $display(planner.to_string());
```

解释

考点：这是一道"把一个真实产品需求翻译成约束+类设计"的综合题，Google这类公司很喜欢用这种贴近实际应用场景（而不是纯电路/纯UVM）的题来考察通用的编程和建模能力，SystemVerilog只是载体，考的是需求分解和约束表达能力。

设计的核心思路：

1. 用一个enum把6种食材定义清楚，用一个动态数组rand food_e diet[]配合.size()约束表达"这个diet到底有几种食材"——这是处理"不同分支元素个数不一样"这类需求的标准做法（这道题里3种diet的食材数量分别是3/1/4，用固定大小的数组没法优雅表达）。

2. 用diet_type做分支选择器，三条约束块分别用if(diet_type==N)包起来，只在对应分支生效——同一个类通过一个字段驱动三种完全不同的约束逻辑，比写三个独立的子类更紧凑，适合这种"共享同一个数据结构、只是规则不同"的场景。

3. diet3的"3个互不相同"用unique{}约束直接表达，这是处理"若干变量两两不等"这类需求最简洁的写法，比手写多个"!="两两比较可读性好得多，也是这道题里最容易被漏掉的语言特性（很多人会写一堆diet[1]!=d

iet[2], diet[1]!=diet[3], diet[2]!=diet[3]这种笨拙但等价的写法）。

接口设计的思路：request_diet()作为对外暴露的入口，调用方只需要说"我要diet1"或者"随便给我一个"，不需要了解内部约束是怎么写的，这是把随机化细节封装在类内部、只暴露简洁接口的良好实践，也是题目里"including interface to get the
diet"这句话真正想考察的点——不是只会写约束，还要知道怎么包装成一个好用的对外API。

延伸考点：面试官可能会追问如果以后要加"diet4"该怎么扩展设计（在这个"单类+分支约束"的设计下，加一个新的if(diet_type==4){...}约束块+扩展diet_type的合法范围即可，改动集中、不影响已有逻辑），以及如果某些食材有库存限制（比如香蕉今天卖完了）要怎么临时排除（可以用constraint_mode()动态关掉某个约束块，或者用inline randomize() with
{}临时加一条排除条件，这两种技巧在之前的约束包那道题里都讲过）。

Q42. SystemVerilog: Generate Valid Solved 9×9 Sudoku Puzzle

QUESTION

Write UVM SystemVerilog constraints to generate a valid, complete 9×9 Sudoku solution. Constraints: Each row contains 1-9 exactly once, each column contains 1-9 exactly once, each 3×3 box contains 1-9 exactly once.

ANSWER

```
// Sudoku Puzzle Constraints
class sudoku extends uvm_object;
  rand bit [3:0] grid[9][9]; // values 1-9

  constraint c_range {
    foreach (grid[i,j]) grid[i][j] inside {[1:9]};
  }

  // Row uniqueness
  constraint c_rows {
    foreach (grid[i,]) {
      unique {grid[i][0], grid[i][1], grid[i][2], grid[i][3],
              grid[i][4], grid[i][5], grid[i][6], grid[i][7], grid[i][8]};
    }
  }

  // Column uniqueness
  constraint c_cols {
    foreach (grid[,j]) {
      unique {grid[0][j], grid[1][j], grid[2][j], grid[3][j],
              grid[4][j], grid[5][j], grid[6][j], grid[7][j], grid[8][j]};
    }
  }

  // 3x3 box uniqueness (iterate over 9 boxes)
  constraint c_boxes {
    foreach (grid[i,j]) {
      if (i % 3 == 0 && j % 3 == 0) {
        unique {grid[i][j],     grid[i][j+1],   grid[i][j+2],
                grid[i+1][j],   grid[i+1][j+1], grid[i+1][j+2],
                grid[i+2][j],   grid[i+2][j+1], grid[i+2][j+2]};
      }
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q43. SystemVerilog: Array with Exactly 3 Identical Elements and 7 Unique Elements

QUESTION

Write UVM SystemVerilog constraint for integer array of 10 elements where exactly 3 elements share the same value (triplicate), and the remaining 7 elements are all unique and different from the triplicate value. Total: 8 distinct values (1 appearing 3 times, 7 appearing once each).

ANSWER

```
class three_same extends uvm_object;
  `uvm_object_utils(three_same)

  rand bit [7:0] arr[10];
  rand bit [7:0] dup_val;
  rand int unsigned dup_idx[3];

  function new(string name = "three_same");
    super.new(name);
  endfunction

  constraint c_dup_idx {
    unique {dup_idx};
    foreach(dup_idx[i]) dup_idx[i] inside {[0:9]};
  }

  constraint c_triplicate {
    foreach(arr[i]) {
      if ((i == dup_idx[0]) || (i == dup_idx[1]) || (i == dup_idx[2]))
        arr[i] == dup_val;
      else
        arr[i] != dup_val;
    }
  }

  constraint c_bounds {
    dup_val inside {[0:255]};
    foreach(arr[i]) arr[i] inside {[0:255]};
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q44. SystemVerilog: Generate Two 3×3 Matrices with Different Minimum Values

QUESTION

Write SystemVerilog uvm constraints to generate two 3×3 matrices such that the minimum value in matrix A differs from the minimum value in matrix B. Optionally enforce minimum uniqueness within each matrix.

ANSWER

```
// Two Matrices Unique Minimums
class unique_minimums extends uvm_object;
  rand bit [7:0] m1[3][3];
  rand bit [7:0] m2[3][3];
  rand bit [7:0] min1, min2;

  constraint c_bounds {
    foreach (m1[i,j]) m1[i][j] inside {[1:50]};
    foreach (m2[i,j]) m2[i][j] inside {[1:50]};
  }

  constraint c_min1 {
    foreach (m1[i,j]) min1 <= m1[i][j];
    min1 == m1[0][0] || min1 == m1[0][1] || min1 == m1[0][2] ||
    min1 == m1[1][0] || min1 == m1[1][1] || min1 == m1[1][2] ||
    min1 == m1[2][0] || min1 == m1[2][1] || min1 == m1[2][2];
  }

  constraint c_min2 {
    foreach (m2[i,j]) min2 <= m2[i][j];
    min2 == m2[0][0] || min2 == m2[0][1] || min2 == m2[0][2] ||
    min2 == m2[1][0] || min2 == m2[1][1] || min2 == m2[1][2] ||
    min2 == m2[2][0] || min2 == m2[2][1] || min2 == m2[2][2];
  }

  constraint c_unique_mins {
    min1 != min2;
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q45. SystemVerilog: Matrix Where Each Row Has Unique Maximum and All Maximums Distinct

QUESTION

Write uvm sv constraints to generate 2D array (M rows, N columns) with constraints: (1) Each row has exactly one maximum value (strict maximum, appears once in that row), (2) Maximum values across all rows are distinct (no two rows share same max value).

ANSWER

```
// Unique Row Maximums
class unique_row_max extends uvm_object;
  rand bit [7:0] matrix[4][4];
  rand bit [7:0] row_max[4];

  constraint c_bounds {
    foreach (matrix[i,j]) matrix[i][j] inside {[1:50]};
  }

  constraint c_row_max_val {
    foreach (row_max[i]) {
      row_max[i] >= matrix[i][0];
      row_max[i] >= matrix[i][1];
      row_max[i] >= matrix[i][2];
      row_max[i] >= matrix[i][3];
      row_max[i] == matrix[i][0] || row_max[i] == matrix[i][1] ||
      row_max[i] == matrix[i][2] || row_max[i] == matrix[i][3];
    }
  }

  constraint c_unique_max {
    unique {row_max};
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

总线协议类设计要特别注意握手信号（valid/ready或者addr/data phase）的时序约定是否符合协议规范。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q46. SystemVerilog: Instruction Stream with Repetition Window Constraints

QUESTION

Write uvm sv constraints to generate instruction stream from {ADD, SUB, MUL} with rules: (1) ADD cannot repeat within last 3 cycles, (2) SUB cannot repeat within last 3 valid instructions, (3) NOP is excluded entirely.

ANSWER

```
// Instruction Repetition Constraints
class instr_rep extends uvm_object;
  rand bit [3:0] opcode[10]; // 10 instructions

  constraint c_range {
    foreach (opcode[i]) opcode[i] inside {[0:7]}; // 8 opcodes
  }

  // Same opcode cannot appear more than 3 times consecutively
  constraint c_no_3_consec {
    foreach (opcode[i]) {
      if (i >= 2) !(opcode[i] == opcode[i-1] && opcode[i] == opcode[i-2]);
    }
```

```
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q47. SystemVerilog: 10-bit Variable with Uniform Popcount Distribution

QUESTION

Generate 10-bit variable where popcount (number of 1-bits) uniformly distributed: 10% probability for exactly 1 bit set, 10% for 2 bits, ..., 10% for all 10 bits set. Exclude popcount=0. write uvm sv constraint without $countones if constraint system doesnt support it.

ANSWER

```
// Uniform Distribution by Bit Count
class uniform_bitcount extends uvm_object;
  rand bit [7:0] data;
  rand int unsigned target_ones;

  constraint c_target_dist {
    target_ones inside {[0:8]};
    // uniform across possible popcount values
  }

  constraint c_popcount {
    $countones(data) == target_ones;
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q48. SystemVerilog: Randomly Distribute Array into N Non-Empty Queues

QUESTION

Given an input array, randomly map elements to N output queues (N parameterized). Use UVM SystemVerilog constraints so that: (1) each input element appears in exactly one output queue, (2) all N output queues are non-empty, and (3) feasibility requires N ≤ input_size.

ANSWER

```
// Map to N Non-Empty Queues
class map_to_queues extends uvm_object;
  `uvm_object_utils(map_to_queues)

  parameter int N = 4;
  localparam int M = 12;

  int unsigned values[M];
  int unsigned out_q[N][$];
  rand int unsigned assign_q[M];
  int unsigned queue_count[N];

  constraint c_bounds {
    foreach (values[i]) values[i] inside {[1:50]};
    foreach (assign_q[i]) assign_q[i] inside {[0:N-1]};
  }

  constraint c_all_queues_nonempty {
    foreach (queue_count[k]) {
      queue_count[k] == assign_q.sum() with (int'(item == k));
      queue_count[k] >= 1;
    }
  }

  function void post_randomize();
    foreach (out_q[k]) out_q[k].delete();
    foreach (assign_q[i]) out_q[assign_q[i]].push_back(values[i]);
  endfunction

endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q49. SystemVerilog: Generate Number with All 1-Bits Grouped in Single Contiguous Run

QUESTION

Write uvm sv constraint to generate number whose binary representation has all 1-bits grouped together in single contiguous run. Pattern: some 0s, then all 1s together, then some 0s. Run length can be 0 to W (width). Support all-zeros and all-ones edge cases if specified.

ANSWER

```
// Grouped 1s in Binary (all 1s contiguous)
class grouped_ones extends uvm_object;
  rand bit [15:0] data;
  rand int unsigned start_bit, num_ones;

  constraint c_ones_range {
    num_ones inside {[1:16]};
```

```
    start_bit inside {[0:15]};
    start_bit + num_ones <= 16;
  }

  constraint c_grouped {
    foreach (data[i]) {
      if (i >= start_bit && i < start_bit + num_ones)
        data[i] == 1'b1;
      else
        data[i] == 1'b0;
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q50. SystemVerilog: Solve 8 Queens Problem Using Constraints

QUESTION

Write uvm sv constraints to Place 8 queens on 8×8 chessboard such that no two queens attack each other. Queens attack same row, column, or diagonal. Model as constraint problem: assign column position for each row's queen.

ANSWER

```
// 8 Queens Problem
class eight_queens extends uvm_object;
  rand int unsigned col[8]; // col[row] = column position of queen in that row

  constraint c_range {
    foreach (col[i]) col[i] inside {[0:7]};
  }

  constraint c_unique_cols {
    unique {col};
  }

  constraint c_diagonals {
    foreach (col[i]) {
      foreach (col[j]) {
        if (i < j) {
          (col[i] - col[j]) != (i - j);
          (col[i] - col[j]) != (j - i);
        }
      }
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q51. SystemVerilog: Dynamic Array Size 300 with Value Frequency and Consecutive Constraints

QUESTION

Generate dynamic array of exactly 300 elements. Elements take values from {0,1,2,3,4}. UVM SV Constraints: (1) Each value (0,1,2,3,4) appears at least 40 times, (2) Value 0 can appear consecutively, (3) Values 1,2,3,4 cannot appear consecutively (no adjacent repeats).

ANSWER

```
// Dynamic Array with Repetition Rules
class dyn_array_rep extends uvm_object;
  `uvm_object_utils(dyn_array_rep)

  rand int unsigned arr[];
  int unsigned cnt[5];

  constraint c_size {
    arr.size() == 300;
  }

  constraint c_domain {
    foreach (arr[i]) arr[i] inside {[0:4]};
  }

  constraint c_frequency {
    cnt[0] == arr.sum() with (int'(item == 0));
    cnt[1] == arr.sum() with (int'(item == 1));
    cnt[2] == arr.sum() with (int'(item == 2));
    cnt[3] == arr.sum() with (int'(item == 3));
    cnt[4] == arr.sum() with (int'(item == 4));
    foreach (cnt[v]) cnt[v] >= 40;
  }

  constraint c_no_adjacent_repeat_for_1_to_4 {
    foreach (arr[i]) if ((i > 0) && (arr[i] inside {[1:4]})) arr[i] != arr[i-1];
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q52. SystemVerilog: Stream with Last 4 Generated Values All Different

QUESTION

Write uvm sv constraints to generate stream of random numbers where the most recent four values are always unique (no repeats in last 4). Maintain sliding window history. Define value domain large enough for feasibility (≥5 distinct values).

ANSWER

```
// Last 4 Values Unique
class last4_unique extends uvm_object;
  rand bit [7:0] value;
  bit [7:0] history[$];

  constraint c_bounds {
    value inside {[0:31]};
  }

  constraint c_not_in_history {
    if (history.size() >= 1) value != history[history.size()-1];
    if (history.size() >= 2) value != history[history.size()-2];
    if (history.size() >= 3) value != history[history.size()-3];
    if (history.size() >= 4) value != history[history.size()-4];
  }

  function void post_randomize();
    history.push_back(value);
    if (history.size() > 4) history.pop_front();
  endfunction
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q53. SystemVerilog: Odd-Sized Matrix with Unique Sub-Square Maximums

QUESTION

Write uvm sv constraints to generate odd-sized square matrix where: (1) Divide matrix into non-overlapping sub-squares, (2) Each sub-square has exactly one unique maximum, (3) All other elements in sub-square less than max, (4) Maximum values across all sub-squares are distinct, (5) Design scales to 32-bit integer limits.

ANSWER

```
// Sub-Square Maximum Constraint
class sub_square_max extends uvm_object;
  rand bit [7:0] matrix[4][4];

  constraint c_bounds {
    foreach (matrix[i,j]) matrix[i][j] inside {[1:50]};
  }
```

```
  // Every 2x2 sub-square has max <= 30
  constraint c_sub_square {
    foreach (matrix[i,j]) {
      if (i < 3 && j < 3) {
        matrix[i][j] <= 30;
        matrix[i][j+1] <= 30;
        matrix[i+1][j] <= 30;
        matrix[i+1][j+1] <= 30;
      }
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

总线协议类设计要特别注意握手信号（valid/ready或者addr/data phase）的时序约定是否符合协议规范。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q54. SystemVerilog: Generate Instruction Schedule with Latency-Based Non-Overlap Constraints

QUESTION

Write uvm sv constraints to generate instruction schedule where each instruction has execution latency. Constraint: No two identical instructions (same opcode) can have overlapping execution windows. Different opcodes can overlap. Instructions: ADD, SUB, MUL, NOP with defined latencies.

ANSWER

```
// No Overlapping Instructions (RAW hazard model)
class no_overlap extends uvm_object;
  rand bit [2:0] rd[8];  // destination register
  rand bit [2:0] rs1[8]; // source register 1
  rand bit [2:0] rs2[8]; // source register 2

  constraint c_range {
    foreach (rd[i])  rd[i]  inside {[0:7]};
    foreach (rs1[i]) rs1[i] inside {[0:7]};
    foreach (rs2[i]) rs2[i] inside {[0:7]};
  }

  // No RAW: instruction i+1 sources != instruction i destination
  constraint c_no_raw {
    foreach (rd[i]) {
      if (i < 7) {
        rs1[i+1] != rd[i];
        rs2[i+1] != rd[i];
      }
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q55. SystemVerilog: 5 Set Bits, 80% Consecutive, 20% Non-Consecutive

QUESTION

Write uvm sv constraints to generate random number with exactly 5 bits set. With 80% probability, the 5 bits are consecutive (contiguous run). With 20% probability, the 5 bits are non-consecutive (spread out). Define word width to accommodate.

ANSWER

```
// 5 Bits Set with Consecutive Probability
class five_bits_consec extends uvm_object;
  rand bit [15:0] data;
  rand bit consecutive;

  constraint c_five_bits {
    $countones(data) == 5;
  }

  constraint c_consec_dist {
    consecutive dist {1 := 30, 0 := 70};
  }

  constraint c_apply_consec {
    if (consecutive) {
      // 5 consecutive bits set somewhere
      (data & 16'h001F) == 16'h001F ||
      (data & 16'h003E) == 16'h003E ||
      (data & 16'h007C) == 16'h007C ||
      (data & 16'h00F8) == 16'h00F8 ||
      (data & 16'h01F0) == 16'h01F0 ||
      (data & 16'h03E0) == 16'h03E0 ||
      (data & 16'h07C0) == 16'h07C0 ||
      (data & 16'h0F80) == 16'h0F80 ||
      (data & 16'h1F00) == 16'h1F00 ||
      (data & 16'h3E00) == 16'h3E00 ||
      (data & 16'h7C00) == 16'h7C00 ||
      (data & 16'hF800) == 16'hF800;
    }
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q56. SystemVerilog: Generate Two Dynamic Arrays with Size, Sorted, and Membership Constraints

QUESTION

Generate two related dynamic arrays using UVM SystemVerilog constraints: (1) array1 size is in [6:9], (2) array2 size equals array1 size, (3) array1 is sorted in ascending order, and (4) every array2 element is a member of array1.

ANSWER

```
// Two Arrays with Sorted Relationship
class sorted_relationship extends uvm_object;
  `uvm_object_utils(sorted_relationship)

  rand int unsigned array1[];
  rand int unsigned array2[];
  rand int unsigned pick_idx[];

  constraint c_size {
    array1.size() inside {[6:9]};
    array2.size() == array1.size();
    pick_idx.size() == array2.size();
  }

  constraint c_sorted_array1 {
    foreach (array1[i]) if (i > 0) array1[i] >= array1[i-1];
  }

  constraint c_membership {
    foreach (pick_idx[i]) {
      pick_idx[i] inside {[0:array1.size()-1]};
      array2[i] == array1[pick_idx[i]];
    }
  }

  constraint c_bounds {
    foreach (array1[i]) array1[i] inside {[0:100]};
    foreach (array2[i]) array2[i] inside {[0:100]};
  }
endclass
```

解释

考点：这是一道约束随机化题，重点是能否用简洁的约束表达式描述题目给出的规则，而不是用过程式代码硬写。

约束题的关键是先想清楚用inside/unique/foreach这些声明式结构怎么表达规则，而不是先想过程式的实现步骤。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

SystemVerilog -- Assertions (SVA)

Q57. SystemVerilog: Identify the Design, Write the RTL, Write All Applicable Assertions

QUESTION

Given the waveform below (IN stays high for several clock cycles, then drops low; OUT pulses high for exactly one cycle aligned with IN's high-to-low transition):

CLK: ...toggling normally...

IN: goes high, stays high for N cycles, then goes low

OUT: pulses high for exactly 1 cycle, right where IN falls

(a) What kind of design is this?

(b) Write the RTL.

(c) Write all applicable SVA assertions for it.

ANSWER

```
// (a) This is a FALLING-EDGE DETECTOR: it produces a single-cycle pulse
//     synchronized to clk, exactly on the cycle where IN transitions
//     from high to low.
//
// (b) RTL:
module edge_detector (
  input  logic clk,
  input  logic rst_n,
  input  logic in,
  output logic out
);
  logic in_d;   // 'in', delayed by one clk cycle

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) in_d <= 1'b0;
    else        in_d <= in;

  assign out = in_d & ~in;   // was high last cycle, is low THIS cycle
endmodule

// Verified by simulation (Icarus Verilog): drove in high for 3 cycles
// then low; out stayed 0 the entire time in was high, and pulsed high
// for exactly the one cycle where in dropped to 0, then returned to 0 --
// matches the given waveform exactly.

// (c) SVA assertions:

// Master correctness property: out is high IF AND ONLY IF a falling
// edge of 'in' just occurred (this alone subsumes most of what you'd
// want to check about the core function).
property p_out_iff_fell;
  @(posedge clk) disable iff (!rst_n)
  out == $fell(in);
endproperty
assert property (p_out_iff_fell);

// out must never be a 2-cycle-or-longer pulse -- exactly 1 cycle wide.
property p_out_single_cycle;
  @(posedge clk) disable iff (!rst_n)
  out |=> !out;
endproperty
assert property (p_out_single_cycle);

// out must be low throughout reset and for one cycle after release
// (in_d hasn't captured a real sample yet).
property p_out_low_after_reset;
  @(posedge clk)
  !rst_n |-> !out;
endproperty
assert property (p_out_low_after_reset);
```

```
// out should never assert unless 'in' was actually high at some point
// -- guards against a spurious pulse coming purely from reset/X states.
property p_out_implies_prior_high;
  @(posedge clk) disable iff (!rst_n)
  out |-> $past(in);
endproperty
assert property (p_out_implies_prior_high);

// Optional coverage (not correctness, but useful to confirm the
// interesting case is actually exercised in a given test run):
cover property (@(posedge clk) disable iff (!rst_n) $fell(in) ##1 out);
```

解释

考点：先识别电路类型（下降沿检测器），再写RTL，再写断言，是验证面试里"看波形反推设计意图"这类题的完整链条，三步环环相扣，任何一步理解错了后面全错。

识别设计的关键：注意OUT的高电平窗口和IN从高到低的那个转折点对齐，而不是和IN从低到高对齐——这一步看错就会把"下降沿检测"误判成"上升沿检测"，两者RTL实现的最后一行反相的位置完全相反（in_d & ~in 对下降沿，~in_d & in 对上升沿），面试时一定要对着波形图仔细数清楚对齐的是哪个转折点。

断言设计的层次：核心正确性断言（out是否恰好等于$fell(in)）是"必须有"的一条，直接用系统函数$fell表达意图，比自己手写"上一拍是1这一拍是0"更清晰不容易出错；脉冲宽度断言、复位断言、"不能凭空出现"断言都是围绕核心正确性做的补充和加固，面试官如果只要求"写出所有适用的断言"，这种分层次覆盖（核心功能+边界/复位+防止意外触发）的思路比堆砌一堆断言更有条理。

延伸考点：面试官可能会追问如果要同时检测上升沿和下降沿（脉冲的两个边沿都要触发OUT）RTL和断言分别怎么改（RTL改成 in_d ^ in 做异或即可同时捕捉两种边沿；断言相应改成 out == ($rose(in) || $fell(in))），以及如果IN本身来自异步域、还没同步进clk域，这个电路要不要先加同步器（需要，这是一个很好的主动指出潜在CDC问题的加分点）。

Q58. SystemVerilog: Rising Edge Followed by Falling Edge Within 10 Cycles

QUESTION

Write sequence assertion to verify that a rising edge on signal is followed by a falling edge within 10 clock cycles. Ensure edge ordering and timing.

ANSWER

```
// Rising Then Falling Edge
module assert_rise_fall;
  property p_rise_then_fall;
    @(posedge clk) disable iff (rst)
    $rose(sig) |-> ##[1:10] $fell(sig);
  endproperty

  a_rise_fall: assert property (p_rise_then_fall)
    else $error("FAIL: no falling edge within 10 cycles of rising edge");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q59. SystemVerilog: Array Access Index Within Valid Range

QUESTION

Write assertion to ensure that array "data_array" is accessed only with valid indices within range [0, SIZE-1]. Catch out-of-bounds accesses.

ANSWER

```
// Array Index Valid Range
module assert_index_range;
  property p_valid_index;
    @(posedge clk) disable iff (rst)
    rd_en |-> (addr < DEPTH);
  endproperty

  a_valid_index: assert property (p_valid_index)
    else $error("FAIL: addr=%0d exceeds DEPTH=%0d", addr, DEPTH);
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q60. SystemVerilog: Write UVM Assertion: Transaction Data Validity Check

QUESTION

Write assertion to check transaction data validity in UVM sequence or monitor. Verify data fields meet protocol requirements (parity, range, no X).

ANSWER

```
// Transaction Data Validity
module assert_txn_valid;
  property p_txn_data;
    @(posedge clk) disable iff (rst)
    txn_valid |-> !$isunknown(txn_data) && !$isunknown(txn_addr);
  endproperty

  property p_txn_response;
    @(posedge clk) disable iff (rst)
    txn_valid |-> ##[1:10] txn_done;
  endproperty

  a_data_valid: assert property (p_txn_data);
  a_response:   assert property (p_txn_response);
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q61. SystemVerilog: Arbiter Grants Requests Fairly

QUESTION

Write assertion to verify arbiter grants requests fairly to multiple clients. Define fairness metric (bounded wait or round-robin ordering). Prevent starvation.

ANSWER

```
// Arbiter Fair Granting
module assert_fair_arbiter;
  property p_req_eventually_granted(req_bit, gnt_bit);
    @(posedge clk) disable iff (rst)
    req_bit |-> ##[1:MAX_LATENCY] gnt_bit;
  endproperty

  property p_mutual_exclusive_grant;
    @(posedge clk) disable iff (rst)
    $onehot0(grant); // at most one grant active
  endproperty

  a_req0: assert property (p_req_eventually_granted(req[0], grant[0]));
  a_req1: assert property (p_req_eventually_granted(req[1], grant[1]));
  a_req2: assert property (p_req_eventually_granted(req[2], grant[2]));
  a_mutex: assert property (p_mutual_exclusive_grant);
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

仲裁器类设计的核心考点是"公平性"——不能让某个请求方永远抢不到授权，这也是最容易被断言/覆盖率检查出问题的地方。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q62. SystemVerilog: Detect Rising Edge on Signal

QUESTION

Write SystemVerilog assertion to detect a rising edge on a signal. Typically detect edge of a control signal (not the clock itself). Use $rose() function sampled on clock.

ANSWER

```
// Detect Rising Edge
```

```
module assert_rising_edge;
  // Rising edge of sig should trigger ack within 1 to 3 cycles
  property p_rising_edge_fixed;
    @(posedge clk) disable iff (rst)
    $rose(sig) |-> ##[1:3] ack;
  endproperty

  a_rising: assert property (p_rising_edge_fixed)
    else $error("FAIL: ack not within 1-3 cycles after sig rising edge");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q63. SystemVerilog: Counter Zero Within 1 Cycle After Reset De-assertion

QUESTION

Write temporal assertion to ensure that after reset is de-asserted, counter becomes zero within 1 clock cycle. Handles synchronous reset with timing check.

ANSWER

```
// Counter Zero After Reset
module assert_counter_reset;
  property p_counter_after_reset;
    @(posedge clk)
    $fell(rst_n) |=> (counter == 0);
  endproperty

  a_counter_reset: assert property (p_counter_after_reset)
    else $error("FAIL: counter not zero after reset deasserted");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

涉及跨时钟域的部分要用两级同步器处理，格雷码保证多比特信号跨域时最多只有一位在变化。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q64. SystemVerilog: Data Remains Stable for 5 Cycles After Enable

QUESTION

Write sequence assertion to ensure data signal remains consistent (unchanged) for 5 consecutive clock cycles after enable signal is asserted.

ANSWER

```
// Data Consistent After Enable
module assert_data_after_en;
  property p_data_consistent;
    @(posedge clk) disable iff (rst)
    $rose(enable) |=> $stable(data) [*3];
  endproperty

  a_data_consistent: assert property (p_data_consistent)
    else $error("FAIL: data not stable for 3 cycles after enable rose");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q65. SystemVerilog: Write Coverage for Protocol Event Sequences

QUESTION

Write coverage assertions to measure coverage of different valid event sequences in a communication protocol. Track sequence patterns and protocol phases.

ANSWER

```
// Protocol Event Sequence Coverage
module assert_protocol_seq;
  // REQ -> GNT -> DATA -> ACK sequence
  sequence s_protocol;
    req ##[1:3] gnt ##1 data_valid ##[1:2] ack;
  endsequence

  property p_protocol;
    @(posedge clk) disable iff (rst)
    req |-> s_protocol;
  endproperty

  a_protocol: assert property (p_protocol);
  c_protocol: cover property (p_protocol);

  covergroup cg_protocol @(posedge clk);
    cp_req_to_gnt: coverpoint (req && !gnt) {
      bins wait_1 = {1} iff ($past(req, 1) && !$past(gnt, 1));
    }
  endgroup
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q66. SystemVerilog: Integrate Assertion Coverage into UVM Testbench

QUESTION

Integrate assertion coverage metrics into UVM testbench to track which assertions have been exercised. Report assertion coverage at end of test.

ANSWER

```
// UVM Assertion Coverage Integration
class assert_cov_subscriber extends uvm_subscriber #(my_transaction);
  covergroup cg_txn with function sample(my_transaction t);
    cp_addr: coverpoint t.addr {
      bins low  = {[0:63]};
      bins mid  = {[64:191]};
      bins high = {[192:255]};
    }
    cp_data: coverpoint t.data {
      bins zero  = {0};
      bins small = {[1:127]};
      bins large = {[128:255]};
    }
    cross cp_addr, cp_data;
  endgroup

  function new(string name, uvm_component parent);
    super.new(name, parent);
    cg_txn = new();
  endfunction

  function void write(my_transaction t);
    cg_txn.sample(t);
    // inline assertion checks
    assert(t.addr < 256) else `uvm_error("COV", "addr out of range")
    assert(!$isunknown(t.data)) else `uvm_error("COV", "data is X")
  endfunction
endclass
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

写断言时优先用$rose/$fell/$past这些内置系统函数表达"边沿/历史值"关系，比手写延迟比较更清晰不易出错。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q67. SystemVerilog: Ensure No Starvation in Multi-Client Arbiter

QUESTION

Write assertion to ensure no client is starved in multi-client arbiter. Continuous request must eventually be granted within bounded time.

ANSWER

```
// No Starvation in Arbiter
module assert_no_starvation;
  parameter MAX_WAIT = 100;

  genvar i;
  generate
    for (i = 0; i < NUM_PORTS; i++) begin : g_no_starve
      property p_no_starvation;
        @(posedge clk) disable iff (rst)
        req[i] |-> ##[1:MAX_WAIT] grant[i];
      endproperty

      a_no_starve: assert property (p_no_starvation)
        else $error("Port %0d starved: req held for >%0d cycles", i, MAX_WAIT);
    end
  endgenerate
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

仲裁器类设计的核心考点是"公平性"——不能让某个请求方永远抢不到授权，这也是最容易被断言/覆盖率检查出问题的地方。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q68. SystemVerilog: Output Equals Sum of Two Inputs

QUESTION

Write SystemVerilog assertion to verify that output "out" always equals the sum of input signals "a" and "b". Account for arithmetic width, overflow, and pipeline delay if applicable.

ANSWER

```
// Output Equals Sum
module assert_sum;
  property p_sum;
    @(posedge clk) disable iff (rst)
    valid |-> (out == (a + b));
  endproperty

  a_sum: assert property (p_sum)
    else $error("FAIL: out != a + b when valid");
endmodule
```

解释

考点：这是一道SVA断言题，重点是能否把自然语言描述的时序/数据关系准确翻译成property，而不是语法本身。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

SystemVerilog -- Coding

Q69. SystemVerilog: Blocking vs Non-Blocking Assignment -- Display Timing Trace

QUESTION

Consider the following testbench skeleton:

```
module tb;
  reg a;

  initial begin
    a = 0;
    #2 $display("t=%0t a=%b", $time, a);
    #5 $display("t=%0t a=%b", $time, a);
  end

  initial begin
    #10;
    a = 1;
  end
endmodule
```

(a) Trace the simulation and state exactly what each $display prints, with the simulation time at which it executes.

(b) The interviewer now changes "a = 0" to "a <= 0" (non-blocking) and/or changes the delay values. Does the printed output change? Explain the GENERAL rule for when blocking (=) vs non-blocking (<=) assignment choice actually changes simulation results, with a concrete example where it does.

ANSWER

```
// (a) Trace with the delays as given:
//
//   t=0   : first initial block starts, "a = 0" executes immediately (t=0)
//   t=0   : second initial block starts, begins waiting #10
//   t=2   : first block's "#2" elapses -> $display fires -> prints a=0
//             (second block's a=1 hasn't happened yet; it's scheduled for t=10)
//   t=7   : first block's second "#5" elapses (2 + 5 = 7) -> $display fires
//             -> still prints a=0 (still before t=10)
//   t=10  : second block's a=1 finally executes (both displays already ran)
//
//   OUTPUT:
//     t=2 a=0
//     t=7 a=0
//
// (b) Changing "a = 0" to "a <= 0" does NOT change this particular trace,
//     because there is no event-ordering ambiguity here: the two initial
//     blocks never touch 'a' at the exact same simulation time, so
//     blocking vs non-blocking makes no observable difference in THIS
//     example.
```

```
//
//     GENERAL RULE: blocking (=) executes immediately, in program order,
//     within the current time step; non-blocking (<=) schedules the
//     UPDATE to happen in the NBA (Non-blocking Assign) region, at the
//     very end of the current time step, AFTER all blocking assignments
//     and all "current-value" reads in that time step have completed.
//     The choice only matters when multiple processes touch the SAME
//     variable at the SAME simulation time -- that's exactly the
//     situation non-blocking assignment exists to make deterministic.
//
//     Concrete example where it DOES matter -- change the second block's
//     delay from #10 to #2 (now it fires at the SAME time as the first
//     display):
//
//       initial begin
//         a = 0;
//         #2 $display("t=%0t a=%b", $time, a);   // both events now at t=2
//       end
//       initial begin
//         #2 a = 1;                                 // (or a <= 1;)
//       end
//
//     If "a = 1" is BLOCKING and happens to execute before the $display
//     in simulation scheduling order (order between the two initial
//     blocks at the same time step is NOT guaranteed by the LRM), the
//     display could print a=1 -- a race condition, output depends on
//     simulator/tool scheduling, NOT deterministic.
//     If "a <= 1" is NON-BLOCKING, the assignment is deferred to the NBA
//     region regardless of which process runs first, so the $display
//     (a blocking/"active region" read) is GUARANTEED to still see the
//     OLD value (a=0) at t=2 -- deterministic, simulator-independent.
//     This is exactly why non-blocking assignment is the standard,
//     required style for clocked (always_ff) sequential logic: it makes
//     same-edge read/write ordering between multiple always blocks
//     race-free and simulator-independent.
//
//     Verified with Icarus Verilog: for the blocking version, simply
//     reversing the SOURCE ORDER of the two initial blocks (with no
//     other change) flipped the printed result from a=0 to a=1 --
//     concrete proof this is a genuine, scheduling-order-dependent race.
//     The non-blocking version printed a=0 in BOTH orderings, exactly as
//     the general rule predicts.
```

解释

考点：这是数字验证面试里"追波形/追时序"类题目的经典变体，考的不是语法本身，而是理解仿真器的事件调度模型（active region / NBA region）。

给定的具体延时（#2, #5, #10）之间没有时间上的重合，所以本例里 blocking 和 non-blocking 结果一样，都打印 a=0
两次——这个"坑"就是面试官想看你是不是能分清"这道题恰好没有竞争"和"blocking/non-blocking 永远等价"这两件事，很多人会因为这道题结果一样就误以为两者没区别。

真正的区别只有在多个 process 在同一个仿真时刻同时读写同一个变量时才会体现：blocking 赋值立即生效、按代码执行顺序更新，多个 initial/always
块之间谁先执行没有语言层面的保证，会产生"竞争条件"；non-blocking 赋值把更新推迟到当前时间步的 NBA 区域，保证同一时刻的"读"一定读到旧值，这也是为什么时序逻辑（always_ff）标准写法必须用 <=，组合逻辑（always_comb）标准写法必须用 = 的根本原因。

延伸考点：面试官经常会追问"如果两个 initial 块都用 #0
在同一时刻赋值和读取，谁先执行"，标准答案是"仿真器不保证顺序，这是竞争条件"，恰恰是 non-blocking

存在的意义就是消除这种不确定性。

Q70. SystemVerilog: Generate a 100 MHz Clock with 25% Duty Cycle

QUESTION

Write SystemVerilog testbench code to generate a free-running clock at 100 MHz with a 25% duty cycle (i.e. HIGH for 25% of the period, LOW for the remaining 75%).

ANSWER

```
`timescale 1ns/1ps

module clk_gen_tb;
  bit clk;

  localparam realtime PERIOD    = 10ns;          // 100 MHz -> T = 1/100MHz = 10ns
  localparam realtime HIGH_TIME = PERIOD * 0.25;  // 2.5 ns  (25% duty cycle)
  localparam realtime LOW_TIME  = PERIOD - HIGH_TIME; // 7.5 ns

  initial begin
    clk = 0;
    forever begin
      #(LOW_TIME)  clk = 1;
      #(HIGH_TIME) clk = 0;
    end
  end
endmodule

// Equivalent, slightly more explicit always-block style:
//
//   initial clk = 0;
//   always begin
//     #7.5 clk = 1;   // stay low for 7.5ns (75% of the 10ns period)
//     #2.5 clk = 0;   // stay high for 2.5ns (25% of the 10ns period)
//   end
//
// NOTE: a single toggle statement like "#5 clk = ~clk;" can ONLY ever
// produce a 50% duty cycle, because it spends the same delay in both the
// high and low phases by construction. Any duty cycle other than 50%
// requires two DIFFERENT delay values -- one governing how long the
// clock stays low, and a separate one for how long it stays high, as
// shown above.
```

解释

考点：占空比不是50%的时钟怎么生成，很基础但容易漏掉的点是——单条 "#N clk=~clk"
只能生成50%占空比，因为高低电平停留时间必然相等。

计算方法：先算周期 T = 1/频率 = 1/100MHz =
10ns，再按占空比拆成高电平时间（10ns×25%=2.5ns）和低电平时间（10ns×75%=7.5ns），forever 循环里分别延时这两个值并翻转电平。

延伸考点：面试官可能会追问如果占空比要求是运行时可配置的（参数化），或者要求生成非整数ns的高精度占空比时怎么处理精度问题（涉及 timescale 的精度设置，比如 1ns/1ps
里的第二个参数决定了延时的最小分辨率）。

Q71. SystemVerilog: `logic` vs `bit` Data Types

QUESTION

What is the difference between the `logic` and `bit` data types in SystemVerilog? When would you choose one over the other?

ANSWER

```
// bit    -- a 2-STATE data type: each bit can only be 0 or 1, there is
//           NO X (unknown) or Z (high-impedance) representation.
//           Default initial value: 0.
//           Other 2-state types: byte, shortint, int, longint.
//
// logic  -- a 4-STATE data type: each bit can be 0, 1, X, or Z.
//           Default initial value: X.
//           `logic` is essentially a drop-in replacement for the old
//           Verilog `reg` type, but with one key improvement: `logic`
//           CAN be driven by a continuous assign statement (unlike
//           plain `reg`, which could only be driven procedurally).
//           `logic` still cannot have MULTIPLE drivers though -- for a
//           true multi-driver / tri-state bus, `wire` (net type) is
//           still required, since `logic` is a variable, not a net.
//
// When to use which:
//   Use `bit` (2-state) in TESTBENCH code: scoreboards, loop indices,
//   randomized stimulus values, reference-model variables. 2-state
//   simulation is faster and uses less memory (no X/Z tracking needed
//   per bit), which matters at scale across a large testbench.
//
//   Use `logic` (4-state) for DUT-facing / RTL signals, and generally
//   anywhere you want X to propagate as a deliberate debugging aid.
//   If you forget to drive a 2-state `bit` signal, it silently reads
//   as 0 -- a real bug can hide behind a "valid-looking" default value.
//   If you forget to drive a 4-state `logic` signal, it reads as X,
//   which is loud and visible in waveforms/checks -- exactly the
//   behavior you want when modeling real, physical hardware where an
//   undriven or uninitialized signal genuinely IS unknown.
```

解释

考点：logic 和 bit 的区别，本质是"2态 vs 4态"这个核心差异，面试考的不只是背定义，更看重你知道"什么场景该用哪个"以及为什么。

关键原因：4态的X/Z是在模拟真实硬件的不确定状态（未初始化寄存器、未驱动的三态总线），这个"不确定"本身是有意义的调试信号；2态类型为了仿真速度牺牲了这个能力，一个没被驱动的2态变量会悄悄默认成0，可能掩盖一个真实的bug（比如忘记连线、忘记初始化）。

实践经验：testbench 里大量使用 bit/int 等2态类型是为了让大规模验证环境跑得更快、占用更少内存，同时因为testbench代码通常不需要建模"未知态"这种硬件语义；RTL和DUT信号则几乎总是用 logic，因为X的传播是发现设计bug（比如读到了本不该读的未初始化路径）最直接的手段之一。

延伸考点：面试官可能会追问 logic 和 reg/wire 的关系（logic基本可以替代两者大部分用法，但多驱动源场景仍需要wire），以及为什么仿真里"2态变量默认在RTL里就是危险的"（因为RTL如果误用2态类型，会丢失X传播这个重要的调试信号，掩盖复位/初始化相关的bug）。

Q72. SystemVerilog: `==` (Logical Equality) vs `===` (Case Equality)

QUESTION

What is the difference between `==` (logical equality) and `===` (case equality) in SystemVerilog? Give an example where they produce different results, and explain why `===`/`!==` are not synthesizable.

ANSWER

```
// == (logical equality):
//   Standard arithmetic/logical comparison. If EITHER operand has ANY
//   bit that is X or Z, the result is itself X (unknown) -- exactly
//   like real hardware: you cannot definitively say two signals are
//   equal or unequal if part of either one is in an unknown state.
//
// === (case equality):
//   4-state IDENTITY comparison -- compares bit-for-bit INCLUDING the
//   X and Z states literally: X matches only X, Z matches only Z, 0
//   matches only 0, 1 matches only 1. ALWAYS returns a definite 1 or 0,
//   NEVER X -- because it isn't modeling a real hardware comparator, it
//   is purely a simulation/testbench construct for checking an EXACT
//   4-state bit pattern (same semantics as a `casex`/`case` statement's
//   literal pattern match -- hence "case equality").
//
// Example where they differ:
//   logic [3:0] a = 4'b10x1;
//   logic [3:0] b = 4'b10x1;
//
//   a == b;    // X  -- the x bit makes the comparison result unknown
//   a === b;   // 1  -- the bit patterns match EXACTLY, x included
//
//   logic [3:0] c = 4'b1010, d = 4'b1010;
//   c == d;    // 1  -- no X/Z involved, both operators agree here
//   c === d;   // 1
//
// Why === / !== are NOT synthesizable:
//   Real hardware comparators physically cannot produce a "definite"
//   answer when comparing against a truly unknown electrical state --
//   X and Z are SIMULATION-ONLY abstractions with no direct physical
//   circuit equivalent to compare against. === / !== only belong in
//   testbenches and assertions (e.g. checking that a signal is
//   correctly X during reset, or comparing against a reference value
//   that may legitimately contain X/Z). RTL intended for synthesis
//   should only ever use == / != (or the wildcard forms ==? / !=?,
//   which treat X/Z in the right-hand operand as don't-care bits, used
//   in casez-style comparisons).
```

解释

考点：== 和 === 的区别，是验证方向必考的基础题，考察对SystemVerilog四态逻辑（0/1/X/Z）的理解，以及"仿真语义"和"可综合语义"之间的边界在哪。

核心区别：== 是"数值比较"，只要有一方含有X/Z就说不清楚到底相不相等，结果本身也变成X；=== 是"位模式比较"，不管X/Z，只要两边每一位字面上完全一样（X对X，Z对Z）就返回1，这个操作符本身就是为了在仿真里精确比对四态模式而存在的。

为什么不可综合：X/Z是仿真器为了建模"未知/高阻"这种抽象状态而存在的记号，真实晶体管电路里没有一个物理比较器能够"识别"出对方是不是恰好处于未知态，所以===/!==只能用在testbench和断言里，写到打算综合的RTL代码里是错误用法。

延伸考点：面试官可能会追问 ==? 和 !=? 这两个"通配符相等"操作符（casez语义，X/Z在右操作数里当don't-care），以及为什么复位检测这类代码经常会用 === 而不是 ==（比如断言"复位期间某信号必须是X"，只有===才能精确匹配到X这个状态，用==结果永远是X，没法用来做"真"/"假"的判断）。

Q73. SystemVerilog: fork...join vs join_any vs join_none

QUESTION

Explain the difference between fork...join, fork...join_any, and fork...join_none in SystemVerilog. Give a short example of when you would use each, and mention how to avoid leaving orphaned background processes running.

ANSWER

```
// fork...join       -- parent BLOCKS until ALL forked branches finish.
//                       Use when every parallel branch must complete
//                       before the parent can safely continue.
fork
  drive_addr_phase();
  drive_data_phase();
join
// (both branches must finish before code after "join" executes)

// fork...join_any    -- parent BLOCKS until AT LEAST ONE (the first)
//                        forked branch finishes; the other branch(es)
//                        keep running in the BACKGROUND (not killed
//                        automatically). Classic use: race a real event
//                        against a timeout.
fork
  begin : real_event
    wait_for_response();
  end
  begin : timeout
    #1000ns;
    `uvm_error("TIMEOUT", "no response received")
  end
join_any
disable fork;   // *** kill whichever branch did NOT win the race ***
                // otherwise the losing branch (e.g. the timeout still
                // counting down, or wait_for_response() still pending)
                // keeps running silently in the background and can
                // fire/interfere later in the test -- an orphaned
                // process leak.

// fork...join_none   -- parent does NOT block AT ALL; it continues
//                        immediately past the fork statement while all
//                        forked branches run concurrently in the
//                        background. Use for "fire and forget"
//                        background work, e.g. starting several
//                        sequences in parallel from a loop.
fork
  background_monitor();
join_none
// ... parent keeps executing here immediately, monitor runs alongside ...
//
// To later know when ALL join_none-forked children (from anywhere in
// the current process) have finished, use:
wait fork;
```

解释

考点：三种 fork
变体的区别本身不难记，但面试更想考察你知不知道"忘记清理后台进程"这个常见的实际坑，尤其是 join_any 之后。

核心区别：join 等全部；join_any 等第一个，其余继续在后台跑；join_none 完全不等，全部丢到后台跑。

最容易被问到的坑：用 join_any 做超时竞赛（等真实事件 vs 超时）之后，如果不加 disable fork，"输掉"的那个分支（比如还在倒计时的timeout，或者还在等待的real_event）会一直挂在后台，可能在之后的测试阶段莫名其妙触发、干扰别的逻辑，属于典型的"进程泄漏"问题，disable fork
就是用来清理这些残留分支的标准做法。

延伸考点：join_none 配合循环并行启动多个进程时，涉及到 automatic
变量捕获循环下标的经典坑（在之前"并行启动10条sequence"那道题里有详细展开），以及最后用 wait fork 而不是各自加 join 来统一等待所有后台进程结束——这几个知识点经常被连起来一起考。

Q74. SystemVerilog: Indexed Part-Select (+: and -:) to Repack Unpacked Arrays

QUESTION

You have a source unpacked array with N elements of width W1 bits each, and need to repack that same data into a destination unpacked array with M elements of width W2 bits each, where N*W1 == M*W2 (same total bit count, different partitioning -- e.g. 8 elements of 4 bits repacked into 4 elements of 8 bits). Show how to do this using the indexed part-select operators (+: and -:), and explain what each one means and why they're needed here instead of a plain range select.

ANSWER

```
logic [3:0] src_arr [0:7];    // 8 elements x 4 bits = 32 bits total
logic [7:0] dst_arr [0:3];    // 4 elements x 8 bits = 32 bits total
logic [31:0] flat;

// (the loops below must live inside a procedural block -- a task here,
// or equally an initial/always block -- for-loops with assignments
// cannot sit directly at module scope)
task repack();
  // Step 1: flatten src_arr into one packed vector using +: (ascending
  // indexed part-select): "base +: width" selects bits [base+width-1 : base]
  for (int i = 0; i < 8; i++)
    flat[i*4 +: 4] = src_arr[i];     // element i lands at bit offset i*4, 4 bits wide

  // Step 2: re-slice the flat vector into dst_arr, again with +:
  for (int j = 0; j < 4; j++)
    dst_arr[j] = flat[j*8 +: 8];     // extract an 8-bit chunk starting at offset j*8

  // Equivalent flatten step written with -: (descending indexed part-select)
  // instead: "base -: width" selects bits [base : base-width+1] -- base is
  // now the TOP bit of the slice, and the slice extends DOWNWARD from it.
  for (int i = 0; i < 8; i++)
    flat[(i+1)*4-1 -: 4] = src_arr[i];   // top-anchored equivalent of the +: version above
endtask

// Why +:/-: instead of a plain range select like flat[hi:lo]:
//   A plain range select requires BOTH bounds to be constant (elaboration-
//   time) expressions -- you cannot write flat[i*4+3 : i*4] with a loop
//   variable i, because the tool can't statically determine a fixed bit
//   range from a variable. +:/-: solve exactly this: only the WIDTH must
//   be a constant, the BASE can be any variable/loop-indexed expression,
//   which is exactly what's needed to address into an array at a
//   computed, per-iteration offset.
//
// Verified with Icarus Verilog: with src_arr = {0,1,...,7}, both the +:
// and -: flatten methods produce the identical 32-bit pattern, and the
// resulting dst_arr = {16, 50, 84, 118} matches hand-computed values
// (e.g. dst_arr[0] = {src_arr[1],src_arr[0]} = {4'd1,4'd0} = 8'd16).
```

解释

考点：unpacked array 之间做"重新打包"（entry数量和位宽都不一样，但总位数相同）是数据通路/总线宽度转换类设计里的常见场景，+:/-: 索引式 part-select
是实现这类操作最基础也最容易在面试时被反复考的语法点。

核心理解：普通的位选择 [hi:lo]
要求上下界都是编译期常量，一旦想用循环变量算出一个"随下标变化"的位偏移，就必须用 +:/-: ——base 可以是变量，width
必须是常量，这正好匹配"从数组第i个元素算出对应的位偏移，宽度固定不变"这个需求。

+: 和 -: 的区别只是"锚点在哪、往哪个方向延伸"：+: 是以 base 为最低位往高位延伸；-: 是以 base 为最高位往低位延伸，两者本质等价，只是描述同一个位范围时选择的锚点不同，实际写代码时按照场景哪个更直观选哪个即可。

延伸考点：面试官经常会追问这两个操作符在真实场景里的应用（总线位宽转换、多个窄寄存器拼成一个宽寄存器、流式协议里数据按不同粒度重新分帧），以及和 SystemVerilog 的流操作符 {>>{}}/{<<{}}
相比各自的适用场景（流操作符更适合"整体打包/解包"，+:/-:
更适合"按计算出的偏移量做精确的变宽度切片"）。

Q75. SystemVerilog: Four Parallel Tasks -- First to Finish Cancels the Rest

QUESTION

You start 4 parallel tasks with fork. As soon as ANY ONE of the four finishes, you must immediately stop the other three -- they must not be allowed to keep running or complete. Write SystemVerilog for this pattern, and briefly recap which fork/join variant makes it possible (see the companion question on fork...join / join_any / join_none for the full comparison and the "orphaned process" pitfall).

ANSWER

```
task race_four();
  int winner = -1;

  fork
    begin : t0
      task1();
      if (winner == -1) winner = 0;
    end
    begin : t1
      task2();
      if (winner == -1) winner = 1;
    end
    begin : t2
      task3();
      if (winner == -1) winner = 2;
    end
    begin : t3
      task4();
      if (winner == -1) winner = 3;
    end
  join_any                // unblock as soon as the FIRST of the 4 finishes
  disable fork;            // *** immediately kill the other 3 (still-running) ***

  `uvm_info("RACE", $sformatf("task%0d finished first, others cancelled", winner), UVM_LOW)
endtask
```

```
// Mechanism recap:
//   join_any blocks the parent until the FIRST forked branch completes;
//   the other branches are NOT automatically stopped -- they keep running
//   in the background unless you explicitly kill them.
//   disable fork kills every child process forked by the CURRENT process
//   that is still outstanding -- exactly the 3 "losing" branches here.
//   It has no effect on the branch that already finished (there's nothing
//   left to disable there), and it works for any number of branches, not
//   just 2 -- the same mechanism scales to 4, 10, or N parallel tasks.
//   One subtlety: disable fork recursively tears down the WHOLE process
//   subtree -- if task2(), for example, itself forked additional
//   sub-processes internally, those grandchild processes are killed too,
//   not just the top-level branch.
//
// Tool note (verified directly): this pattern is correct per the LRM and
// works as described in full-featured commercial simulators. However,
// testing this exact shape (delay/work happening inside a CALLED task,
// as task1()..task4() do here) in Icarus Verilog specifically showed
// disable fork failing to stop the losing branches -- the task-internal
// delay ran to completion anyway. Inlining the same delay/work directly
// in the fork's begin/end blocks (no task call) worked correctly in the
// same tool. This is a known gap in Icarus's fork/disable support, not a
// flaw in the pattern itself -- worth knowing if you ever sanity-check
// this kind of code in Icarus rather than a commercial simulator.
```

解释

考点：这是"多路竞赛，谁先完成就取消其他所有"这个模式的一般化版本，本质机制和更早"两路竞赛"（真实事件 vs 超时）那道题完全一样，只是这里换成了4路而不是2路，考察你是否理解这个机制天然可以扩展到任意路数。

关键点：join_any只负责"谁先完成就解除阻塞"，并不会自动帮你把其他分支关掉，必须显式调用disable fork才能真正终止那些还在跑的"输家"分支，忘记这一步会导致后台残留进程继续跑，可能在之后的仿真阶段冒出来干扰别的逻辑。

延伸考点：disable fork是"递归"生效的——如果某个被fork出来的分支自己内部又fork了子进程，disable fork会把整棵进程子树都清理掉，这个细节面试官可能会追问；另外还可能问如果需要知道具体是哪一路赢了（不只是"随便哪路先完成就行"），怎么设计（本题用一个共享变量winner在每个分支内部尝试写入并且判断是否已经被写过，是常见的写法之一）。

Q76. SystemVerilog: Measuring Clock Cycles Between Two Events

QUESTION

Given two events, event_a and event_b, where event_a is GUARANTEED to always occur before event_b, write SystemVerilog code that measures and reports the number of clk cycles between event_a and event_b.

ANSWER

```
module latency_monitor (
  input  logic        clk,
  input  logic        rst_n,
  input  logic        event_a,
  input  logic        event_b,
  output logic [31:0] latency,        // valid for exactly 1 cycle, see latency_valid
  output logic        latency_valid
);
```

```
  logic counting;
  logic [31:0] cnt;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      counting      <= 1'b0;
      cnt           <= '0;
      latency       <= '0;
      latency_valid <= 1'b0;
    end else begin
      latency_valid <= 1'b0;              // 1-cycle pulse by default each cycle

      if (!counting) begin
        if (event_a) begin
          counting <= 1'b1;
          cnt      <= '0;                 // start counting from the NEXT cycle
        end
      end else begin
        if (event_b) begin
          latency       <= cnt;
          latency_valid <= 1'b1;
          counting      <= 1'b0;          // re-arm, ready for the next A..B pair
        end else begin
          cnt <= cnt + 1'b1;
        end
      end
    end
  end

endmodule

// Definition used here (state explicitly -- "cycles between" is ambiguous
// unless you pin it down): 'latency' counts the number of FULL clk cycles
// that elapse strictly BETWEEN the cycle event_a fires and the cycle
// event_b fires -- i.e. it does not count the event_a cycle itself, and
// does not count the event_b cycle itself. If event_b fires on the very
// next cycle after event_a, latency = 0.
//
// Verified by trace: event_a at cycle 0, event_b at cycle 3 ->
//   cycle 0: event_a seen, counting armed, cnt reset to 0 (no increment yet)
//   cycle 1: counting, event_b not yet seen -> cnt becomes 1
//   cycle 2: counting, event_b not yet seen -> cnt becomes 2
//   cycle 3: event_b seen -> latency = cnt = 2, latency_valid pulses
//   (2 full cycles elapsed strictly between the two events, as intended)
```

解释

考点：这是"测量两个事件之间延迟"这一类监测器（monitor）代码的标准写法，重点不是语法难度，而是能不能把"之间多少个周期"这种口语化描述转成一个无歧义的精确定义，并且用代码证明自己的定义。

关键设计点：用一个 counting 标志位区分"还没等到A"和"正在A到B之间计数"两个阶段，从A的下一拍开始计数、B到达那一拍停止计数并锁存结果，这样锁存的cnt值天然就是"A和B之间夹着的完整周期数"，不多不少。

易错点：很多人会直接从A那一拍就开始加1，导致最后结果多算一拍，或者在B那一拍还继续加了一次，都需要仔细过一遍边界情况（用具体的时序图画一遍A在第0拍、B在第3拍这种例子）来确认自己代码的计数语义和口头描述完全对应。

延伸考点：面试官可能会追问如果event_a和event_b可能同一拍同时出现怎么处理（本题设计里latency=0，因为一进counting状态就立刻检测同一拍的event_b——需要确认这个边界case的期望行为），以及如果event_b可能先于event_a出现（违反题目给的前提）要不要加保护逻辑（防御性设计的加分点，可以加一个错误标志位而不是静默给出错误的负延迟或者卡死）。

Q77. SystemVerilog: Debug a Broken UVM Monitor Class (4 Errors)

QUESTION

The broken UVM monitor class below has four distinct errors. Find each one and explain the fix:

```
class my_monitor extends uvm_monitor;
  `uvm_component_utils(my_monitor)

  interface my_if;                          // (1)
    logic clk, data;
  endinterface

  wire [7:0] data_captured;                 // (2)

  function run();                            // (3)
    data_captured = 8'h0;
  endfunction

  task collect();
    always @(posedge vif.clk)                // (4)
      data_captured = vif.data;
  endtask
endclass
```

ANSWER

```
// Corrected version, with each fix marked:

class my_monitor extends uvm_monitor;
  `uvm_component_utils(my_monitor)

  // FIX (1): an actual `interface ... endinterface` DEFINITION cannot
  // live inside a class body at all -- interfaces are static, module-
  // like constructs that belong at the same scope as modules, not
  // inside a class. A class may only hold a VIRTUAL INTERFACE HANDLE
  // (a reference to a real interface instance that lives elsewhere, in
  // the static module hierarchy):
  virtual my_if vif;     // 'my_if' is defined OUTSIDE this class, e.g.
                          // in a separate file, same as any interface.

  // FIX (2): `wire` is a NET type, meant to be driven by a continuous
  // `assign` or by a port connection -- it CANNOT be the target of a
  // procedural (blocking `=`) assignment inside a function/task/always
  // block. Use `logic` (a variable type) for anything assigned
  // procedurally:
  logic [7:0] data_captured;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  // FIX (3): a function that returns nothing must explicitly declare
  // `void` as its return type. Without it, SystemVerilog does NOT treat
  // this as a syntax error outright -- it silently falls back to an
  // IMPLICIT 1-bit return type (legacy Verilog behavior). The real
  // problem shows up when you then try to call it as a plain statement
  // ("run();" on its own line, ignoring any return value): per the LRM
  // this is illegal for anything except a void function or a task, and
  // stricter tools will flag it (verified: Icarus Verilog only WARNS
```

```
  // "User function is being called as a task" here, but VCS/Questa-class
  // tools with standard lint settings will error). Declare the return
  // type explicitly:
  function void run();
    data_captured = 8'h0;
  endfunction

  // FIX (4): `always @(...)` is a MODULE-level (or interface-level)
  // procedural block -- it can NEVER appear inside a task, a function,
  // or a class method. Icarus confirms this is a hard syntax error, not
  // just a style nitpick. Inside a task, the correct equivalent
  // construct is `forever` combined with an explicit event-control
  // statement:
  task collect();
    forever begin
      @(posedge vif.clk);
      data_captured = vif.data;
    end
  endtask

endclass

// Summary table:
//   (1) interface defined inside a class      -> illegal; use `virtual my_if vif;`
//   (2) wire assigned procedurally             -> illegal; use `logic`
//   (3) function with no explicit return type  -> legal syntax, but wrong
//                                                 intent; must be `function void`
//                                                 to be safely callable as a
//                                                 bare statement
//   (4) `always` inside a task                 -> illegal; use `forever` instead
```

解释

考点：这是一道"挑错"题，把新手（有时候也是有经验的人一时手滑）在写UVM组件时最常踩的几个坑集中在一起考，考察点是对"类的世界 vs 模块的世界"这条主线的理解，以及几个具体语法规则的精确记忆。

四个错误的共同主线：(1)和(4)本质都是"模块世界的东西不能直接搬进类的世界"——interface定义和always块都是属于静态结构层次的构造，类只能通过虚接口句柄去"引用"外部的interface，只能用forever去实现"持续执行"而不能用always；(2)是"变量类型选错"——wire是网线类型只能被动接受驱动，不能主动被过程赋值；(3)相对特殊，不是硬性语法错误而是"隐式行为容易踩坑"，没写void不代表编译不过，而是悄悄变成一个有隐式返回值的函数，真正出问题是在你把它当纯过程调用（不接收返回值）的时候。

易错点纠正：很多人会把(3)也当成跟(1)(2)(4)一样的"硬错误"来记，但实测（用Icarus Verilog实际编译验证过）它只是警告而不是报错，是"语义上不对但语法上勉强能过"的一类问题，跟真正的语法错误要分开理解——这也是面试官愿意深挖这道题的原因，能不能准确说出"这条到底是硬错误还是软性最佳实践"本身就是区分度所在。

延伸考点：面试官可能会追问为什么UVM要求类里只能放virtual interface而不能放真实interface（因为类对象是运行时动态构造的，而interface必须在仿真最开始的静态展开阶段就确定，两者的生命周期和创建时机完全不兼容）。

Q78. SystemVerilog: Polymorphism in SV/UVM, with a Worked Example

QUESTION

Explain polymorphism in SystemVerilog/UVM with a concrete example. Why is it fundamental to how UVM components and sequences are written?

ANSWER

```
class base_txn extends uvm_sequence_item;
  `uvm_object_utils(base_txn)
  rand bit [7:0] data;

  function new(string name = "base_txn");
    super.new(name);
  endfunction

  virtual function string convert2string();
    return $sformatf("base_txn: data=0x%0h", data);
  endfunction
endclass

class error_txn extends base_txn;
  `uvm_object_utils(error_txn)
  rand bit inject_error;

  function new(string name = "error_txn");
    super.new(name);
  endfunction

  // OVERRIDE: same method name/signature, different behavior
  virtual function string convert2string();
    return $sformatf("error_txn: data=0x%0h inject_error=%0b", data, inject_error);
  endfunction
endclass

// --- polymorphism in action ---
base_txn handle;
error_txn e;

e = error_txn::type_id::create("e");
handle = e;                          // upcast: legal, error_txn IS-A base_txn

$display("%s", handle.convert2string());
// prints "error_txn: data=... inject_error=..." -- NOT the base class's
// version -- even though 'handle' is DECLARED as base_txn, calling a
// VIRTUAL method through it dispatches to the ACTUAL (run-time) object's
// type, which is error_txn. This is polymorphism: the method that runs
// is determined by the OBJECT's real type, not the HANDLE's declared type.

// If convert2string() were NOT declared 'virtual' in the base class,
// calling handle.convert2string() would instead call base_txn's version
// unconditionally, regardless of what handle actually points to -- this
// is exactly why UVM's own built-in methods (convert2string, do_compare,
// do_copy, do_print, etc.) are all declared virtual in their base
// classes: it's what makes user-defined overrides actually take effect
// when the framework code calls them through a base-typed handle.

// Why this is fundamental to UVM specifically:
//   Almost the entire UVM library is written to operate on BASE class
//   handles (uvm_component, uvm_sequence_item, uvm_sequence, etc.) --
//   the phase mechanism calls build_phase()/run_phase() on a
//   uvm_component handle, a scoreboard's analysis export receives
//   uvm_sequence_item handles, print/compare/copy utilities operate on
//   generic object handles. NONE of this framework code knows or cares
//   about YOUR specific derived classes at compile time -- it only works
//   because every method it calls is virtual, so calling it through a
//   base handle still correctly reaches YOUR derived class's
//   implementation. Polymorphism is what lets one generic, reusable
//   framework operate correctly on an open-ended variety of
//   user-defined component/transaction types.
```

解释

考点：多态是面向对象最核心的概念之一，在UVM里几乎无处不在，但很多人只会背"父类指针指向子类对象"这句话，说不清楚为什么UVM框架必须依赖这个特性才能work。

关键例子设计：故意让convert2string()在子类里重写、用父类类型的handle去调用，观察到底跑的是哪个版本，这是验证"是否理解virtual关键字作用"最直接的方式——如果面试官追问"如果不加virtual会怎样"，一定要能准确说出"会固定调用父类版本，不管handle实际指向什么"这个对比结果。

为什么UVM离不开多态：UVM整个框架的phase机制、组件树遍历、scoreboard的analysis port等等，全部是通过uvm_component/uvm_object这类基类的接口在操作，框架代码本身完全不知道你具体定义了哪些派生类——如果没有多态，框架调用build_phase()永远只会执行uvm_component基类里那个空的默认实现，你自己写的组件逻辑根本不会被执行，这也是为什么UVM要求几乎所有可能被子类重写的方法必须声明为virtual。

延伸考点：面试官可能会追问多态和之前讲过的factory override是什么关系（factory override解决的是"创建哪个具体类型的对象"，多态解决的是"调用哪个具体类型的方法实现"，两者经常配合使用——factory决定了实际创建的是哪个派生类，多态保证了后续所有虚方法调用都能正确落到这个派生类的实现上），以及$cast在这个场景下的作用（如果需要反过来，通过一个base_txn的handle去访问error_txn独有的成员inject_error，就必须用$cast做显式向下转型）。

（补充说明：这次同时提到的 fork-join 相关问题，在框架里已经有三道题深入覆盖过——"fork...join / join_any / join_none"的区别（含orphaned process陷阱）、"4个并行任务谁先完成就disable其余的"、以及"顺序 vs 并行启动多条sequence"（含automatic变量捕获的经典坑），这里就不重复展开，如果这次想问的是某个具体的新场景，可以再补充细节我来针对性地写。）

Verification -- Test Plan

Q79. Verification -- Test Plan: Frequency Monitor (FMON) DUT

QUESTION

You are verifying a Frequency Monitor (FMON) DUT with the interface: clk, rst_n, and a 32-bit output period_out reporting the measured period of clk (e.g. in reference-clock counts). Write a test plan -- a list of test scenarios, each with an objective, stimulus, and checking strategy -- covering at minimum: reset behavior, a no-clock condition, steady-state accuracy across multiple frequencies, a dynamic frequency change, and glitch detection on the clock input.

ANSWER

```
// Test Plan: Frequency Monitor (FMON)
// DUT interface: clk, rst_n, period_out[31:0]
//
// TC1 -- Reset behavior
//   Objective: period_out holds a defined value while rst_n is asserted.
//   Stimulus:  assert rst_n for N cycles (with clk toggling), then release.
//   Check:     period_out == RESET_VALUE throughout reset, and does NOT
//              update even if clk keeps toggling while rst_n is low.
//
// TC2 -- No-clock condition
//   Objective: DUT must not report a bogus reading when clk is static.
```

```
//   Stimulus:  release reset, then hold clk at a constant level (no
//              edges at all) for a long simulated time.
//   Check:     period_out does not silently change to 0 or garbage; it
//              either holds its last valid value or reports a defined
//              "no-clock / timeout" code, per the DUT spec.
//
// TC3 -- Steady-state accuracy across multiple frequencies
//   Objective: measured period matches the actual clk period within the
//              DUT's specified accuracy/resolution.
//   Stimulus:  drive clk at several known, stable frequencies (e.g.
//              1 MHz, 10 MHz, 100 MHz, plus the DUT's rated min/max),
//              each held long enough for the measurement to settle.
//   Check:     for each frequency, period_out matches the expected
//              count within +/-1 LSB quantization tolerance; sweep the
//              full supported range, INCLUDING the min/max corners.
//
// TC4 -- Dynamic frequency change (clk1 -> clk2)
//   Objective: DUT correctly re-locks when the input frequency changes
//              on the fly, without a reset in between.
//   Stimulus:  run at clk1 until settled, then switch the generator to
//              clk2 (different frequency) with no reset pulse.
//   Check:     period_out tracks clk1 correctly before the switch, then
//              transitions to correctly tracking clk2 within a bounded,
//              spec'd number of cycles after the switch -- no stale or
//              averaged-together bogus value should persist.
//
// TC5 -- Glitch rejection on the clock input
//   Objective: DUT does not report a corrupted period when clk has a
//              runt pulse narrower than a real clock period.
//   Stimulus:  inject a glitch on clk -- a pulse whose width is
//              deliberately set below a defined glitch_criteria
//              threshold (via a force/release, or a clock-gen task that
//              occasionally inserts a short extra edge).
//   Check:     in the testbench/scoreboard, use $time to measure the
//              delta between consecutive edges seen by the monitor:
//                if ((edge_time - previous_edge_time) < glitch_criteria)
//                  flag_as_glitch();
//              verify the DUT either (a) filters the glitch and
//              period_out is unaffected, or (b) asserts a defined
//              glitch/error flag -- per spec -- but never silently
//              reports the short glitch width AS IF it were a real,
//              faster clock period.
//
// TC6 -- Glitch-width boundary sweep
//   Objective: characterize the DUT's actual glitch/real-edge decision
//              boundary against the spec'd glitch_criteria.
//   Stimulus:  sweep glitch width from very narrow (must be filtered)
//              up toward a genuine half-period (must be treated as a
//              real edge), injected at multiple points in the clock
//              cycle.
//   Check:     confirm the filtering boundary matches glitch_criteria;
//              this is exactly the corner where the spec is most
//              ambiguous and where bugs are most likely to hide, so it
//              deserves dedicated directed + boundary-value coverage,
//              not just "one glitch test."
```

解释

考点：这是一道典型的"给DUT功能描述、自己列测试计划"的验证综合题，考察点是能不能系统性地覆盖一个DUT的正常场景、边界场景和异常场景，而不是只测"能跑通"的那一条路径。

列测试计划的通用套路：先覆盖复位类（TC1），再覆盖输入缺失/异常类（TC2），再覆盖正常功能在参数空间内的准确性（TC3，尤其要扫最小/最大边界频率），再覆盖动态场景（TC4，运行中切换而不是只在复

位后配置一次），最后覆盖异常注入类（TC5/TC6，毛刺/干扰信号），这个顺序基本适用于任何"测量类"DUT的测试计划骨架，不只是频率监测器。

关键实现点：用 $time 记录相邻边沿的时间差和一个 glitch_criteria
阈值比较，这是验证环境里判定"这是不是毛刺"的标准做法，本质上就是用 testbench
自己实现一个参考模型（reference model）来判断黄金结果。

延伸考点：面试官很可能追问"毛刺宽度正好等于阈值时应该怎么处理"（边界值该算真边沿还是毛刺，这个模糊地带恰恰是 TC6 边界扫描要覆盖的），以及"如果DUT本身没有毛刺过滤功能，测试计划要不要测这一项"（答案是仍然要测，用来确认DUT的真实行为边界，即使结果是"确认它不过滤"也是有效的验证结论）。

Q80. Verification -- Test Plan: Basic SRAM Verification

QUESTION

Write a basic test plan for verifying an SRAM (assume a simple single-port SRAM, or note where dual-port matters), covering at minimum: basic write/read, data pattern tests, address coverage, boundary addresses, back-to-back access, and any read/write collision cases.

ANSWER

```
// Test Plan: SRAM (single read/write port, depth D, width W)
//
// TC1 -- Basic write-then-read
//   Objective: fundamental data integrity for a simple write followed by
//              a read at the same address.
//   Stimulus:  write a few fixed addresses with fixed data, read them
//              back immediately.
//   Check:     read data == written data, at each address tested.
//
// TC2 -- Data pattern tests (walking 1s / walking 0s / checkerboard)
//   Objective: catch stuck-at-0/1 faults and adjacent-bit coupling
//              faults on individual data lines.
//   Stimulus:  at a FIXED address, write a "walking 1" pattern (0001,
//              0010, 0100, 1000, ...), then a "walking 0" pattern, then
//              an alternating checkerboard pattern (0xAA / 0x55);
//              read back after each write.
//   Check:     each pattern reads back exactly as written.
//
// TC3 -- Address coverage / address walking pattern
//   Objective: catch address-decode faults (wrong location gets written)
//              and address-line coupling faults.
//   Stimulus:  write a UNIQUE data value to every address (e.g.
//              data = address, or an LFSR-derived unique value), across
//              the full address range, then read every address back.
//   Check:     each address returns its own unique expected value, not
//              a neighbor's value (this specifically catches address
//              decode / address-line bugs that a single fixed-address
//              test like TC1/TC2 cannot).
//
// TC4 -- Boundary addresses
//   Objective: catch off-by-one address-decode bugs at the edges.
//   Stimulus:  directed writes/reads at address 0, address (D-1)
//              (max address), and the addresses immediately adjacent to
//              both boundaries.
//   Check:     correct data at the true boundary addresses; NO aliasing
//              onto address D (out of range) or address -1 wrap.
//
```

```
// TC5 -- Back-to-back access, no idle cycles
//   Objective: catch pipeline/timing bugs that only show up when the
//              memory is hammered without idle gaps.
//   Stimulus:  issue back-to-back writes (or back-to-back reads, or
//              alternating write/read) every cycle, no bubbles, for many
//              consecutive cycles across many addresses.
//   Check:     every access still returns/stores the correct data --
//              no data loss or corruption purely due to lack of idle
//              time between accesses.
//
// TC6 -- Read/write collision (dual-port only)
//   Objective: verify defined behavior when a read and a write target
//              the SAME address on the SAME cycle on two different
//              ports.
//   Stimulus:  drive a simultaneous same-address read (port A) and
//              write (port B).
//   Check:     the read returns whatever the SPEC defines for this case
//              (old data, new data, or explicitly undefined/don't-care)
//              -- the key deliverable here is confirming the DUT matches
//              whatever the spec promises, not assuming a "sensible"
//              default.
//
// TC7 -- Random stimulus with a reference-model scoreboard
//   Objective: broad, unpredictable coverage beyond the directed cases
//              above, checked automatically rather than by hand.
//   Stimulus:  randomize address/data/wr_rd per the constraint package
//              from the earlier constraint question, run a long mixed
//              random sequence.
//   Check:     a scoreboard using an associative array (keyed by
//              address, see the companion "associative array in
//              scoreboard" question) as the reference model catches any
//              mismatch automatically across the whole random run.
```

解释

考点：SRAM测试计划是存储类DUT验证的标准入门题，考察点是能不能系统性地覆盖"基本功能、数据完整性、地址完整性、边界、时序压力、并发冲突、随机覆盖"这几个维度，而不是只写"写了读一下"这一条。

覆盖维度的逻辑顺序：先验证基本读写能work（TC1），再验证每一位数据线本身没有粘连故障（TC2，固定地址换数据），再验证地址译码没问题（TC3，固定数据换地址——这两类测试要分开做，因为它们分别针对数据通路和地址通路的不同故障模型），然后是边界（TC4）和时序压力（TC5）这两类"角落场景"，最后TC6/TC7分别覆盖并发冲突这种DUT特有行为和随机测试兜底。

延伸考点：面试官经常会追问TC3"地址走一遍"和TC2"数据走一遍"为什么要分开测（如果两个同时变，出了问题分不清是地址故障还是数据故障，控制变量是测试设计的基本原则），以及为什么TC7的随机测试不能替代前面的定向测试（随机测试覆盖面广但没有针对性，容易大量重复测到简单场景而很难精确命中边界这类小概率角落情况，定向+随机搭配是标准做法）。

Q81. Verification: Searching for Legal PLL Divider Coefficients (M, N, P)

QUESTION

A clock signal passes through a 3-stage chain, each stage controlled by one integer coefficient:

Stage A: clk_in = 100 MHz, divided by M -> clk_a = 100/M, must land in [10, 100] MHz

Stage B: clk_a multiplied by N -> clk_b = clk_a * N, must land in [900, 1600] MHz

Stage C: clk_b divided by P -> clk_c = clk_b / P, must equal 1000 MHz (within a small tolerance)

Write a program that searches for a legal (M, N, P) combination satisfying all three constraints, given the overall relationship out = in/M * N / P.

ANSWER

```
// First, the algebraic shortcut worth spotting before brute-forcing:
//   out = (100/M) * N / P = 1000
//   => N / (M*P) = 10
//   => N = 10 * M * P
// i.e. the ENTIRE 3-stage chain reduces to "output is always exactly
// 10x the input", and any (M,P) pair with N = 10*M*P chosen to also
// satisfy the two INTERMEDIATE range constraints (stage A and stage B)
// is a valid answer. A good candidate should mention this even while
// still delivering the brute-force search the question is actually
// asking for -- the ranges given for stage A/B rule out most (M,P)
// pairs, so the search space is small in practice.
//
// Brute-force nested search (bounds chosen generously; M,P <= 63 and
// N <= 255 are typical PLL divider-width assumptions -- adjust to match
// the real design's register widths):
task automatic find_pll_params(output int M, N, P, output bit found);
  real stageA, stageB, stageC;
  found = 1'b0;

  for (M = 1; M <= 63; M++) begin
    stageA = 100.0 / M;
    if (stageA < 10.0 || stageA > 100.0) continue;    // stage A out of range

    for (N = 1; N <= 255; N++) begin
      stageB = stageA * N;
      if (stageB < 900.0 || stageB > 1600.0) continue; // stage B out of range

      for (P = 1; P <= 63; P++) begin
        stageC = stageB / P;
        if (stageC > 999.99 && stageC < 1000.01) begin  // stage C hits target
          found = 1'b1;
          return;                                       // first legal combo found
        end
      end
    end
  end
endtask

// Verified (Python model of the identical search): 10 legal (M,N,P)
// combinations exist for these exact bounds, e.g. (M=1,N=10,P=1),
// (M=2,N=20,P=1), (M=3,N=30,P=1) ... and every one of them satisfies
// N == 10*M*P exactly, confirming the closed-form shortcut above.
```

解释

考点：这道题的关键陷阱就在笔记里写的那句话——"一开始以为是dut，仔细看是个数学问题"。题目用电路的语言包装（三级时钟链、每级一个系数），但求解本质是纯粹的约束满足问题：给定输入输出的固定倍数关系，中间两级各有一个范围约束，求一组满足所有约束的整数系数。

两层答题策略：先一眼看出"输出恒为输入的10倍"这个代数关系（N=10MP），这体现你有分析问题本质、而不是一上来就写三层循环暴力搜索的能力；但题目明确要求写程序，所以还是要老老实实交付这个嵌套循环搜索，只是可以在讲解时主动提一句"这里其实有个更快的代数捷径"，两者都展示出来是最完整的回答。

延伸考点：面试官会追问"为什么用嵌套for循环，这在真实验证里有什么用"——嵌套范围搜索/参数扫描（parameter sweep）正是验证里常见的手法，比如给定DUT规格约束，反过来搜索所有能通过spec的合法配置空间，用于生成配置覆盖率或者反向验证spec本身是否自洽；这道题笔记里提到"interviewer said they used

that in real design verification"正是在印证这一点。

Q82. Verification -- Test Plan: Divide-by-2 Pulse Circuit

QUESTION

How would you verify the divide-by-2 pulse circuit from the previous question (one flip-flop, one AND gate, producing a pulse once every 2 input clock cycles)? Write a test plan.

ANSWER

```
// Test Plan: clk_div2_pulse
//
// TC1 -- Reset behavior
//   Objective: pulse_out is deterministically LOW immediately after reset,
//              not X, regardless of what clk_in is doing.
//   Stimulus:  assert rst_n while toggling clk_in, then release.
//   Check:     pulse_out == 0 throughout reset; q == 0 the cycle after
//              reset releases (before any further clk_in edges).
//
// TC2 -- Basic divide-by-2 pulse pattern
//   Objective: confirm the exact once-every-2-cycles pulse pattern.
//   Stimulus:  drive a clean, free-running clk_in for many cycles after
//              reset.
//   Check:     pulse_out is high on exactly every OTHER clk_in high
//               phase (period = 2x clk_in period), matching the given
//               waveform sample-for-sample; NEVER two pulses in a row,
//               NEVER a gap of more than one missing cycle.
//
// TC3 -- Phase / alignment check
//   Objective: confirm the pulse is aligned to the CORRECT clk_in cycle
//              (not the "off" one) relative to the reset release point --
//              this is deterministic given a fixed reset value of q, so
//              it should be 100% repeatable, not just "roughly every 2
//              cycles."
//   Stimulus:  reset, then step through the first several clk_in cycles
//              one at a time.
//   Check:     the very FIRST pulse occurs on the exact cycle predicted
//              from q's known reset value (q=0 after reset means the
//              first clk_in rising edge toggles q to 1, so the pulse
//              appears starting the SECOND rising edge onward, matching
//              the given example exactly).
//
// TC4 -- Pulse width / no-glitch check
//   Objective: verify pulse_out never glitches (extra unexpected edges)
//              around the moment q toggles, especially with realistic
//              (non-zero) gate and flop delays applied via SDF-annotated
//              gate-level simulation rather than an ideal RTL sim.
//   Stimulus:  run the same free-running clk_in stimulus at gate level
//              with back-annotated timing, not just RTL/zero-delay sim.
//   Check:     count edges on pulse_out per input cycle; any EXTRA edge
//              beyond the expected one pulse-rising + one pulse-falling
//              per active cycle indicates a real glitch -- this is
//              exactly the propagation-delay hazard from the design
//              question, and RTL-only simulation with zero delays will
//              NOT catch it; it requires gate-level/timing-aware sim.
//
// TC5 -- Reset mid-operation
//   Objective: confirm re-asserting rst_n in the middle of normal
//              operation correctly re-syncs the pattern (doesn't leave
```

```
//              q or pulse_out in a stale/incorrect phase).
//   Stimulus:  let the circuit run normally for several pulses, assert
//              rst_n for one or more cycles, release, resume clk_in.
//   Check:     pulse pattern restarts from the correct, predictable
//              phase relative to the NEW reset release point, exactly
//              as in TC3.
```

解释

考点：给一个已经设计好的小电路写验证计划，考察的是"这么简单的电路能不能想全测试维度"，而不是复杂的方法学。

测试维度的设计逻辑：先测复位这个最基本的初始状态（TC1），再测稳态下的核心功能模式是否正确（TC2），然后是这道题特有的关键点——相位对齐必须是确定性的、可预测的，不能只满足"平均每两周期一个脉冲"这种统计意义上的正确（TC3），接下来才是本题真正的难点延伸——用RTL零延迟仿真是测不出propagation delay毛刺问题的，必须上门级时序仿真才能真正验证这个已知隐患是否存在（TC4），最后补一个"复位不是只在最开始，运行中途也可能发生"的场景（TC5）。

延伸考点：面试官很可能会追问"如果电路换成用ICG单元而不是裸与门，测试计划要不要变"（TC4依然需要，但预期结果不同——用ICG的版本应该在门级时序仿真里也不出现毛刺，如果出现了反而说明ICG例化或者约束有问题，这条测试从"发现已知问题"变成了"确认问题已经被修复"，是同一套测试思路在不同实现下的复用）。

Q83. Verification: Block-Level vs Top-Level Verification Strategy

QUESTION

Given a chip made of many blocks, how do you approach verifying it? Specifically: why not JUST verify the smaller blocks individually, since block-level verification runs faster (this is an open-ended question -- the interviewer wants your reasoning/methodology, not a single "correct" answer)?

ANSWER

```
// Short answer: you need BOTH block-level (unit) verification AND
// top-level/system (integration) verification -- they catch
// fundamentally DIFFERENT classes of bugs, and neither can substitute
// for the other.
//
// Why block-level verification alone is NOT sufficient:
//   1) Integration/interface bugs: two blocks can each be individually
//      PERFECT against their own spec, and still fail together if their
//      interface assumptions don't actually match (e.g. block A assumes
//      a FIFO's "almost full" threshold is 4 entries early, block B was
//      designed assuming 2 -- neither block-level testbench, testing
//      against its OWN spec in isolation, would ever catch this
//      mismatch).
//   2) Emergent/system-level behavior: things like end-to-end latency,
//      real traffic patterns across multiple blocks, resource
//      contention (two blocks competing for the same shared bus/memory
//      port), and system-level power states only exist once the blocks
//      are connected together -- there is no smaller unit to test them
//      at.
//   3) Real use-case validation: does the chip actually do what the
//      PRODUCT needs it to do, running a realistic workload/software
//      stack end to end? Block-level tests validate a block against ITS
//      spec, not that the specs of all the blocks TOGETHER add up to a
//      correct, useful system.
```

```
//
// Why you STILL want block-level verification (not just top-level):
//   1) Controllability & observability: at the block level you can
//      directly force any input, corner-case value, or error injection
//      that would be extremely hard (or nearly impossible) to steer to
//      from the chip's external pins once it's buried inside a large
//      system -- e.g. forcing a specific FIFO to be exactly one entry
//      from full requires deep, targeted control only available with
//      direct block-level access.
//   2) Speed and bug-finding efficiency: block-level testbenches
//      simulate MUCH faster (smaller design, no need to bring up
//      unrelated blocks) and can run vastly more directed + random
//      cycles in the same wall-clock time -- most bugs are cheaper to
//      find AND cheaper to DEBUG (smaller waveform, fewer signals,
//      clearer root cause) at the block level.
//   3) Parallelization across a team: different blocks can be verified
//      by different engineers/teams IN PARALLEL, long before the full
//      chip integration is even ready -- top-level verification can
//      only meaningfully start once enough of the chip exists to
//      connect together.
//
// The standard, practical methodology (bottom-up + top-down combined):
//   - Verify each block thoroughly in isolation first (deep, fast,
//     parallelizable -- catches the majority of bugs cheaply).
//   - Then verify at progressively larger levels of integration
//     (sub-system, then full chip) with LIGHTER, more directed/scenario-
//     based tests focused specifically on INTERFACES and end-to-end use
//     cases -- you don't need to re-run exhaustive random regression at
//     the top level for things already proven at the block level, but
//     you DO need dedicated tests for exactly the things that can ONLY
//     be exercised once blocks are connected.
//   - This is sometimes summarized as "verify depth at the block level,
//     verify breadth/integration at the top level."
```

解释

考点：这是一道开放性的验证方法论题目，面试官明确说了想听的是逻辑和思路，不是一个标准答案——回答的关键是能不能讲出"为什么只测小模块不够"和"为什么小模块测试依然有价值"这两个方向都要覆盖，而不是简单地选边站队说"只测顶层"或者"只测模块"。

核心论点：模块级验证和顶层验证抓的是两类完全不同的bug——模块级测的是"这个模块自己是否符合它自己的spec"，顶层测的是"所有模块拼在一起是否真的能协同工作、模块之间的接口假设是否一致"，这两类bug互相不能替代对方发现，模块级测试再彻底也测不出接口不匹配的问题，因为每个模块测试用的都是自己认为对的假设。

方法论的落地：实际工业界的标准做法是"模块级深度测试+顶层集成轻量测试"分层进行——模块级追求覆盖率和边界情况的穷尽，顶层追求端到端场景和接口正确性，两者投入的测试深度和侧重点不同，是互补关系而不是谁替代谁。

延伸考点：面试官可能会追问"如果时间/资源有限，应该优先投入模块级还是顶层"（通常建议是模块级要保证基本质量后再谨慎压缩顶层测试，因为顶层测试的调试成本远高于模块级——顶层出问题定位到具体哪个模块的哪个场景往往要花更久），以及UVM里怎么支持"模块级验证环境复用到顶层"这种需求（这正是virtual sequencer、UVM环境的分层可配置设计要解决的问题——同一套agent/monitor能同时用在模块级和顶层环境里）。

Q84. Verification: General Methodology for Verifying a Given FSM

QUESTION

Given a state-machine diagram to verify -- for example, state A can only transition to state C when a specific enable signal is asserted -- how do you approach verifying it? (Open-ended: the interviewer wants your reasoning/plan, not just a list of test names. Don't forget the reset signal.)

ANSWER

```
// General methodology for verifying ANY given FSM, organized as a
// checklist of REASONING categories (not just a fixed list of tests --
// this is the actual thought process to walk an interviewer through):
//
// 1) Enumerate every LEGAL transition, and test each one is REACHABLE
//    under its correct enabling condition.
//    e.g. A -> C requires the specific enable signal: drive that signal
//    active while in state A, confirm the FSM actually moves to C on
//    the expected clock edge.
//
// 2) For every legal transition, ALSO test the NEGATIVE case: the
//    transition must NOT happen when its condition is NOT met.
//    e.g. while in state A, hold the enable signal INACTIVE (across
//    multiple clock cycles, not just one sample) and confirm the FSM
//    stays in A (or takes whatever OTHER transition is legal without
//    the signal) -- never sneaks into C.  This is just as important as
//    testing the positive case, and is the one people forget most
//    often: a design can look correct on the "happy path" while still
//    having a decode bug that lets an illegal transition slip through.
//
// 3) Test every state is actually REACHABLE at all, and every legal
//    transition is actually EXERCISED at least once -- this is exactly
//    what FSM state coverage and transition coverage (covergroups on
//    the state register, or state/transition bins) are for. A test
//    plan without coverage goals can "look thorough" while silently
//    never visiting some corner state or transition.
//
// 4) RESET behavior, from EVERY state (not just from the initial/idle
//    state):
//    - Assert reset while sitting in EACH state (not only from A/idle)
//      and confirm the FSM always lands in the correct, defined initial
//      state -- a design might correctly reset from idle but have a
//      bug that only shows up resetting from some other specific state.
//    - Test reset asserted for exactly 1 cycle, for many cycles, and
//      right in the middle of an in-progress multi-cycle transition
//      (if any transitions take more than one cycle) -- confirm no
//      partial/corrupted state results.
//    - If reset is asynchronous, also test asserting/de-asserting it
//      with no relation to the clock edge (async assert/de-assert
//      timing), not just synchronized reset pulses.
//
// 5) ILLEGAL/undefined state recovery (especially relevant for
//    one-hot or gray-coded state encodings): if the state register
//    could ever glitch or be corrupted (e.g. an SEU/soft error in real
//    silicon, or simply an encoding that has more bit-patterns than
//    defined states) into a state that ISN'T one of the defined legal
//    states, does the FSM correctly recover (e.g. via a default case in
//    the RTL that forces it back to a safe/reset state), rather than
//    hanging or behaving unpredictably forever?
//
// 6) Concurrent/conflicting signal scenarios: if MULTIPLE transition
//    conditions could theoretically be true at the same time (e.g. two
//    different enable signals both asserted, each nominally pointing to
//    a different next state), confirm the FSM's actual PRIORITY/
//    arbitration behavior matches the spec -- this is exactly the kind
//    of corner case that's easy to miss with purely directed testing
//    and is a good candidate for constrained-random stimulus.
```

```
//
// 7) Timing of the enabling signal relative to the state itself: does
//    the enable need to be asserted for the FULL duration of state A,
//    or just sampled at one specific edge? Test both a signal that's
//    asserted for exactly one cycle at the right moment AND one that's
//    a longer pulse spanning multiple cycles, to nail down the exact
//    sampling semantics the design actually implements (and confirm it
//    matches what the spec intends -- if the spec is ambiguous on this
//    point, this is exactly the kind of question worth clarifying with
//    the designer up front, rather than guessing).
```

解释

考点：这是一道开放性的FSM验证方法论题，面试官想听到的是一套系统化的思考框架，而不是背出几个具体的测试用例名字——回答时最好按"正向覆盖、反向覆盖、覆盖率、复位、异常恢复、冲突场景、时序语义"这样分层次讲，展示的是思路的完整性。

最容易被忽略但很关键的两点：(1) 反向测试——很多人只测"条件满足时确实跳转"，容易忘记同样重要的"条件不满足时确实不跳转"，这个信号解码逻辑写错导致的bug往往就藏在反向测试里；(2) 复位要从每一个状态测，不能只测"从初始状态复位"这一种最简单的情况，题目里特意提醒"don't forget reset"正是在暗示这一点很容易被漏掉。

覆盖率驱动的重要性：单纯写一堆定向测试用例容易有遗漏的死角，用状态覆盖率和转移覆盖率（对状态寄存器和转移路径打covergroup）能客观地确认"是否真的走到了每一个状态、每一条合法转移"，而不是主观地觉得"应该测得差不多了"。

延伸考点：面试官很可能会追问"如果spec本身对某个转移条件的时序语义写得不清楚（比如使能信号到底要保持多久）你怎么办"，标准答案是主动去跟设计者澄清而不是自己猜一个假设去测——这是验证工程师非常重要的软技能，能主动提出这一点是明显的加分项；另外还可能追问非法状态恢复这一条在什么场景下特别重要（比如高可靠性/航天/汽车芯片对SEU容错有硬性要求，非法态恢复几乎是必测项）。

UVM

Q85. UVM: Starting Multiple Sequences on Multiple Sequencers in Parallel

QUESTION

You have 10 sequences (seq[0..9]) and 10 corresponding sequencers (seqr[0..9]). From the test, how do you start all 10 sequences IN PARALLEL, each on its own sequencer, and then wait for all of them to finish before proceeding (e.g. before dropping the run_phase objection)?

ANSWER

```
task run_phase(uvm_phase phase);
  phase.raise_objection(this);

  foreach (seq[i]) begin
    automatic int j = i;   // *** MUST capture the loop index locally ***
    fork
      seq[j].start(seqr[j]);
    join_none
  end

  wait fork;               // block here until ALL forked sequences finish
```

```
  phase.drop_objection(this);
endtask

// Why "automatic int j = i;" is required:
//   fork...join_none launches a NEW background process but does NOT
//   block the enclosing loop -- the loop keeps advancing immediately.
//   If the forked process referenced the loop variable 'i' DIRECTLY
//   (instead of a freshly captured per-iteration copy), all 10 forked
//   processes would share the SAME 'i' variable. By the time each
//   process actually gets scheduled and executes seq[i].start(...), the
//   loop may have already advanced 'i' to its final value (e.g. 10) --
//   so most or all forked processes would incorrectly end up starting
//   seq[9]/seqr[9] (or worse, an out-of-range index), not the sequence
//   that was "intended" for that loop iteration.
//   Declaring "automatic int j = i;" INSIDE the loop body creates a
//   brand-new instance of j on every iteration, snapshotting the
//   current value of i at that exact point -- so each forked process
//   correctly captures and uses its own distinct index.
//
// Why "wait fork;" (not join_none alone) at the end:
//   join_none returns control immediately without waiting for the
//   forked branch, which is exactly what we want inside the loop (so
//   all 10 sequences launch back-to-back without blocking each other).
//   But the ENCLOSING task/process still needs a way to know when ALL
//   of its OWN forked child processes (from ANY fork...join_none it
//   has spawned) have completed. "wait fork;" does exactly that -- it
//   blocks the current process until every outstanding child process it
//   has forked has finished, which is what lets run_phase safely drop
//   its objection only after all 10 sequences are truly done.
```

解释

考点：这是 UVM 里"并行跑多条 sequence"的标准写法，考察两个非常容易踩坑的细节，而不是 fork/join_none 语法本身。

坑1（变量捕获）：for 循环里直接 fork 一个引用循环变量的进程，是经典的"闭包变量共享"bug——所有 fork 出来的分支实际执行时很可能看到的都是循环结束后的同一个值，必须用 automatic 变量在每次循环内重新声明一份局部拷贝来"固定"当次的值。

坑2（等待方式）：join_none 只是让 fork 出去的分支不阻塞当前循环继续往下走，但如果最后不用 wait fork（或者用 disable fork 主动终止），run_phase 可能在10条 sequence 都还没跑完的时候就已经 drop_objection，导致仿真提前结束、部分 sequence 被截断。

延伸考点：面试官可能会追问 join_none / join / join_any 三者的区别，以及如果某条 sequence 里有 fatal 错误，是否会影响其他并行 sequence 的执行（通常不会互相阻塞，因为它们是独立的 fork 出来的进程，但共享的 scoreboard/环境状态可能会受影响，需要具体分析）。

Q86. UVM: Build Phase (Top-Down) vs Connect Phase (Bottom-Up) Execution Order

QUESTION

Given a simple two-level UVM hierarchy, test -> env (test is the parent, env is the child), each printing its own name inside build_phase() and connect_phase(). What is the print order for build_phase, and what is the print order for connect_phase? Explain WHY the two phases execute in opposite directions through the component hierarchy.

ANSWER

```
class my_env extends uvm_env;
  `uvm_component_utils(my_env)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    $display("BUILD:   env");
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    $display("CONNECT: env");
  endfunction
endclass

class my_test extends uvm_test;
  `uvm_component_utils(my_test)
  my_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    $display("BUILD:   test");
    env = my_env::type_id::create("env", this);   // env is CREATED here
  endfunction

  virtual function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    $display("CONNECT: test");
  endfunction
endclass

// OUTPUT:
//   BUILD:   test
//   BUILD:   env
//   CONNECT: env
//   CONNECT: test
//
// build_phase()   executes TOP-DOWN  (parent before child)
// connect_phase() executes BOTTOM-UP (child before parent)
//
// WHY build_phase is top-down:
//   build_phase's job is to CONSTRUCT the component hierarchy. A child
//   component literally does not exist as an object until its parent's
//   build_phase calls create() on it (as "env = my_env::type_id::create
//   (...)" does above) -- so the parent MUST run its build_phase first,
//   simply because it is the one responsible for instantiating the
//   child in the first place. Top-down order isn't a stylistic choice;
//   it's required by construction order.
//
// WHY connect_phase is bottom-up:
//   connect_phase's job is to wire up TLM ports/exports/analysis
//   connections between components. By the time it's a given
//   component's turn to run connect_phase, UVM guarantees that ALL of
//   its descendants have already finished their OWN connect_phase --
//   so a parent can safely reference ports/exports on its children that
//   are already fully wired up internally, rather than connecting to a
//   child sub-hierarchy that might still be half-connected. Every UVM
```

```
//   function phase EXCEPT build_phase runs bottom-up for this same
//   reason (connect, end_of_elaboration, start_of_simulation, extract,
//   check, report).
```

解释

考点：UVM phase 执行顺序里"只有 build_phase 是自顶向下，其余几乎所有 phase 都是自底向上"这个规则，是验证面试里非常高频的基础题，考的是理解为什么会这样设计，而不是死记顺序。

记忆逻辑：build_phase 负责"造子"，父类不跑 build_phase 子类根本不存在，所以必须父先跑；其余 phase（尤其是 connect_phase）负责"接线"，接线要保证子模块内部先接好，父模块接的时候才不会碰到"半成品"，所以必须子先跑完。

延伸考点：run_phase 是 task phase，跟 build/connect 这类 function phase 不一样——它是所有组件的 run_phase 同时并行启动（不是顺序执行），面试官经常会追问 task phase 和 function phase 在调度机制上的区别，以及 phase jumping、super.build_phase()/super.connect_phase() 忘记调用会有什么后果（子类没显式调用 super，可能导致某些基类里该做的初始化被跳过，是常见的低级bug来源）。

Q87. UVM: uvm_config_db -- set()/get() and Passing a Virtual Interface

QUESTION

What is uvm_config_db? What problem does it solve, and how do you use set()/get()? Give a short example of passing a virtual interface from the top-level testbench module into the UVM environment via config_db.

ANSWER

```
// uvm_config_db is UVM's parameterized, hierarchical configuration
// database -- a global key-value store used to pass configuration
// (virtual interfaces, config objects, plain int/bit knobs) from
// ANYWHERE in the testbench -- most commonly the static top-level
// module or the test class -- DOWN into components that may be
// arbitrarily deep in the dynamically-built UVM component hierarchy,
// WITHOUT manually threading constructor arguments through every
// level or resorting to global variables.
//
// It solves two related problems:
//   1) The static top-level module (where real interfaces/DUT live)
//      and the dynamic UVM class hierarchy (built at runtime) are two
//      separate "worlds" -- config_db is the standard bridge between
//      them.
//   2) Components shouldn't hardcode configuration values inside
//      themselves; config_db lets the TEST decide configuration and
//      push it down, keeping components reusable across different
//      tests/environments.

// --- top-level testbench module ---
module tb_top;
  logic clk, rst_n;
  my_if dut_if(clk, rst_n);          // a REAL interface instance,
                                     // connected to the DUT

  initial begin
    uvm_config_db#(virtual my_if)::set(
      null,                          // context: null = "outside the
                                     //   component hierarchy"
```

```
      "uvm_test_top.env.agent.*",      // hierarchical path (wildcard)
      "vif",                           // field name (key)
      dut_if                           // value: handle to the real if
    );
    run_test();
  end
endmodule

// --- inside the agent's driver ---
class my_driver extends uvm_driver #(my_txn);
  `uvm_component_utils(my_driver)
  virtual my_if vif;

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual my_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "virtual interface not found in config_db")
  endfunction
endclass

// Key mechanics:
//   - set() from OUTSIDE the component hierarchy (a module): context =
//     null, and you must give a full hierarchical path string.
//   - get() from INSIDE a component: context = this, path = "" (means
//     "resolve against my own instance path"), and UVM automatically
//     matches the most specific applicable set() call if more than one
//     set() could apply to this component (more precise/narrower paths
//     win over broader wildcard ones).
//   - get() returns 1 on success, 0 if nothing matched -- ALWAYS check
//     the return value and `uvm_fatal` if a required field is missing,
//     rather than silently continuing with an unset virtual interface.
```

解释

考点：uvm_config_db 是 UVM 环境搭建里最基础也最高频的考点，几乎每个UVM相关的面试都会问，核心是理解它解决的是"静态模块世界"和"动态类世界"之间传递信息的问题。

关键用法记忆点：从模块（静态世界）往里 set，context 填 null、路径要写全路径字符串；从组件（动态世界）内部 get，context 填 this、路径填空字符串（代表"匹配我自己的实例路径"）——这一对"里外"的用法差异是最容易搞混的地方。

易错点：get() 有返回值，返回0说明没取到，必须显式检查并且 fatal 出来，很多人写代码时忽略返回值，虚接口没取到时 vif 保持默认值（通常是null），运行到driver驱动信号那一步才会报空指针错误，排查起来比在build_phase直接报错麻烦得多。

延伸考点：面试官经常会追问如果多个 set() 的路径都能匹配到同一个组件，UVM 怎么决定用哪一个（路径更具体的优先级更高，且按set()调用的顺序也有影响，具体规则建议结合文档回答"更精确的路径覆盖更宽泛的路径"这个核心原则），以及 config_db 和更早版本UVM里 set_config_* / get_config_* 这类过时API的关系（config_db是现在的标准替代方案）。

Q88. UVM: What is a Virtual Interface, and Why Is It Needed

QUESTION

What is a "virtual interface" in SystemVerilog/UVM, and why is it needed? Why can't a UVM class-based driver simply instantiate and use a regular `interface` directly?

ANSWER

```
// A plain `interface` in SystemVerilog is a STATIC, module-like
// construct: it must be instantiated at elaboration time, inside the
// static structural module hierarchy, right alongside the DUT. It is
// NOT a class -- it cannot be dynamically created, destroyed, or
// passed around as an object/handle the way a class instance can.
//
// UVM testbench components (drivers, monitors, scoreboards, etc.) are
// CLASSES -- dynamic objects, created at simulation runtime, living in
// a completely separate "world" (the class/object hierarchy) from the
// static module hierarchy where the DUT and its physical interfaces
// actually live.
//
// A `virtual interface` is a special variable TYPE that acts as a
// HANDLE (pointer-like reference) to an interface INSTANCE that
// already exists somewhere in the static module hierarchy. It is the
// bridge that lets a dynamic class object reach across into the static
// world and drive/sample real DUT-connected signals -- without a
// virtual interface, a class has literally no mechanism to touch a
// physical interface instance at all.
//
// Typical flow:
//   1. The top-level testbench module instantiates the REAL interface,
//      connected to the DUT's pins.
//   2. The top module passes a VIRTUAL interface handle pointing at
//      that real instance into the UVM hierarchy via uvm_config_db
//      (see the companion config_db question).
//   3. A driver/monitor class declares "virtual my_if vif;", retrieves
//      it via uvm_config_db::get() in build_phase.
//   4. From then on the class uses vif.signal_name to drive or sample
//      DUT pins directly, e.g. inside run_phase:
//          vif.data  <= txn.data;
//          vif.valid <= 1'b1;
//          @(posedge vif.clk);
```

解释

考点：虚接口（virtual interface）是UVM里"类的世界"和"模块的世界"之间唯一的桥梁，这道题考的是理解为什么需要它，而不只是背"vif就是接口的指针"这句话。

核心原因：interface 本质上是静态的结构化实体，必须在仿真一开始的模块例化阶段就摆在结构层次里，和DUT在一起；UVM组件是运行时动态new出来的类对象，两者活在两个不同的"世界"里，类对象没有任何原生机制能直接摸到模块层次里的东西——虚接口就是专门设计出来打通这两个世界的句柄类型。

标准使用流程：顶层module例化真实interface连到DUT，通过config_db把虚接口句柄传进UVM环境，driver/monitor在build_phase里get到这个句柄，之后就能像操作普通接口信号一样通过vif.信号名去驱动/采样DUT引脚。

延伸考点：面试官经常会追问如果忘记在config_db里set虚接口会发生什么（vif保持默认值null，driver运行到真正驱动信号那一步会因为空指针解引用而报错或者仿真直接崩溃，而不是在build_phase就能发现，所以强烈建议get失败时立刻uvm_fatal），以及虚接口和普通接口在语法上具体怎么声明使用（本质就是在类里把 virtual 关键字加在 interface 类型前面）。

Q89. UVM: Verification Environment Architecture -- What Each Component Does

QUESTION

Describe how you would set up a UVM verification environment for a design, and explain what each major component does: driver, monitor, sequencer, agent, scoreboard, environment, and test.

ANSWER

```
// Typical hierarchy (top -> bottom):
//
//   uvm_test
//     `-- uvm_env
//           |-- uvm_agent (active)
//           |     |-- uvm_sequencer
//           |     |-- uvm_driver
//           |     `-- uvm_monitor
//           |-- uvm_agent (passive, optional -- e.g. a second bus to watch)
//           |     `-- uvm_monitor
//           `-- scoreboard (uvm_component / uvm_scoreboard)
//
// Component responsibilities:
//
//   uvm_test        -- top of the hierarchy; selects and configures which
//                      environment/agents to build (often via
//                      uvm_config_db), and kicks off the top-level
//                      sequence(s) in run_phase. Different tests reuse
//                      the SAME env with different configuration/
//                      sequences -- this is the main point of separating
//                      test from env.
//
//   uvm_env         -- the container: instantiates the agent(s),
//                      scoreboard, and any environment-level components
//                      (coverage collector, reference model). Wires
//                      everything together in connect_phase.
//
//   uvm_agent       -- bundles one driver + one monitor + one sequencer
//                      for ONE DUT interface. Can be ACTIVE (drives AND
//                      monitors -- normal case) or PASSIVE (monitor
//                      only, no driver/sequencer -- used for an
//                      interface you only need to observe, not drive,
//                      e.g. watching a bus you don't control).
//
//   uvm_sequencer   -- arbitrates between sequences and feeds sequence
//                      items (transactions) to the driver, one at a
//                      time, via a TLM seq_item port/export handshake.
//
//   uvm_driver      -- pulls transactions from the sequencer
//                      (get_next_item / item_done) and drives them onto
//                      the DUT's actual pins through a virtual
//                      interface -- this is the ONLY component that
//                      does cycle-accurate pin wiggling on the input
//                      side.
//
//   uvm_monitor     -- PASSIVELY observes DUT pins (through a virtual
//                      interface, typically read-only) without ever
//                      driving anything, reconstructs transactions from
//                      the pin-level activity, and broadcasts them
//                      through an analysis port to whoever is
//                      subscribed (scoreboard, coverage collector).
//
//   scoreboard      -- receives transactions from monitor(s) via
//                      analysis exports, compares actual DUT behavior
//                      against an expected/reference model, and flags
//                      mismatches (`uvm_error). This is where
//                      correctness is actually CHECKED, as opposed to
//                      just stimulated.
```

解释

考点：UVM环境的标准分层结构和每个组件的职责边界，是验证方向面试几乎必问的基础题，考察点是能不能讲清楚"为什么要分这么多层"，而不只是背组件名字。

分层的核心价值：test和env分离让同一套env能被不同test复用（只改配置和跑的sequence，不用重新搭环境）；driver和monitor分离让"往DUT里写"和"从DUT里看"是两条独立路径，driver只管照着sequence给的transaction驱动信号，monitor只管被动观察不参与驱动，两者互不干扰，也方便monitor被复用在被动的观察场景（比如passive agent）。

scoreboard的定位容易被问混：它不是"生成激励"的，是"检查结果"的，激励生成是sequence/driver的事，scoreboard只负责比对期望和实际，这个职责边界要分清楚。

延伸考点：面试官可能会追问 driver 和 sequencer 之间具体的握手机制（get_next_item/item_done，以及为什么要有这个握手而不是sequencer直接把transaction塞给driver），以及 passive agent
具体的应用场景（比如DUT旁边有一条你不需要控制、只需要观察是否符合协议的总线）。

Q90. UVM: Using an Associative Array in a Scoreboard

QUESTION

How would you use a SystemVerilog associative array inside a scoreboard -- for example, to check the SRAM described in the companion test-plan question? Show a simple implementation.

ANSWER

```
class sram_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(sram_scoreboard)

  // associative array: key = address, value = expected data.
  // Sparse by nature -- a 32-bit address space has up to 2^32 possible
  // keys, but the test only ever touches a small, unpredictable subset
  // of them, so a dense array would be both impossible to allocate and
  // wasteful; an associative array only stores entries you actually use.
  bit [31:0] expected_mem [bit [31:0]];

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void write_txn(mem_txn t);
    if (t.wr_rd) begin
      // WRITE: update the reference model
      expected_mem[t.addr] = t.data;
    end else begin
      // READ: check against the reference model
      if (!expected_mem.exists(t.addr)) begin
        `uvm_warning("SB", $sformatf(
          "read from an address never written: 0x%0h", t.addr))
      end else if (expected_mem[t.addr] !== t.data) begin
        `uvm_error("SB", $sformatf(
          "MISMATCH addr=0x%0h expected=0x%0h got=0x%0h",
          t.addr, expected_mem[t.addr], t.data))
      end
    end
  endfunction

  // useful associative-array methods along the way:
  //   .exists(key)      -- does this key have an entry? (used above)
  //   .delete(key)       -- remove one entry (or .delete() with no args
  //                         to clear the whole array)
```

```
  //   .num() / .size()   -- how many entries currently exist
  //   foreach (expected_mem[addr]) ...  -- iterate all touched addresses,
  //                           e.g. for an end-of-test leftover/coverage
  //                           report of every address the test exercised
endclass
```

解释

考点：关联数组（associative array）在scoreboard里的典型用法，是"用数据结构解决验证问题"这一类题的代表——考察点不是关联数组的语法本身（这个很基础），而是知道为什么这个场景选它而不是普通数组。

选型理由：地址空间通常很大（比如32位地址有40多亿种可能），但一次测试真正会碰到的地址往往只是其中很小的一部分，普通数组要么开不出那么大的空间，要么开小了又可能越界，关联数组按需分配、只存实际用到的key，天然适合这种"地址稀疏"的场景。

exists()的价值：能区分"这个地址从没被写过就被读"（很可能是个bug，比如测试序列有问题，或者是DUT真的返回了不该返回的数据）和"这个地址被写过、现在读回来的值不对"（数据本身出错），这两种失败模式如果不用exists()做区分，日志里会很难一眼看出问题出在哪一类。

延伸考点：面试官可能会追问关联数组和动态数组（dynamic array）的区别和各自适用场景（动态数组适合"连续下标、大小运行时确定但地址空间本身不稀疏"的场景，关联数组适合"下标稀疏或者根本不是连续整数"的场景，比如按地址、按事务ID做索引），以及大规模随机测试跑很久之后关联数组内存持续增长要不要清理（视测试策略而定，长时间跑的测试可能需要定期清理已经确认无误的旧entry，避免内存无限增长）。

Q91. UVM: Starting Sequences on Multiple Sequencers Sequentially (via foreach)

QUESTION

You have 5 sequences seq[0..4] and 5 corresponding sequencers seqr[0..4]. Write the code to start each sequence on its matching sequencer ONE AT A TIME (sequentially -- seq[1] does not start until seq[0] has fully finished), using a foreach loop. (The PARALLEL version of this same scenario -- using fork + an automatic loop-index capture + wait fork -- is covered in a companion question; no need to repeat that mechanism here.)

ANSWER

```
task run_all_sequential();
  foreach (seq[i]) begin
    seq[i].start(seqr[i]);   // BLOCKS here until seq[i] fully completes,
                               // only then does the loop move on to i+1
  end
endtask

// Why this is simpler than the parallel version, and why the
// "automatic int j = i" capture trick from the parallel version is NOT
// needed here:
//   seq[i].start(seqr[i]) is a BLOCKING task call -- it does not return
//   control to the loop until that sequence has actually finished
//   running. That means the loop body fully consumes the current value
//   of i (by calling start() and waiting for it to return) BEFORE the
//   loop ever advances i to the next value. There is no background
//   process capturing a stale/shared copy of i, because there IS no
//   background process here at all -- everything happens synchronously,
//   in-line, one iteration fully completing before the next begins.
//   The capture bug in the parallel version exists specifically BECAUSE
```

```
//   fork...join_none lets the loop keep advancing i WHILE the forked
//   branches are still pending -- that situation simply doesn't arise
//   in a plain, un-forked foreach loop.
```

解释

考点：这是"顺序跑多条sequence"的基础写法，跟"并行跑多条sequence"（配套的另一道题）放在一起考，目的是让你分清两者的本质区别，而不只是死记两段代码。

关键理解：seq[i].start(seqr[i]) 本身是阻塞式调用，不加fork的话，foreach循环天然就是顺序执行——这也是为什么这里完全不需要automatic变量去"固定"循环下标：根本没有后台进程会在循环变量变化之后才去读取它，每次循环i的值在被用掉的那一刻就已经完成了它的使命。

延伸考点：面试官问完顺序版本几乎一定会追问"如果要并行呢"，这时候要能清楚地说出"因为改成了fork+join_none，循环不再等待，i会在后台进程真正执行之前继续变化，所以必须用automatic变量重新声明一份局部拷贝"——顺序版本和并行版本放在一起对比着讲，是回答这类问题最容易讲清楚、也最容易让面试官满意的方式。

Q92. UVM: PLL Sequence Item / Sequence / Driver, and a Two-Queue Scoreboard

QUESTION

For the 3-stage clock-divider/PLL chain above (control signals en, sel, and coefficients M, N, P; clk_in and clk_out are the physical clocks), write the UVM sequence item, sequence, and driver skeleton needed to apply a randomized (M,N,P) configuration to the DUT. Then describe a scoreboard architecture for this DUT, given TWO separate input monitors (one for the configuration bus, one for clk_in) and ONE output monitor (for clk_out) -- explain why the scoreboard needs two separate queues on the input side.

ANSWER

```
// --- sequence item ---
class pll_cfg_item extends uvm_sequence_item;
  `uvm_object_utils(pll_cfg_item)

  rand bit       en;
  rand bit       sel;
  rand int unsigned M;    // divider, stage A
  rand int unsigned N;    // multiplier, stage B
  rand int unsigned P;    // divider, stage C

  constraint c_M { M inside {[1:63]};  }
  constraint c_N { N inside {[1:255]}; }
  constraint c_P { P inside {[1:63]};  }

  function new(string name = "pll_cfg_item");
    super.new(name);
  endfunction
endclass

// --- sequence ---
class pll_cfg_seq extends uvm_sequence #(pll_cfg_item);
  `uvm_object_utils(pll_cfg_seq)

  function new(string name = "pll_cfg_seq");
    super.new(name);
  endfunction
```

```
  virtual task body();
    pll_cfg_item item;
    item = pll_cfg_item::type_id::create("item");
    start_item(item);
    assert(item.randomize());
    finish_item(item);
  endtask
endclass

// --- driver ---
class pll_driver extends uvm_driver #(pll_cfg_item);
  `uvm_component_utils(pll_driver)
  virtual pll_if vif;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual pll_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "virtual interface not found")
  endfunction

  virtual task run_phase(uvm_phase phase);
    forever begin
      pll_cfg_item item;
      seq_item_port.get_next_item(item);
      vif.en  <= item.en;
      vif.sel <= item.sel;
      vif.M   <= item.M;
      vif.N   <= item.N;
      vif.P   <= item.P;
      @(posedge vif.clk_in);          // let the config settle for 1 cycle
      seq_item_port.item_done();
    end
  endtask
endclass

// --- scoreboard architecture ---
// Two INPUT monitors:
//   1) cfg_monitor  -- watches en/sel/M/N/P, pushes an entry every time a
//      new configuration is applied
//   2) clk_in_monitor -- watches clk_in edges, measures the ACTUAL input
//      frequency being fed to the PLL right now
// One OUTPUT monitor:
//   3) clk_out_monitor -- watches clk_out edges, measures the actual
//      output frequency
//
// WHY TWO SEPARATE QUEUES ON THE INPUT SIDE (not one combined queue):
//   A PLL has SETTLING TIME -- clk_out does not instantly reflect a new
//   (M,N,P) configuration; it takes some number of cycles to lock. The
//   scoreboard cannot naively pair "the Nth config applied" with "the
//   Nth clk_out sample" 1:1 in arrival order, because config changes and
//   clk_in edges are asynchronous, DIFFERENT-RATE event streams (a
//   config might be applied once while clk_in ticks hundreds of times
//   before clk_out reflects it). Keeping them in SEPARATE queues lets
//   the scoreboard:
//     - look up "what was the most recently applied (M,N,P) as of a
//       given point in time" from the cfg queue,
//     - independently confirm clk_in's measured frequency matches the
//       expected 100 MHz from the clk_in queue (sanity-checking the
//       INPUT itself, not just trusting it),
```

```
//     - and only THEN compute the expected clk_out frequency
//       (freq_in * N / (M*P)) and compare it against what the output
//       monitor measured, once enough settling time has passed.
//   Merging everything into one queue would destroy this ability to
//   independently correlate "what config was active" against "what the
//   input actually was" against "what came out," which are three
//   logically separate facts needed to compute one expected result.
```

解释

考点：这是一道综合题，把PLL的具体电路场景和UVM组件设计结合在一起考，前半段（seq_item/sequence/driver）是标准模板套用，后半段（scoreboard架构）才是真正的难点和区分度所在。

scoreboard设计的核心难点：PLL锁定需要时间，配置事务流和时钟边沿流是两条完全独立、速率不同的异步事件流，不能简单假设"第N个配置对应第N个输出采样"，必须分开维护队列，各自独立记录"什么时候应用了什么配置"和"输入时钟实际测得的频率"，再结合PLL的黄金公式（out = in × N/(M×P)）和一个合理的锁定延迟窗口，才能正确判定某一次clk_out的测量结果是否符合预期。

延伸考点：面试官很可能追问"锁定时间怎么确定"（通常是DUT spec里给出的一个规格值，或者通过观察clk_out频率连续多少个周期稳定不变来判定已经锁定），以及如果多个配置连续快速写入（PLL还没锁定就又换了新配置）scoreboard该怎么处理（这种情况下期望行为本身就需要跟设计spec确认，比如是否需要等待锁定后才允许换配置，这是一个很好的主动澄清需求的追问点）。

Q93. UVM: Base/Derived Handle Assignment vs Factory Type Override

QUESTION

In plain SystemVerilog OOP: given class base and class derived extends base, with "base b; derived d; d = new(); b = d;" -- explain what this assignment does and why it's legal (and why the reverse, "d = b;", is NOT legal without an explicit cast). Then explain the UVM mechanism that achieves an ANALOGOUS effect at the environment level: transparently substituting a derived type wherever a base type was originally going to be constructed, WITHOUT editing the existing component code.

ANSWER

```
// Plain SV OOP: "b = d;" is a legal UPCAST.
//   base b;
//   derived d;
//   d = new();
//   b = d;        // legal: derived IS-A base (inheritance), so a base-typed
//                  // handle can always safely point at a derived object.
//   Through 'b', you can only directly call methods/access members that
//   exist in 'base' -- UNLESS those methods are declared 'virtual' in
//   base and overridden in 'derived', in which case calling them via 'b'
//   still correctly dispatches to derived's override (polymorphism).
//
//   The REVERSE, "d = b;", is NOT legal without an explicit cast, because
//   the compiler cannot guarantee at compile time that 'b' actually
//   points at a 'derived' object (it might point at a plain 'base'
//   instance, or some OTHER class extending base) -- you must use
//   $cast(d, b) to attempt this DOWNCAST, which checks the actual
//   run-time type and fails gracefully (returns 0) if it doesn't match.
//
// UVM equivalent: FACTORY OVERRIDE.
//   Instead of manually reassigning handles, UVM lets you tell the
//   FACTORY itself: "whenever anything asks to create() an object of
//   type base_X, actually construct a derived_X instead." Every
```

```
//   existing ::type_id::create() call anywhere in the environment
//   stays completely unchanged -- the substitution happens silently,
//   at construction time, driven entirely by factory configuration set
//   from the TEST.

class base_driver extends uvm_driver #(my_txn);
  `uvm_component_utils(base_driver)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass

class derived_driver extends base_driver;
  `uvm_component_utils(derived_driver)
  // e.g. adds extra protocol checking, or injects errors for a
  // negative test -- functionality the base driver doesn't have
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass

class my_test extends uvm_test;
  `uvm_component_utils(my_test)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    // TYPE override: EVERY place in the entire env hierarchy that would
    // have created a base_driver via type_id::create() now gets a
    // derived_driver instead -- no other file needs to change.
    set_type_override_by_type(base_driver::get_type(), derived_driver::get_type());

    // or, to override only ONE specific instance path instead of every
    // instance of the type:
    // set_inst_override_by_type("env.agent.drv",
    //                           base_driver::get_type(), derived_driver::get_type());
    super.build_phase(phase);   // (env gets built here, using the override above)
  endfunction
endclass

// Why this works: the env/agent code creates its driver with
//   drv = base_driver::type_id::create("drv", this);
// -- it NEVER says "new base_driver(...)" directly. Because creation is
// always routed through the factory (::type_id::create), the factory is
// free to substitute a different, registered type at that exact call
// site, which is exactly what makes override-based test customization
// possible without touching env/agent source code.
```

解释

考点：这道题把"面向对象基础"（向上转型合法、向下转型需要显式cast）和"UVM factory override"两个知识点连起来问，考察你能不能看出两者背后是"同一个思想在不同层面的应用"——都是"用一个更具体的类型去替代一个更抽象的类型，同时让使用方代码保持不变"。

关键类比：普通OOP里，b=d这行代码是"手动"完成的一次性替换，作用范围就是这一个变量；UVM factory override是"声明式"的，你只在test里配置一次"以后创建base_driver统统换成derived_driver"，环境里成百上千处调用type_id::create()的代码全部自动生效，不需要逐个去改——这正是为什么UVM强制要求所有组件用type_id::create()而不是new()来构造：new()是硬编码的具体类型，没办法被factory拦截替换。

延伸考点：面试官经常会追问 set_type_override_by_type 和 set_inst_override_by_type 的区别（前者影响所有实例，后者只影响指定路径的那一个实例），以及为什么override要在build_phase里、并且要在真正create之前调用（因为override配置本身也是"先设置规则、后面创建的时候才生效"的声明式机制，如果创建动作已经发生了再设置override，那次创建已经错过了替换的时机）。

Q94. UVM: How a Sequence Item Travels From Sequence to Driver

QUESTION

Describe, mechanically, how a sequence_item travels from a sequence to a driver in UVM.

ANSWER

```
// The sequence side:
class my_seq extends uvm_sequence #(my_txn);
  `uvm_object_utils(my_seq)
  function new(string name = "my_seq");
    super.new(name);
  endfunction

  virtual task body();
    my_txn item;
    item = my_txn::type_id::create("item");

    start_item(item);              // (1)
    assert(item.randomize());      // (2)
    finish_item(item);              // (3)
  endtask
endclass

// The driver side:
class my_driver extends uvm_driver #(my_txn);
  `uvm_component_utils(my_driver)
  virtual task run_phase(uvm_phase phase);
    forever begin
      my_txn item;
      seq_item_port.get_next_item(item);   // (4)
      drive_transaction(item);              // (5) -- actual pin wiggling
      seq_item_port.item_done();            // (6)
    end
  endtask
endclass

// Step-by-step mechanics:
//
//  (1) start_item(item): the sequence requests permission to send this
//      item. It BLOCKS until the SEQUENCER grants it -- if multiple
//      sequences are running in parallel on the same sequencer, the
//      sequencer's arbiter decides which one goes next (default is
//      FIFO order, but this is configurable).
//
//  (2) Between start_item and finish_item is where you typically
//      randomize (or otherwise fill in) the item's fields -- this
//      window exists specifically so a parent/virtual sequence can
//      modify the item AFTER the grant but BEFORE it's actually sent,
//      if needed (e.g. via a pre_do() callback).
//
//  (3) finish_item(item): actually SENDS the item across the TLM
//      connection to the sequencer, and BLOCKS the sequence until the
//      driver signals it has consumed the item (via item_done() below).
```

```
//
//  (4) seq_item_port.get_next_item(item): the driver, on its
//      seq_item_port (connected to the sequencer's seq_item_export in
//      connect_phase), BLOCKS here until a sequence has an item ready
//      (i.e. until some sequence's finish_item() call delivers one).
//      This call POPULATES 'item' with the transaction the sequencer
//      just handed over.
//
//  (5) The driver does its real job: drives the item's fields onto the
//      DUT's pins through a virtual interface, over however many clock
//      cycles the protocol actually requires -- this is NOT
//      instantaneous, it's the bulk of the driver's run_phase logic.
//
//  (6) seq_item_port.item_done(): tells the sequencer (and, through it,
//      the waiting sequence's finish_item() call) that this item has
//      been fully consumed. This is what UNBLOCKS the sequence's
//      finish_item(), letting the sequence proceed to build/send its
//      NEXT item. item_done() can optionally carry a response item back
//      to the sequence (get_response() on the sequence side) if the
//      protocol needs to report something back (e.g. read data).
//
// The whole thing is a strict, blocking handshake in both directions:
// start_item/get_next_item pairs up "a sequence has something to send"
// with "the driver is ready for it," and finish_item/item_done pairs up
// "the driver has consumed it" with "the sequence may proceed."
```

解释

考点：这是UVM sequence/driver握手机制的核心问题，几乎所有UVM相关面试都会直接或间接问到，考察点是能不能把start_item/finish_item和get_next_item/item_done这两对API的配对关系和阻塞语义讲清楚，而不只是背出这几个函数名。

关键理解：这是一个双向阻塞的握手协议——sequence端的start_item等sequencer仲裁授权，finish_item等driver真正消费完；driver端的get_next_item等sequencer送来新item，item_done告诉sequencer/sequence这一条已经处理完了。四个调用严格配对，任何一边忘了调用对应的另一半，另一边就会永久阻塞（比如driver忘了调用item_done，sequence的finish_item会一直卡住，测试直接挂死）。

易错点：很多人会以为item是通过某种"直接传递"的方式从sequence给到driver，实际上中间隔着sequencer这一层，sequencer既负责多个并行sequence之间的仲裁（谁先发送），也是这个握手协议真正的TLM连接点——driver的seq_item_port是连到sequencer的seq_item_export，而不是直接连到某个具体的sequence。

延伸考点：面试官可能会追问start_item和finish_item之间可以做什么（这正是随机化、以及parent sequence通过pre_do/mid_do/post_do回调插手修改item内容的地方），以及item_done()带响应参数的用法（比如读操作需要把读到的数据通过response传回给发起请求的sequence，sequence那边用get_response()接收）。

Q95. UVM: Design Monitor for Serial 4-bit Interface with Start/Stop Framing Patterns

QUESTION

Implement monitor for serial nibble (4-bit) interface with framing protocol. Start delimiter: 0xABCD (four consecutive nibbles 0xA, 0xB, 0xC, 0xD). Stop delimiter: 0xCAFE (four consecutive nibbles 0xC, 0xA, 0xF, 0xE). Monitor detects start pattern, captures all payload nibbles until stop pattern detected, then publishes complete payload as single transaction via analysis port. Handle partial patterns, false starts, and nested patterns.

ANSWER

```
// Framing Monitor ABCD->CAFE
class framing_monitor extends uvm_monitor;
  `uvm_component_utils(framing_monitor)

  uvm_analysis_port #(frame_txn) ap;
  virtual frame_if vif;

  typedef enum {HUNT, SYNC} state_e;
  state_e state;
  byte unsigned buffer[$];

  task run_phase(uvm_phase phase);
    state = HUNT;
    forever begin
      @(posedge vif.clk);
      if (vif.data_valid) begin
        case (state)
          HUNT: begin
            buffer.push_back(vif.data);
            if (buffer.size() >= 4) begin
              // Check for ABCD marker (start of frame)
              if (buffer[$-3] == 8'hAB && buffer[$-2] == 8'hCD) begin
                state = SYNC;
                buffer = {};
              end else begin
                buffer.pop_front();
              end
            end
          end
          SYNC: begin
            buffer.push_back(vif.data);
            // Check for CAFE end marker
            if (buffer.size() >= 2 &&
                buffer[$-1] == 8'hCA && buffer[$] == 8'hFE) begin
              frame_txn t = frame_txn::type_id::create("frame");
              t.payload = new[buffer.size() - 2];
              foreach (t.payload[i]) t.payload[i] = buffer[i];
              ap.write(t);
              buffer = {};
              state = HUNT;
            end
          end
        endcase
      end
    end
  endtask
endclass
```

解释

考点：这是UVM testbench组件设计题，重点是组件的职责边界和与其他组件的接口是否设计合理。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q96. UVM: Design Scoreboard Using uvm_tlm_analysis_fifo for Timing Decoupling

QUESTION

Implement scoreboard that uses uvm_tlm_analysis_fifo to decouple monitor sampling timing from scoreboard checking logic. Monitor writes transactions to FIFO via analysis port. Scoreboard checking thread

gets transactions from FIFO at its own pace using get() blocking call. FIFO buffers transactions, allowing monitor and checker to run independently. Handles burst traffic and timing variations.

ANSWER

```
// Analysis FIFO Scoreboard
class fifo_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(fifo_scoreboard)

  uvm_analysis_fifo #(my_txn) expected_fifo;
  uvm_analysis_fifo #(my_txn) actual_fifo;

  // Export ports for external connection
  uvm_analysis_export #(my_txn) expected_export;
  uvm_analysis_export #(my_txn) actual_export;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    expected_fifo   = new("expected_fifo", this);
    actual_fifo     = new("actual_fifo", this);
    expected_export = new("expected_export", this);
    actual_export   = new("actual_export", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    expected_export.connect(expected_fifo.analysis_export);
    actual_export.connect(actual_fifo.analysis_export);
  endfunction

  task run_phase(uvm_phase phase);
    my_txn exp_txn, act_txn;
    forever begin
      expected_fifo.get(exp_txn);
      actual_fifo.get(act_txn);

      if (!exp_txn.compare(act_txn))
        `uvm_error("SCB", $sformatf("Mismatch: exp=%s act=%s",
                   exp_txn.sprint(), act_txn.sprint()))
      else
        `uvm_info("SCB", "Match", UVM_HIGH)
    end
  endtask
endclass
```

解释

考点：这是UVM testbench组件设计题，重点是组件的职责边界和与其他组件的接口是否设计合理。

FIFO类设计的核心永远是满/空判断和读写指针的正确同步，是这道题的关键分数点。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q97. UVM: Implement Driver that Splits 64-bit Transaction into Two 32-bit Beats

QUESTION

Design UVM driver for 32-bit data interface that receives 64-bit transactions from sequencer and splits each into two 32-bit beats. First beat transmits lower 32 bits [31:0], second beat transmits upper 32 bits [63:32]. Handle ready/valid handshake with backpressure. Support configurable inter-beat gap. Call item_done() only after both beats transmitted.

ANSWER

```
// Split-Beat Driver
class split_beat_driver extends uvm_driver #(burst_txn);
  `uvm_component_utils(split_beat_driver)

  virtual bus_if vif;

  task run_phase(uvm_phase phase);
    forever begin
      burst_txn txn;
      seq_item_port.get_next_item(txn);
      drive_burst(txn);
      seq_item_port.item_done();
    end
  endtask

  task drive_burst(burst_txn txn);
    // Drive address phase
    @(posedge vif.clk);
    vif.addr   <= txn.addr;
    vif.len    <= txn.beat_count - 1;
    vif.valid  <= 1'b1;

    // Wait for address accept
    @(posedge vif.clk iff vif.ready);
    vif.valid <= 1'b0;

    // Drive data beats
    for (int i = 0; i < txn.beat_count; i++) begin
      @(posedge vif.clk);
      vif.wdata  <= txn.data[i];
      vif.wvalid <= 1'b1;
      vif.wlast  <= (i == txn.beat_count - 1);
      @(posedge vif.clk iff vif.wready);
    end
    vif.wvalid <= 1'b0;
    vif.wlast  <= 1'b0;
  endtask
endclass
```

解释

考点：这是UVM testbench组件设计题，重点是组件的职责边界和与其他组件的接口是否设计合理。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q98. UVM: Implement Driver with Response Path Back to Sequencer

QUESTION

Design UVM driver that not only drives stimulus to DUT but also returns response transactions back to sequencer. After completing request on DUT, driver creates response transaction, populates with DUT response data, and sends to sequencer using item_done(rsp). Sequence retrieves response using get_response(rsp) and can use response data for future requests (reactive sequence).

ANSWER

```
// Driver Response Path
class resp_driver extends uvm_driver #(my_txn, my_txn);
  `uvm_component_utils(resp_driver)

  virtual bus_if vif;

  task run_phase(uvm_phase phase);
    forever begin
      my_txn req, rsp;
      seq_item_port.get_next_item(req);

      // Drive request
      drive_request(req);

      // Collect response from DUT
      rsp = my_txn::type_id::create("rsp");
      rsp.set_id_info(req); // copy sequence/transaction IDs
      collect_response(rsp);

      // Send response back to sequence
      seq_item_port.item_done(rsp);
    end
  endtask

  task drive_request(my_txn req);
    @(posedge vif.clk);
    vif.addr  <= req.addr;
    vif.data  <= req.data;
    vif.valid <= 1'b1;
    @(posedge vif.clk iff vif.ready);
    vif.valid <= 1'b0;
  endtask

  task collect_response(my_txn rsp);
    @(posedge vif.clk iff vif.resp_valid);
    rsp.data   = vif.resp_data;
    rsp.status = vif.resp_status;
  endtask
endclass
```

解释

考点：这是UVM testbench组件设计题，重点是组件的职责边界和与其他组件的接口是否设计合理。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Algorithms & Scripting

Q99. Algorithms: Rearrange a Sorted Array in Alternating Max-Min Order

QUESTION

Given a sorted array of positive integers, rearrange it alternately: the first element should be the maximum value, the second the minimum, the third the second-maximum, the fourth the second-minimum, and so on.

Example 1:

Input: arr = [1, 2, 3, 4, 5, 6, 7]

Output: arr = [7, 1, 6, 2, 5, 3, 4]

Example 2:

Input: arr = [1, 2, 3, 4, 5, 6]

Output: arr = [6, 1, 5, 2, 4, 3]

Write a correct, clean solution and state its time/space complexity.

ANSWER

```
def rearrange_max_min(arr):
    # arr is assumed already SORTED in ascending order.
    result = []
    left, right = 0, len(arr) - 1

    while left < right:
        result.append(arr[right])   # next largest remaining
        result.append(arr[left])    # next smallest remaining
        left  += 1
        right -= 1

    if len(arr) % 2 == 1:
        # odd length: one element is left over exactly in the middle
        result.append(arr[left])

    return result

// Trace, Example 1: arr = [1,2,3,4,5,6,7]  (n=7, odd)
//   left=0,right=6: append 7, append 1 -> [7,1]           left=1,right=5
//   left=1,right=5: append 6, append 2 -> [7,1,6,2]        left=2,right=4
//   left=2,right=4: append 5, append 3 -> [7,1,6,2,5,3]    left=3,right=3
//   left<right fails (3<3 false) -> loop ends
//   n odd -> append arr[left]=arr[3]=4 -> [7,1,6,2,5,3,4]  matches expected

// Trace, Example 2: arr = [1,2,3,4,5,6]  (n=6, even)
//   left=0,right=5: append 6, append 1 -> [6,1]            left=1,right=4
//   left=1,right=4: append 5, append 2 -> [6,1,5,2]        left=2,right=3
//   left=2,right=3: append 4, append 3 -> [6,1,5,2,4,3]    left=3,right=2
//   left<right fails (3<2 false) -> loop ends
//   n even -> no leftover element to append              -> matches expected

// Complexity:
//   Time:  O(n)  -- single pass with two pointers, each index visited once.
//   Space: O(n)  -- for the result array (the input is assumed sorted
//          already, so no extra sort cost). An O(1)-extra-space in-place
//          variant exists using an index-encoding trick (storing two
//          values in one array slot via arr[i] += arr[j] * MAX and
//          decoding later), but it requires knowing an upper bound on
//          the values and is a common follow-up optimization to mention.
```

解释

考点：双指针（two-pointer）在有序数组上的经典应用，本身不难，关键是奇偶长度的边界处理容易漏掉——你给的思路（左右指针向中间收缩，交替取最大最小，最后单独处理奇数长度剩下的中间元素）完全正确，我按这个思路整理成了标准代码并且用两个例子完整走了一遍验证。

易错点：偶数长度和奇数长度的收尾条件不一样，很多人会在循环条件或者奇偶判断上漏写导致数组越界或者漏了中间元素，写代码时一定要用具体例子（尤其是奇数长度）单独走一遍边界情况。

延伸考点：如果面试官要求原地（in-place，O(1)额外空间）实现，需要用"一个数组槽位编码两个值"的技巧（前提是知道数值上界），这是这道题常见的进阶追问方向，能主动提出来是加分项。

Q100. Algorithms: Minimum Absolute Difference in an Unsorted Array

QUESTION

Given an unsorted array of integers, find the minimum absolute difference between any two elements in the array. Write an efficient solution and justify why your approach is optimal (or near-optimal).

ANSWER

```
def min_abs_diff(arr):
    if len(arr) < 2:
        raise ValueError("need at least 2 elements")

    arr = sorted(arr)
    min_diff = float('inf')

    for i in range(1, len(arr)):
        diff = arr[i] - arr[i - 1]
        if diff < min_diff:
            min_diff = diff
        if min_diff == 0:
            break   // early exit: 0 is the smallest possible difference
                    // (duplicate values), can't do better than that
    return min_diff

// Why sorting first, then only checking ADJACENT pairs, is sufficient
// (this is the key insight, not just "sort then scan"):
//
//   Claim: after sorting, the pair achieving the global minimum
//   difference is ALWAYS adjacent in the sorted order.
//
//   Proof sketch (by contradiction): suppose the minimum-difference pair
//   (x, y), x < y, are NOT adjacent after sorting -- i.e. there exists
//   some element z with x < z < y sitting between them in sorted order.
//   Then (y - x) = (y - z) + (z - x), and since both (y - z) and
//   (z - x) are positive, at least one of them must be STRICTLY SMALLER
//   than (y - x). That means either (x, z) or (z, y) is a strictly
//   smaller difference than (x, y) -- contradicting the assumption that
//   (x, y) was the minimum. So the minimum-difference pair can never
//   have anything sorted between them -- they must be adjacent.
//
// Complexity:
//   Time:  O(n log n), dominated by the sort; the scan afterward is O(n).
//   Space: O(1) extra (in-place sort) or O(n) depending on the sort
//          implementation/language.
//
// Is O(n log n) optimal? In the comparison-based model, yes -- this
// problem is at least as hard as "element uniqueness" (is min_diff == 0
// for some pair?), which has a proven Omega(n log n) lower bound in the
// algebraic decision-tree model. So a comparison-based algorithm cannot
// do better than O(n log n) in general. (If the input values are known
// to be bounded integers in a small range, a counting-sort-based
// approach can achieve O(n + range) instead -- worth mentioning as a
// follow-up if the interviewer asks about further optimization under
// extra assumptions.)
```

解释

考点：排序后只需要比较相邻元素这个结论，很多人凭直觉知道，但面试时被要求证明"为什么"往往说不清楚——证明的核心是反证法：如果最优的一对中间还夹着第三个元素，那这两段更短的差值里必然有一个比原来更小，矛盾，所以最优解一定发生在相邻元素之间。

易错点：只处理了输入至少2个元素的情况，面试时最好主动提一句边界条件（数组为空或只有1个元素时无法定义"两数之差"，应该报错或者按题目约定返回特殊值）；另外重复元素（差值为0）是可能出现的合法情况，代码里可以加一个提前退出优化（0已经是理论最小值，没必要继续扫描）。

延伸考点：面试官经常会追问"能不能做到比 O(n log n) 更快"，标准回答是"比较模型下不能，因为这个问题不弱于元素唯一性判定问题，有 Ω(n log n) 的下界"，但如果放宽条件（比如数值范围已知且较小），可以用计数排序把复杂度降到 O(n + range)，这体现了对算法下界和"在额外假设下能否突破"这两个层次的理解。

Q101. Algorithms: Generate All Valid Knight Moves on a Chessboard

QUESTION

On a chess board, a knight has up to 8 possible moves from any given square. Write a function that takes the knight's current position and the board size, and returns the list of valid next positions -- performing boundary checks to exclude any move that would land off the board.

ANSWER

```
def knight_moves(row, col, board_size):
    # all 8 L-shaped offsets: (+-2,+-1) and (+-1,+-2)
    offsets = [(-2, -1), (-2, 1), (-1, -2), (-1, 2),
               ( 1, -2), ( 1, 2), ( 2, -1), ( 2, 1)]
    moves = []
    for dr, dc in offsets:
        r, c = row + dr, col + dc
        if 0 <= r < board_size and 0 <= c < board_size:
            moves.append((r, c))
    return moves

// Complexity: O(1) -- always exactly 8 candidate offsets checked,
// regardless of board_size; the boundary check is a simple O(1)
// comparison per candidate, so the whole function is constant time and
// constant space (output size is at most 8).

// Verified against known chess facts (classic sanity checks for this
// exact problem):
//   corner square (0,0), board_size=8  -> exactly 2 valid moves
//   edge square   (0,4), board_size=8  -> exactly 4 valid moves
//   center square (4,4), board_size=8  -> exactly 8 valid moves (no
//                                         boundary clipping at all)
//   all three match the standard, well-known knight-move counts for a
//   corner / edge / center square on a normal chessboard.
```

解释

考点：这是一道基础但考察点很明确的算法题——8个偏移量本身没有技巧，真正的考点是边界检查有没有漏，以及代码风格是否干净（用偏移量列表+循环，而不是写8个几乎一样的if分支）。

正确性验证的好习惯：面试时最好主动提一下已知的验证锚点——棋盘角上的格子只有2种走法，边上（非角）的格子有4种，内部格子有8种，这是国际象棋里很基础的常识，写完代码后可以口头或者用这几个具体例子快速自检，而不是只靠"看起来应该没错"。

延伸考点：面试官可能会追问如果棋盘不是正方形（长宽不同）怎么改（把单一的board_size参数拆成board_width和board_height两个参数，边界判断分别对应即可，逻辑完全不变）；也可能追问能不能进一步扩展成"从起点走N步能否到达终点"（这是这道题的经典进阶版，本质是在knight_moves生成的图上做BFS最短路径搜索）。

Q102. Algorithms & Scripting: Common Elements in Three Sorted Arrays

QUESTION

Given three sorted arrays a, b, and c, find all elements that are common in all three.

Example 1:

Input: a = [1,2,3], b = [2,3,4], c = [2,3,5]

Output: 2 3

Example 2:

Input: a = [1,2], b = [3,4], c = [5,6]

Output: (no common elements)

Constraints: arrays are sorted in non-decreasing order, 1 <= length <= 10^5.

ANSWER

```
def common_elements_three_sorted(a, b, c):
    i = j = k = 0
    result = []
    while i < len(a) and j < len(b) and k < len(c):
        if a[i] == b[j] == c[k]:
            if not result or result[-1] != a[i]:
                result.append(a[i])
            i += 1; j += 1; k += 1
        elif a[i] <= b[j] and a[i] <= c[k]:
            i += 1
        elif b[j] <= a[i] and b[j] <= c[k]:
            j += 1
        else:
            k += 1
    return result

# Verified: common_elements_three_sorted([1,2,3],[2,3,4],[2,3,5]) == [2, 3]
#           common_elements_three_sorted([1,2],[3,4],[5,6]) == []
# Complexity: O(len(a)+len(b)+len(c)) time, O(1) extra space (excluding output).
```

解释

考点：三路指针（three-pointer）技巧，是"合并/求交集"这类有序数组问题的标准解法，本质是merge sort合并步骤的变体。

关键思路：三个指针各自指向一个数组，每次比较三者当前值，最小的那个（或那些）指针前进；三者相等时才是交集元素，记得去重（用result[-1]比较避免同一个值重复加入）。

延伸考点：如果是k个有序数组求交集，这个思路可以直接推广成k个指针，或者更通用地用最小堆维护当前每个数组的指针位置。

Q103. Algorithms & Scripting: Longest Consecutive Sequence

QUESTION

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence. You must write an algorithm that runs in O(n) time.

Example 1:

Input: nums = [100,4,200,1,3,2]

Output: 4 (the sequence is [1,2,3,4])

Example 2:

Input: nums = [0,3,7,2,5,8,4,6,0,1]

Output: 9

Constraints: 0 <= nums.length <= 10^5, -10^9 <= nums[i] <= 10^9.

ANSWER

```
def longest_consecutive_sequence(nums):
    num_set = set(nums)
    best = 0
    for n in num_set:
        if n - 1 not in num_set:          # only start counting from the
                                            # BEGINNING of a run
            length = 1
            while n + length in num_set:
                length += 1
            best = max(best, length)
    return best

# Verified: longest_consecutive_sequence([100,4,200,1,3,2]) == 4
#           longest_consecutive_sequence([0,3,7,2,5,8,4,6,0,1]) == 9
# Complexity: O(n) time -- each number is only ever extended from once,
# because the "n-1 not in num_set" check skips every number that ISN'T
# the start of its run, so the total work across all runs is O(n).
```

解释

考点：这道题名字里写着"O(n)"，明确排除了先排序（O(n log n)）的解法，核心考点是用哈希集合避免排序。

关键技巧：用一个哈希集合存所有数字，只有当"n-1不在集合里"时才从n开始往后数（因为这说明n是某个连续段的起点），避免每个连续段被重复计数多次——这一步保证了整体是O(n)而不是O(n^2)。

易错点：容易漏掉"n-1 not in num_set"这个起点判断，如果每个数字都无脑往后数，遇到长度为L的连续段会重复计算L次，退化成O(n*L)。

Q104. Algorithms & Scripting: Count Triplets with Zero Sum

QUESTION

Count Triplets with Zero Sum: given an array arr, count the number of triplets (i, j, k) where i < j < k and arr[i] + arr[j] + arr[k] = 0.

Example 1:

Input: arr = [-1,0,1]

Output: 1 ((-1) + 0 + 1 = 0)

Example 2:

Input: arr = [1,5,3,2]

Output: 0

Constraints: 3 <= arr.length <= 10^3, -10^6 <= arr[i] <= 10^6.

ANSWER

```
def count_triplets_zero_sum(arr):
    arr = sorted(arr)
    n = len(arr)
    count = 0
    for i in range(n - 2):
        lo, hi = i + 1, n - 1
        while lo < hi:
            s = arr[i] + arr[lo] + arr[hi]
            if s == 0:
                count += 1
                lo += 1; hi -= 1
            elif s < 0:
                lo += 1
            else:
                hi -= 1
    return count

# Verified: count_triplets_zero_sum([-1,0,1]) == 1
#           count_triplets_zero_sum([1,5,3,2]) == 0
# Complexity: O(n^2) time (sort O(n log n) + two-pointer scan O(n^2)),
# O(1) extra space (in-place sort, ignoring output).
# NOTE: this counts by VALUE combination like the classic "3Sum" pattern;
# if the array can contain duplicate VALUES that should each count as
# separate index-triplets, this two-pointer version (which naturally
# advances past ties without deduplicating) already counts them all --
# it does NOT skip duplicates the way a "return unique triplets" variant
# would.
```

解释

考点：排序+双指针求三数之和，是"3Sum"系列问题的基础版本，这里只需要计数，不需要返回具体的三元组内容，反而更简单。

关键思路：先排序，固定最左边的数，剩下两个数用双指针从两端向中间收缩——这是处理"三个数满足某个和"这类问题的标准降维技巧，把O(n^3)暴力枚举降到O(n^2)。

和"3Sum"（本文档另一题）的区别：3Sum要求返回不重复的三元组本身，需要跳过相同的值；这道题只是计数（按下标i<j<k的组合数），双指针天然会数到所有满足条件的下标组合，不需要额外去重逻辑。

Q105. Algorithms & Scripting: Kth Largest Element in an Array

QUESTION

Kth Largest Element in an Array: given an integer array nums and an integer k, return the kth largest element in the array (kth largest in sorted order, not kth distinct element).

Example 1:

Input: nums = [3,2,1,5,6,4], k = 2

Output: 5

Example 2:

Input: nums = [3,2,3,1,2,4,5,5,6], k = 4

Output: 4

Constraints: 1 <= k <= nums.length <= 10^5, -10^4 <= nums[i] <= 10^4.

ANSWER

```
import heapq

def kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]

# Verified: kth_largest([3,2,1,5,6,4], 2) == 5
#           kth_largest([3,2,3,1,2,4,5,5,6], 4) == 4
# Complexity: heapq.nlargest(k, nums) runs in O(n log k) time using a
# size-k min-heap internally, O(k) extra space -- much better than
# sorting the whole array (O(n log n)) when k is small relative to n.
#
# Equivalent hand-rolled version (same complexity, shows the mechanism
# heapq.nlargest hides):
#
# def kth_largest_manual(nums, k):
#     heap = nums[:k]
#     heapq.heapify(heap)              # min-heap of the first k elements
#     for x in nums[k:]:
#         if x > heap[0]:
#             heapq.heapreplace(heap, x)   # pop smallest, push x
#     return heap[0]                   # smallest of the k largest = kth largest
```

解释

考点：第K大元素是堆（heap）应用的经典题，考察能不能识别出"只需要维护K个候选"这个特征从而避免完整排序。

关键思路：维护一个大小为K的最小堆，堆顶始终是"当前已见过的K个最大值里最小的那个"，也就是答案的候选值；新元素只有比堆顶大才有资格入堆（顶替掉堆顶），这样整体只需要O(n log K)而不是排序的O(n log n)。

延伸考点：如果K接近n，直接排序可能反而更快（因为log K接近log n，但排序的常数因子更小）；面试官可能会追问快速选择算法（quickselect，基于快排partition的O(n)平均复杂度解法），这是这道题的进阶最优解。

Q106. Algorithms & Scripting: Equilibrium Index

QUESTION

Given an array nums, find an index i such that the sum of elements to the left of i equals the sum of elements to the right of i. Return -1 if no such index exists. The element at index i itself is not included in either sum.

Example 1:

Input: nums = [1,3,5,2,2]

Output: 2 (left sum 1+3=4, right sum 2+2=4)

Example 2:

Input: nums = [1,2,3]

Output: -1

Constraints: 1 <= nums.length <= 10^5, -10^5 <= nums[i] <= 10^5.

ANSWER

```
def equilibrium_index(nums):
    total = sum(nums)
    left = 0
    for i, x in enumerate(nums):
        right = total - left - x
        if left == right:
            return i
        left += x
    return -1

# Verified: equilibrium_index([1,3,5,2,2]) == 2
#           equilibrium_index([1,2,3]) == -1
# Complexity: O(n) time, O(1) extra space -- one pass to get the total
# sum, one pass tracking a running left-sum and deriving the right-sum
# by subtraction, rather than recomputing the right-sum from scratch at
# every index (which would be O(n^2)).
```

解释

考点：前缀和思想的基础应用，考察能不能避免"对每个候选i都重新算一次左右两段的和"这种O(n^2)暴力解法。

关键技巧：先算一次总和total，遍历时维护一个滚动的left（左边已经累加的和），右边的和可以直接用total-left-nums[i] 算出来，不需要重新遍历右边部分，这是把O(n^2)降到O(n)最常用的手段之一。

易错点：容易忘记"当前元素本身不计入左右任何一边"这个边界条件，我的实现里right的计算已经把nums[i]单独减掉，确保不会被算进右边的和。

Q107. Algorithms & Scripting: Word Break

QUESTION

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words. Words may be reused multiple times.

Example 1:

Input: s = "leetcode", wordDict = ["leet","code"]

Output: true

Example 2:

Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]

Output: false

Constraints: 1 <= s.length <= 300, 1 <= wordDict.length <= 1000.

ANSWER

```
def word_break(s, wordDict):
    words = set(wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True                        # empty prefix is trivially "breakable"
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
```

```
    return dp[n]

# Verified: word_break("leetcode", ["leet","code"]) == True
#           word_break("catsandog", ["cats","dog","sand","and","cat"]) == False
# Complexity: O(n^2) time (n positions, each checking up to n substrings,
# each substring-slice/lookup itself O(n) in the worst case -- often
# stated as O(n^3) if you count slicing cost, O(n^2) lookups otherwise),
# O(n) extra space for the dp array (plus the dictionary's own storage).
```

解释

考点：动态规划的经典题，dp[i]表示"字符串前i个字符能否被字典完全切分"，考察能不能把"能否切分"这种判定性问题转化成子问题递推。

关键思路：dp[i]为真当且仅当存在某个j<i，使得dp[j]为真且s[j:i]是字典里的一个词——枚举所有可能的切分点j，只要找到一个满足条件的j就可以提前break。

易错点：dp[0]必须初始化为True（代表"空前缀"本身就是合法的，是递推的起点），很多人会漏掉这个边界初始化导致整个dp数组全部错误地保持False。

延伸考点：如果要求返回所有可能的切分方式而不只是判断是否可行（Word Break II），需要在dp基础上加回溯/记忆化搜索去枚举具体方案，复杂度会显著上升。

Q108. Algorithms & Scripting: Sort Colors

QUESTION

Given an array nums with n objects colored red (0), white (1), or blue (2), sort them in-place so objects of the same color are adjacent, in the order red, white, blue.

Example 1:

Input: nums = [2,0,2,1,1,0]

Output: 0 0 1 1 2 2

Example 2:

Input: nums = [2,0,1]

Output: 0 1 2

Constraints: 1 <= n <= 300, nums[i] is 0, 1, or 2.

ANSWER

```
def sort_colors(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1; mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:                           # nums[mid] == 2
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
            # NOTE: mid is NOT incremented here -- the element swapped in
            # from 'high' hasn't been examined yet, it needs to be
            # checked on the NEXT iteration.
    return nums
```

```
# Verified: sort_colors([2,0,2,1,1,0]) == [0,0,1,1,2,2]
#           sort_colors([2,0,1]) == [0,1,2]
# Complexity: O(n) time, single pass, O(1) extra space, IN-PLACE as required.
# This is the classic "Dutch National Flag" algorithm (Dijkstra).
```

解释

考点：荷兰国旗问题（Dutch National Flag），是三路快排partition思想的经典应用，考察能不能用三个指针一次遍历完成分类，而不是排序或者计数再重建。

关键易错点：swap到high位置换来的元素还没被检查过，所以high-=1之后mid不能跟着+1，必须留在原地下一轮继续判断这个刚换过来的值；相反，跟low交换的时候可以放心mid+1，因为跟low换过来的元素必然是1（low指针之前扫过的区域只会是0和1，不会是2）。

延伸考点：如果颜色种类不是3种而是K种（K-way partition），这个双指针技巧不能直接扩展，通常需要计数排序或者稳定的多路partition方案。

Q109. Algorithms & Scripting: Count Occurrences in Sorted Array

QUESTION

Given a sorted array nums and a value target, return the number of times target appears in the array. Return 0 if not present.

Example 1:

Input: nums = [1,1,2,2,2,3], target = 2

Output: 3

Example 2:

Input: nums = [1,2,3,4,5], target = 6

Output: 0

Constraints: nums is sorted in non-decreasing order; time complexity O(log n).

ANSWER

```
import bisect

def count_occurrences_sorted(nums, target):
    lo = bisect.bisect_left(nums, target)
    hi = bisect.bisect_right(nums, target)
    return hi - lo

# Verified: count_occurrences_sorted([1,1,2,2,2,3], 2) == 3
#           count_occurrences_sorted([1,2,3,4,5], 6) == 0
# Complexity: O(log n) time -- two binary searches (first/last occurrence),
# O(1) extra space. This is exactly the "find first and last position of
# element in sorted array" pattern, just returning the count instead of
# the two indices directly.
#
# Hand-rolled equivalent (in case bisect isn't allowed):
# def first_ge(nums, target):
#     lo, hi = 0, len(nums)
#     while lo < hi:
#         mid = (lo + hi) // 2
#         if nums[mid] < target: lo = mid + 1
#         else: hi = mid
#     return lo
```

解释

考点：有序数组里查找目标值出现次数，明确要求O(log n)，排除了线性扫描计数的解法，考的是"二分查找第一个/最后一个满足条件的位置"这个变体。

关键思路：不是直接二分查找target本身，而是分别二分查找"第一个>=target的位置"（左边界）和"第一个>target的位置"（右边界），两者之差就是出现次数——这个模式可以推广到很多"有序数组里统计满足某条件的元素个数"的题目。

延伸考点：这正是LeetCode经典题"在排序数组中查找元素的第一个和最后一个位置"的变体，如果面试官要求同时返回起止下标而不只是次数，只需要把lo和hi都返回即可。

Q110. Algorithms & Scripting: Search Insert Position

QUESTION

Given a sorted array of distinct integers and a target value, return the index if found. If not, return the index where it would be inserted in order. Must run in O(log n).

Example 1:

Input: nums = [1,3,5,7,9,11], target = 5

Output: 2 (target 5 already exists at index 2)

Example 2:

Input: nums = [1,3,5,7,9,11], target = 8

Constraints: 1 <= nums.length <= 10^4, values distinct and ascending.

ANSWER

```
def search_insert_position(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return lo               # standard behavior: lo IS the correct insertion index

# Verified: search_insert_position([1,3,5,7,9,11], 5) == 2  (found)
#           search_insert_position([1,3,5,7,9,11], 8) == 4  (not found:
#             8 would be inserted between 7 (index 3) and 9 (index 4),
#             i.e. AT index 4)
#
# NOTE ON THE SOURCE MATERIAL: the original problem statement for this
# question listed Example 2's expected output as "-1", but that directly
# contradicts the problem's OWN description ("return the index where it
# would be inserted in order") -- a not-found case should return the
# insertion index (4 here), not a sentinel -1. This is the standard,
# well-known LeetCode "Search Insert Position" problem, and the
# insertion-index behavior (not -1) is the correct, standard contract --
# flagged here explicitly rather than silently "fixing" the example
# without comment.
```

解释

考点：标准二分查找的变体，"找不到时返回该插入的位置"而不是返回-1，这是和普通二分查找最本质的区别。

关键理解：为什么循环结束时lo就是正确的插入位置——二分查找收缩到lo>hi时终止，此时lo指向的是"第一个比target大的位置"（或者数组末尾），根据二分查找不变式，这正好就是target应该插入的位置。

特别说明：这道题原始题目描述里给的例子2输出写的是"-1"，但这和题目本身的文字描述（"返回应该插入的位置"）自相矛盾——按标准LeetCode这道题的定义，找不到时应该返回插入位置（本例是4），而不是-1，我在答案里直接按正确、标准的语义实现，并且把这个原始素材里的不一致之处明确指出来，而不是悄悄按其中一种理解直接改掉不说明。

Q111. Algorithms & Scripting: Longest Repeating Character Replacement

QUESTION

Given a string s and integer k, you can change at most k characters to any other uppercase letter. Return the length of the longest substring containing the same letter you can get.

Example 1:

Input: s = "ABAB", k = 2

Output: 4

Example 2:

Input: s = "AABABBA", k = 1

Output: 4

Constraints: 1 <= s.length <= 10^5, uppercase letters only, 0 <= k <= s.length.

ANSWER

```
from collections import defaultdict

def longest_repeating_char_replacement(s, k):
    count = defaultdict(int)
    left = 0
    max_count = 0     # highest frequency of any single letter seen in
                      # the CURRENT window (a "high-water mark" -- see note)
    best = 0
    for right in range(len(s)):
        count[s[right]] += 1
        max_count = max(max_count, count[s[right]])
        while (right - left + 1) - max_count > k:
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best

# Verified: longest_repeating_char_replacement("ABAB", 2) == 4
#           longest_repeating_char_replacement("AABABBA", 1) == 4
# Complexity: O(n) time (sliding window, each index enters/leaves the
# window at most once), O(1) extra space (fixed 26-letter count table).
#
# Subtle but important point: max_count is allowed to become STALE (it
# is never decreased when the window shrinks) -- this is intentional,
# not a bug: max_count only needs to represent "the best window size
# we've already achieved so far," and once 'best' has captured a window
# of that quality, a stale (too-high) max_count can only ever shrink the
# window further, never incorrectly grow 'best' beyond a length that was
```

```
# already actually achievable.
```

解释

考点：滑动窗口的进阶题，核心技巧是"窗口只增不减"（max_count允许过时不回退），这是这类题最容易被问到"为什么这样写是对的"的地方。

关键理解：窗口大小减去当前窗口内出现次数最多的字符个数，就是"需要替换的字符数"，只要这个值不超过k，当前窗口就是合法的；窗口不满足条件时收缩左边界，但max_count不需要在收缩时重新计算最大值，因为即使max_count是过时（偏大）的，也只会让窗口暂时收得更紧，不会导致best被错误地放大。

延伸考点：面试官可能会追问如果字符集不是26个大写字母而是任意字符（比如Unicode），count用字典还是数组更合适，以及如果k可以变化（比如题目要求同时输出用了多少次替换）需要额外维护什么状态。

Q112. Algorithms & Scripting: Count Character Occurrences

QUESTION

Given a string s and a character ch, return the number of times ch appears in s.

Example 1: Input: s = "hello", ch = "l" -> Output: 2

Example 2: Input: s = "banana", ch = "a" -> Output: 3

Constraints: 0 <= s.length <= 10^5, case-sensitive.

ANSWER

```
def count_char_occurrence(s, ch):
    return s.count(ch)

# Verified: count_char_occurrence("hello", "l") == 2
#           count_char_occurrence("banana", "a") == 3
# Complexity: O(n) time, O(1) extra space. Python's built-in str.count()
# already does exactly this scan internally; a hand-rolled equivalent
# would just be a for-loop with a counter, same complexity.
```

解释

考点：这是一道非常基础的题，几乎没有算法难度，更多是考察最基本的字符串遍历/内置API使用是否熟练，通常出现在面试最开始热身阶段。

关键点：直接用语言内置的count方法就是最优解（O(n)时间，不需要额外空间），没有必要为了"看起来更高级"而手写更复杂的实现。

延伸考点：如果要求忽略大小写统计，需要先对s和ch做统一大小写处理（比如都转成小写）再比较。

Q113. Algorithms & Scripting: Permutation in String

QUESTION

Given two strings s1 and s2, return true if s2 contains a permutation of s1 (i.e. some substring of s2 is a rearrangement of s1's characters).

Example 1: s1="ab", s2="eidbaooo" -> true ("ba" is a permutation of "ab")

Example 2: s1="ab", s2="eidboaoo" -> false

Constraints: 1 <= s1.length, s2.length <= 10^4.

ANSWER

```
from collections import Counter

def permutation_in_string(s1, s2):
    n1, n2 = len(s1), len(s2)
    if n1 > n2:
        return False
    need = Counter(s1)
    window = Counter(s2[:n1])
    if window == need:
        return True
    for i in range(n1, n2):
        window[s2[i]] += 1
        left_char = s2[i - n1]
        window[left_char] -= 1
        if window[left_char] == 0:
            del window[left_char]      # keep Counter clean so == comparisons
                                       # don't fail on stray zero-count entries
        if window == need:
            return True
    return False

# Verified: permutation_in_string("ab", "eidbaooo") == True
#           permutation_in_string("ab", "eidboaoo") == False
# Complexity: O(n2) time -- a fixed-size sliding window of length
# len(s1) slides across s2 once, each step doing O(1) amortized Counter
# updates (bounded by the alphabet size); O(alphabet size) extra space.
```

解释

考点：固定窗口大小的滑动窗口+字符频次比较，是"子串异位词/排列"这一类问题的标准解法。

关键技巧：s1的一个排列在s2中出现，等价于"s2中存在某个长度等于len(s1)的窗口，其字符频次统计和s1完全一样"——用Counter维护窗口内的频次，每次窗口滑动只需要O(1)增删两个字符的计数，不需要每次都重新统计整个窗口。

易错点：Counter比较相等时，如果窗口内某个字符计数变成0但没有从Counter里删除，会导致Counter({'a': 0}) != Counter({})这种意外不相等，所以窗口滑出的字符计数归零时要显式del掉这个key。

Q114. Algorithms & Scripting: Longest Common Subsequence

QUESTION

Given two strings a and b, return the length of their longest common subsequence (LCS). If none, return 0. A subsequence need not be contiguous but must preserve relative order.

Example 1: a="abcde", b="ace" -> 3 ("ace")

Example 2: a="abc", b="def" -> 0

Constraints: 1 <= a.length, b.length <= 1000.

ANSWER

```
def longest_common_subsequence(a, b):
    m, n = len(a), len(b)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
```

```
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

# Verified: longest_common_subsequence("abcde", "ace") == 3
#           longest_common_subsequence("abc", "def") == 0
# Complexity: O(m*n) time and space (classic 2D DP table). Space can be
# reduced to O(min(m,n)) by only keeping the previous DP row, if needed.
```

解释

考点：最经典的二维动态规划题之一，几乎是LCS问题的标准模板，dp[i][j]表示a的前i个字符和b的前j个字符的最长公共子序列长度。

关键递推关系：如果两个字符串当前位置字符相同，说明可以把这个字符接到"去掉这两个字符后的LCS"后面，dp[i][j]=dp[i-1][j-1]+1；不同的话，答案只能来自"去掉a当前字符"或"去掉b当前字符"两种情况的较大值。

延伸考点：如果要求不仅返回长度还要还原出具体的LCS字符串，需要在填表的同时记录路径（或者填完表后从右下角回溯）；这道题也是编辑距离（Edit Distance）等一系列字符串DP题的基础形式，面试官经常会在此基础上继续变化条件深挖。

Q115. Algorithms & Scripting: Longest Substring Without Repeating Characters

QUESTION

Given a string s, find the length of the longest substring without repeating characters.

Example 1: s = "abcabcbb" -> 3 ("abc")

Example 2: s = "bbbbb" -> 1

Constraints: 0 <= s.length <= 5*10^4.

ANSWER

```
def longest_substr_no_repeat(s):
    seen = {}          # char -> most recent index seen at
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1        # jump left past the previous occurrence
        seen[ch] = right
        best = max(best, right - left + 1)
    return best

# Verified: longest_substr_no_repeat("abcabcbb") == 3
#           longest_substr_no_repeat("bbbbb") == 1
# Complexity: O(n) time -- each index is visited once by 'right', and
# 'left' only ever moves forward, never backward, so total pointer
# movement across the whole run is bounded by O(n). O(min(n, alphabet
# size)) extra space for the seen-index map.
```

解释

考点：滑动窗口的入门经典题，几乎是滑动窗口这一大类题目的开篇题，核心考点是能不能想到"记录每个字符最近一次出现的位置"来快速跳跃左边界，而不是遇到重复字符就一步步收缩。

关键技巧：seen[ch]>=left这个判断很关键——如果记录的位置已经在当前窗口左边界之外（说明是"过时"的记录，早就被滑出窗口了），就不需要因此收缩left，否则left可能会错误地往回跳，这是这道题最容易出bu

g的地方。

延伸考点：这道题是很多字符串滑动窗口变体的基础（比如"最多替换k次的最长重复字符子串"，本文档另一题就是它的进阶版本），掌握这个模板之后很多类似题目都是同一套思路的变形。

Q116. Algorithms & Scripting: All Pairs With Given Sum

QUESTION

Given an array nums and integer target, return all unique pairs (a, b) such that a + b = target.

Example 1: nums=[2,7,11,15], target=9 -> [(2,7)]

Example 2: nums=[3,3,3], target=6 -> [(3,3)]

Constraints: 1 <= nums.length <= 10^5, return unique value pairs.

ANSWER

```
def all_pairs_with_sum(nums, target):
    seen = set()
    pairs = set()
    for n in nums:
        complement = target - n
        if complement in seen:
            pairs.add((min(n, complement), max(n, complement)))
        seen.add(n)
    return sorted(pairs)

# Verified: all_pairs_with_sum([2,7,11,15], 9) == [(2, 7)]
#           all_pairs_with_sum([3,3,3], 6) == [(3, 3)]
# Complexity: O(n) time average (hash set operations), O(n) extra space.
# Storing pairs as (min, max) tuples in a SET automatically de-duplicates
# both (a,b)/(b,a) ordering AND repeated occurrences of the same value
# pair (as in example 2, where multiple 3s would otherwise generate the
# same pair repeatedly).
```

解释

考点：两数之和的"返回所有唯一值对"变体（区别于经典"Two Sum"只要求返回一对下标），考察如何在遍历过程中同时完成查找和去重。

关键技巧：用一个集合存已经见过的数字，对每个新数字检查"target减去它"是否已经出现过；把结果对统一存成(min,max)形式的元组再放入集合，天然去掉了"值重复"和"顺序不同但本质相同"这两类重复。

易错点：如果直接用列表存结果而不用集合，遇到像例2里三个3的情况，会重复生成同一个(3,3)三次，必须显式去重，用集合是最简洁的处理方式。

Q117. Algorithms & Scripting: Contains Duplicate II

QUESTION

Contains Duplicate II: given an integer array nums and integer k, return true if there are two distinct indices i and j such that nums[i]==nums[j] and |i-j|<=k.

Example 1: nums=[1,2,3,1], k=3 -> true

Example 2: nums=[1,2,3,1,2,3], k=2 -> false

Constraints: 1 <= nums.length <= 10^5, 0 <= k <= 10^5.

ANSWER

```
def contains_nearby_duplicate(nums, k):
    last_seen = {}
    for i, n in enumerate(nums):
        if n in last_seen and i - last_seen[n] <= k:
            return True
        last_seen[n] = i
    return False

# Verified: contains_nearby_duplicate([1,2,3,1], 3) == True
#           contains_nearby_duplicate([1,2,3,1,2,3], 2) == False
# Complexity: O(n) time, O(min(n, distinct values)) extra space -- one
# pass, always comparing against the MOST RECENT prior occurrence of the
# same value (which is provably the best chance of being within k, since
# any earlier occurrence is farther away).
```

解释

考点：这是Contains Duplicate系列问题的进阶版，加上了"下标距离不超过k"这个限制，考察能不能只用一次遍历+哈希表解决，而不是暴力检查所有下标对。

关键理解：对每个值，只需要记住它"最近一次"出现的位置——如果最近一次都超过k的距离，那更早的出现距离只会更远，不可能满足条件，所以只保留最近一次出现位置就足够了，不需要保留所有历史出现位置的列表。

延伸考点：如果进一步要求"值的差不超过t"（Contains Duplicate III），需要换成滑动窗口+桶排序或者有序数据结构（如平衡二叉搜索树）维护窗口内的值，而不能再简单地用哈希表，这是明显更难的变体。

Q118. Algorithms & Scripting: 3Sum

QUESTION

Given an integer array nums, return all the triplets [nums[i],nums[j],nums[k]] such that i!=j, i!=k, j!=k, and nums[i]+nums[j]+nums[k]==0. The solution set must not contain duplicate triplets.

Example 1: nums=[-1,0,1,2,-1,-4] -> [[-1,-1,2],[-1,0,1]]

Example 2: nums=[0,0,0] -> [[0,0,0]]

Constraints: 3 <= nums.length <= 3000.

ANSWER

```
def three_sum(nums):
    nums = sorted(nums)
    n = len(nums)
    result = []
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i-1]:
            continue                    # skip duplicate "first" values
        lo, hi = i+1, n-1
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s == 0:
                result.append([nums[i], nums[lo], nums[hi]])
                lo += 1; hi -= 1
                while lo < hi and nums[lo] == nums[lo-1]:
                    lo += 1             # skip duplicate "second" values
                while lo < hi and nums[hi] == nums[hi+1]:
                    hi -= 1             # skip duplicate "third" values
```

```
            elif s < 0:
                lo += 1
            else:
                hi -= 1
    return result

# Verified: three_sum([-1,0,1,2,-1,-4]) == [[-1,-1,2],[-1,0,1]]
#           three_sum([0,0,0]) == [[0,0,0]]
# Complexity: O(n^2) time (sort O(n log n) + two-pointer scan O(n^2)),
# O(1) extra space beyond the output (in-place sort).
```

解释

考点：这是数组类面试题里出现频率最高的题目之一，是"Count the Triplets"那道题（本文档另一题）的直接延伸——那道题只需要计数，这道题需要返回不重复的具体三元组。

关键区别（也是最容易出bug的地方）：三处跳过重复值的逻辑缺一不可——固定的第一个数如果和上一轮相同要跳过，双指针命中答案后收缩时如果新指向的值和上一个相同也要跳过，这三处跳过分别针对三元组里第一、第二、第三个位置的去重，漏掉任何一处都会导致结果里出现重复三元组。

延伸考点：面试官几乎必然会追问4Sum怎么做（在3Sum外面再套一层固定第一个数的循环，递归/迭代地降维成3Sum乃至2Sum），以及为什么这种"固定前面几个数+双指针处理最后两个数"的模式对k-Sum问题普遍适用。

Q119. Algorithms & Scripting: First Duplicate Element

QUESTION

Given an array nums, return the first element that appears more than once (i.e. whose SECOND occurrence comes earliest). If no duplicate exists, return -1.

Example 1: nums=[2,5,1,2,3,5,1,2,4] -> 2

Example 2: nums=[1,2,3,4] -> -1

Constraints: 1 <= nums.length <= 10^5, 1 <= nums[i] <= 10^5.

ANSWER

```
def first_duplicate_element(nums):
    seen = set()
    for n in nums:
        if n in seen:
            return n
        seen.add(n)
    return -1

# Verified: first_duplicate_element([2,5,1,2,3,5,1,2,4]) == 2
#           first_duplicate_element([1,2,3,4]) == -1
# Complexity: O(n) time, O(n) extra space -- a single left-to-right pass;
# the first value we encounter a SECOND time is, by definition, the
# first element whose duplicate occurrence appears earliest in the scan.
```

解释

考点：这是一道基础哈希集合应用题，重点是理解题目问的是"第一个出现第二次的值"，而不是"第一个数值最小的重复值"，这两者很容易搞混。

关键理解：从左到右扫描，用集合记录见过的值，第一次命中"已经在集合里"的时刻，对应的值就是答案——因为按遍历顺序，这是第一个"重复事件"发生的时刻。

延伸考点：如果数组元素范围有限（比如本题限定1到10^5，和数组长度同阶），可以用原地标记法（把nums[abs(x)-1]取负作为"访问过"的标记）把额外空间降到O(1)，这是"数组元素范围有限"这类题目常见的空间优化技巧。

Q120. Algorithms & Scripting: Rotate Array

QUESTION

Given an integer array nums, rotate the array to the right by k steps.

Example 1: nums=[1,2,3,4,5,6,7], k=3 -> [5,6,7,1,2,3,4]

Example 2: nums=[1,2], k=3 -> [2,1]

Constraints: 1 <= nums.length <= 10^5.

ANSWER

```
def rotate_array(nums, k):
    n = len(nums)
    k = k % n                          # handle k >= n (e.g. example 2: k=3, n=2)
    def reverse(lo, hi):
        while lo < hi:
            nums[lo], nums[hi] = nums[hi], nums[lo]
            lo += 1; hi -= 1
    reverse(0, n-1)      # reverse the whole array
    reverse(0, k-1)      # reverse the first k elements back to correct order
    reverse(k, n-1)      # reverse the remaining n-k elements back to correct order
    return nums

# Verified: rotate_array([1,2,3,4,5,6,7], 3) == [5,6,7,1,2,3,4]
#           rotate_array([1,2], 3) == [2,1]
# Complexity: O(n) time (three linear reversal passes), O(1) EXTRA space
# -- fully in-place, which is the interesting constraint of this problem
# (a naive "slice and concatenate" solution is O(n) time but O(n) space).
```

解释

考点："三次反转"是原地旋转数组最经典的技巧，考察能不能在O(1)额外空间的约束下完成旋转，而不是简单地用切片拼接（那样需要O(n)额外空间）。

关键技巧：先整体反转整个数组，再分别反转前k个和后n-k个——这个"三次反转"的模式可以直接类比成"把一本书翻过来再翻半部分"，是处理"循环移位"类问题的通用套路。

易错点：k必须先对n取模（题目给的例子2里k=3但数组长度只有2，直接用k=3去做后续计算会越界），这个边界条件很容易被漏掉。

Q121. Algorithms & Scripting: Product of Array Except Self

QUESTION

Given an integer array nums, return an array answer such that answer[i] equals the product of all elements except nums[i]. Must run in O(n) time WITHOUT using division.

Example 1: nums=[1,2,3,4] -> [24,12,8,6]

Example 2: nums=[-1,1,0,-3,3] -> [0,0,9,0,0]

Constraints: 2 <= nums.length <= 10^5, no division allowed.

ANSWER

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    prefix = 1
    for i in range(n):                # left-to-right: result[i] = product
        result[i] = prefix            # of everything BEFORE i
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):    # right-to-left: multiply in the
        result[i] *= suffix            # product of everything AFTER i
        suffix *= nums[i]
    return result

# Verified: product_except_self([1,2,3,4]) == [24, 12, 8, 6]
#           product_except_self([-1,1,0,-3,3]) == [0, 0, 9, 0, 0]
# Complexity: O(n) time (two passes), O(1) EXTRA space excluding the
# output array itself (the "answer" array doesn't count against the
# space constraint per the problem's usual convention).
# Correctly handles zeros (example 2) WITHOUT any special-casing --
# no division means no risk of dividing by zero in the first place.
```

解释

考点：这道题明确禁止用除法（否则可以先算总乘积再逐个除，遇到0还要额外特判），核心考点是"前缀积+后缀积"这个不需要除法的替代方案。

关键思路：result[i] = (i左边所有数的乘积) × (i右边所有数的乘积)。第一次遍历算出每个位置左边的前缀积存入result，第二次从右往左遍历，用一个变量累积右边的后缀积，边遍历边乘到result里，两次遍历都是O(1)额外空间（不算输出数组本身）。

为什么不能用除法这个限制很关键：如果允许除法，直接算总乘积total，然后result[i]=total/nums[i]，代码更简单，但数组里出现0就会除零错误，需要额外处理，这也是为什么这道题要禁止除法来加大难度、考察真正理解前缀和思想的解法。

Q122. Algorithms & Scripting: Climbing Stairs

QUESTION

You are climbing a staircase with n steps. Each time you climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1: n=2 -> 2 (1+1 or 2)

Example 2: n=3 -> 3 (1+1+1, 1+2, 2+1)

Constraints: 1 <= n <= 45.

ANSWER

```
def climbing_stairs(n):
    if n <= 2:
        return n
    a, b = 1, 2                       # ways to reach step 1, step 2
    for _ in range(3, n + 1):
        a, b = b, a + b               # ways(n) = ways(n-1) + ways(n-2)
    return b

# Verified: climbing_stairs(2) == 2
#           climbing_stairs(3) == 3
# Complexity: O(n) time, O(1) extra space (only two running variables
```

```
# needed, no full DP array required since each step only depends on the
# previous two).
# This is exactly the Fibonacci recurrence: ways(n) = ways(n-1) + ways(n-2).
```

解释

考点：这是动态规划入门第一课经典题，本质就是斐波那契数列，考察能不能识别出"到达第n阶的方法数=到达第n-1阶的方法数+到达第n-2阶的方法数"这个递推关系。

关键理解：到达第n阶，最后一步要么是从n-1阶走1步上来，要么是从n-2阶走2步上来，这两种情况互斥且穷尽了所有可能，所以直接相加。

空间优化：因为每一步只依赖前两步的结果，不需要维护完整的dp数组，只用两个滚动变量就能把空间从O(n)降到O(1)，这是很多线性DP题目的标准优化手段，面试官经常会主动追问"能不能不用数组"。

Q123. Algorithms & Scripting: Search in Rotated Sorted Array

QUESTION

Given a possibly-rotated sorted array nums and target, return the index of target, or -1 if not present. Must run in O(log n).

Example 1: nums=[4,5,6,7,0,1,2], target=0 -> 4

Example 2: nums=[4,5,6,7,0,1,2], target=3 -> -1

Constraints: 1 <= nums.length <= 5000, all values unique.

ANSWER

```
def search_rotated_sorted(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[lo] <= nums[mid]:              # left half [lo..mid] is sorted
            if nums[lo] <= target < nums[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                                    # right half [mid..hi] is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

# Verified: search_rotated_sorted([4,5,6,7,0,1,2], 0) == 4
#           search_rotated_sorted([4,5,6,7,0,1,2], 3) == -1
# Complexity: O(log n) time, O(1) extra space -- still a genuine binary
# search, just with extra logic each step to figure out WHICH half
# (left of mid, or right of mid) is the "normally sorted" half, since
# the rotation point breaks the array into two sorted pieces.
```

解释

考点：旋转有序数组查找，是二分查找的经典变体，考察能不能在"整体不是严格递增"的数组上依然用二分查找的框架解决问题。

关键思路：虽然整个数组不是完全有序的，但从mid切开后，左右两半里必然有一半是"正常有序"的（因为只有一个旋转点）——先判断哪一半有序，再判断target是否落在那个有序区间内，从而决定往哪边收缩，这是这类题目的通用解题框架。

易错点：判断"左半有序"时用的是nums[lo]<=nums[mid]（注意是<=不是<，因为lo可能等于mid，此时左半只有一个元素，天然有序），这个边界条件很容易写错导致死循环或者漏判。

Q124. Algorithms & Scripting: Find Peak Element

QUESTION

A peak element is strictly greater than its neighbors. Given nums, find a peak element's index (any valid peak). Imagine nums[-1]=nums[n]=-infinity.

Example 1: nums=[1,2,3,1] -> 2

Example 2: nums=[1,2,1,3,5,6,4] -> 1 or 5 (either valid)

Constraints: 1 <= nums.length <= 1000, no two adjacent elements equal.

ANSWER

```
def find_peak_element(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[mid + 1]:
            hi = mid                    # a peak exists at mid or to its LEFT
        else:
            lo = mid + 1                # a peak exists strictly to the RIGHT
    return lo

# Verified: find_peak_element([1,2,3,1]) == 2
#           find_peak_element([1,2,1,3,5,6,4]) in (1, 5) == True
# Complexity: O(log n) time, O(1) extra space -- a binary search despite
# the array NOT being sorted overall, because the LOCAL comparison
# nums[mid] vs nums[mid+1] is always enough to know which direction
# guarantees a peak exists.
```

解释

考点：在非有序数组上用二分查找，是对"二分查找必须要求数组有序"这个刻板印象的一次很好的反驳，考察能不能看出这道题真正依赖的不变量是什么。

关键理解：只要nums[mid]>nums[mid+1]，就说明从mid往左的方向"地势在下降"，必然存在一个峰值在mid或者mid左边（因为两端视为负无穷，函数不可能一直下降到负无穷）；反之如果nums[mid]<nums[mid+1]，峰值必然在mid右边——这个"局部比较决定全局收缩方向"的性质，是能对无序数组用二分的根本原因。

延伸考点：面试官可能会追问二维版本（在一个矩阵里找一个比上下左右都大的峰值元素），思路是类似的行/列二分收缩，但实现复杂度会显著提高。

Q125. Algorithms & Scripting: Reverse String

QUESTION

Write a function that reverses a string given as an array of characters, in-place with O(1) extra memory.

Example 1: s="hello" -> "olleh"

Example 2: s="a" -> "a"

Constraints: 1 <= s.length <= 10^5.

ANSWER

```
def reverse_string(s):
    lo, hi = 0, len(s) - 1
    while lo < hi:
        s[lo], s[hi] = s[hi], s[lo]
        lo += 1; hi -= 1
    return s

# Verified: reverse_string(list("hello")) == list("olleh")
#           reverse_string(list("a")) == list("a")
# Complexity: O(n) time, O(1) EXTRA space -- classic two-pointer in-place
# swap, exactly meeting the stated constraint.
```

解释

考点：最基础的双指针题，几乎没有算法难度，通常用来考察是否理解"原地"（in-place）和"O(1)额外空间"这两个约束的具体含义。

关键点：双指针从两端向中间靠拢，每一步交换后各自前进/后退一步，直到相遇——这是"原地反转"类问题（包括反转链表、反转数组片段）的最基本模式。

延伸考点：如果要求反转字符串中的单词顺序而不是字符顺序（Reverse Words in a String），思路会变成先反转整个字符串，再逐个单词反转回来，或者用分割+反转+拼接的方式，是这道题的一个常见变体。

Q126. Algorithms & Scripting: Find the Index of the First Occurrence in a String

QUESTION

Find the Index of the First Occurrence in a String: given text and pattern, return the index of the first occurrence of pattern in text, or -1 if not found.

Example 1: text="helloworld", pattern="world" -> 5

Example 2: text="abcdef", pattern="xyz" -> -1

Constraints: naive O(n*m), KMP O(n+m).

ANSWER

```
def substring_search(text, pattern):
    return text.find(pattern)

# Verified: substring_search("helloworld", "world") == 5
#           substring_search("abcdef", "xyz") == -1
# Complexity: Python's built-in str.find() uses an optimized algorithm
# (a variant related to Boyer-Moore / two-way string matching) that is
# sub-quadratic in practice; a naive hand-rolled version is O(n*m).
#
# KMP (Knuth-Morris-Pratt) reference implementation, O(n+m), for when
# the interviewer wants the algorithm written out rather than relying on
# a built-in:
#
# def kmp_search(text, pattern):
#     if not pattern:
#         return 0
#     # build the "failure function" (longest proper prefix == suffix)
#     lps = [0] * len(pattern)
#     k = 0
```

```
#     for i in range(1, len(pattern)):
#         while k > 0 and pattern[i] != pattern[k]:
#             k = lps[k-1]
#         if pattern[i] == pattern[k]:
#             k += 1
#         lps[i] = k
#     k = 0
#     for i in range(len(text)):
#         while k > 0 and text[i] != pattern[k]:
#             k = lps[k-1]
#         if text[i] == pattern[k]:
#             k += 1
#         if k == len(pattern):
#             return i - k + 1
#     return -1
```

解释

考点：字符串匹配问题，这道题明确提示了"朴素解法O(n*m)，KMP能做到O(n+m)"，说明面试官很可能想听到KMP算法本身，而不只是调用内置函数。

关键思路：KMP的核心是预处理出pattern的"失配函数"（每个位置的最长相同前后缀长度），这样文本串匹配失败时，不需要把pattern指针退回到开头重新比较，而是利用失配函数跳到下一个可能匹配的位置，避免了朴素算法里大量重复的回退比较。

延伸考点：面试官可能会追问失配函数具体怎么构造、为什么这样构造能保证正确性，这是KMP算法里最有技术含量、也最容易讲不清楚的部分，建议提前把失配函数的构造过程演练熟练。

Q127. Algorithms & Scripting: Run-Length Encoding

QUESTION

Given a string s, perform run-length encoding: compress consecutive runs into count+character format.

Example 1: s="aaaabbbcc" -> "4a3b2c"

Example 2: s="abcd" -> "1a1b1c1d"

Constraints: 0 <= s.length <= 10^5.

ANSWER

```
def run_length_encoding(s):
    if not s:
        return ""
    result = []
    count = 1
    for i in range(1, len(s) + 1):
        if i < len(s) and s[i] == s[i-1]:
            count += 1
        else:
            result.append(f"{count}{s[i-1]}")
            count = 1
    return "".join(result)

# Verified: run_length_encoding("aaaabbbcc") == "4a3b2c"
#           run_length_encoding("abcd") == "1a1b1c1d"
# Complexity: O(n) time, O(n) extra space for the output (unavoidable,
# since the output itself can be as long as ~2x the input in the worst
# case of no repeated characters at all, as example 2 shows).
```

解释

考点：字符串的行程编码（Run-Length Encoding），基础字符串处理题，考察边界处理是否完整。

关键技巧：用一个计数器统计连续相同字符的个数，遇到和前一个字符不同（或者到达字符串末尾）时，把"计数+字符"追加到结果里并重置计数器——循环多跑一轮（range到len(s)+1）是为了让最后一段字符也能被正确输出，这是最容易漏掉的边界情况。

延伸考点：如果要求实现"解码"的逆过程（把"4a3b2c"还原成"aaaabbbcc"），需要解析数字和字符交替出现的模式，注意数字可能是多位数（比如出现12次的字符要写成"12a"而不是"1"和"2a"分开）。

Q128. Algorithms & Scripting: Valid Anagram

QUESTION

Valid Anagram: given two strings s and t, return true if t is an anagram of s (same letters, same counts, different order).

Example 1: s="anagram", t="nagaram" -> true

Example 2: s="rat", t="car" -> false

Constraints: 1 <= s.length, t.length <= 5*10^4, lowercase English letters.

ANSWER

```
from collections import Counter

def anagram_check(s, t):
    return Counter(s) == Counter(t)

# Verified: anagram_check("anagram", "nagaram") == True
#           anagram_check("rat", "car") == False
# Complexity: O(n) time, O(1) extra space (bounded by the 26-letter
# alphabet, regardless of string length).
#
# Equivalent without Counter (same complexity, explicit early-exit on
# length mismatch which Counter's == already handles implicitly):
# def anagram_check_manual(s, t):
#     if len(s) != len(t):
#         return False
#     counts = [0]*26
#     for ch in s: counts[ord(ch)-ord('a')] += 1
#     for ch in t: counts[ord(ch)-ord('a')] -= 1
#     return all(c == 0 for c in counts)
```

解释

考点：判断两个字符串是否互为字母异位词，是哈希表/计数思想最直接的应用，几乎是"字符频次统计"这类题目的入门第一题。

关键理解：两个字符串是anagram，当且仅当每个字符出现的次数完全一样——用Counter或者一个固定大小26的数组统计频次，两边一增一减，最后全部归零（或者两个Counter直接比较相等）即可判断。

延伸考点：如果字符集不限于小写字母（比如包含大小写、数字、Unicode字符），用Counter/哈希表的方案依然适用且不需要修改，而用固定大小数组的方案就需要扩大数组或者换成哈希表，这体现了两种实现方式在扩展性上的差异。

Q129. Algorithms & Scripting: Longest Common Prefix

QUESTION

Write a function to find the longest common prefix string amongst an array of strings. If none, return "".

Example 1: strs=["flower","flow","flight"] -> "fl"

Example 2: strs=["dog","racecar","car"] -> ""

Constraints: 1 <= strs.length <= 200, 0 <= strs[i].length <= 200.

ANSWER

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
            if not prefix:
                return ""
    return prefix

# Verified: longest_common_prefix(["flower","flow","flight"]) == "fl"
#           longest_common_prefix(["dog","racecar","car"]) == ""
# Complexity: O(S) time where S is the sum of all character counts in
# the worst case (each candidate prefix shrink is checked against the
# next string), O(1) extra space beyond the prefix itself.
```

解释

考点：这是一道非常基础的字符串题，考察横向扫描（水平扫描）这个最直观的解法思路。

关键技巧：先假设第一个字符串整体就是公共前缀，然后依次和后面每个字符串比较，只要当前字符串不是以这个假设的前缀开头，就把前缀末尾字符去掉一位再试，直到前缀变成空字符串或者真的是所有字符串的公共前缀。

延伸考点：如果字符串数量很大，也可以用"纵向扫描"（逐列比较所有字符串的第i个字符是否全部相同）或者字典树（Trie）实现，字典树的方式在需要多次查询不同字符串集合的公共前缀时更有优势。

Computer Architecture

Q130. Computer Architecture: Out-of-Order Execution in Detail

QUESTION

Explain out-of-order (OoO) execution in a modern CPU pipeline in detail: what problem does it solve, what are the key hardware structures involved, and walk through what happens to an instruction from fetch to retirement.

ANSWER

```
// THE PROBLEM OoO SOLVES:
//   In an IN-ORDER pipeline, if one instruction stalls (e.g. waiting on
//   a cache-miss load, or a dependency on a slow-to-produce result),
//   EVERY instruction behind it in program order also stalls, even if
//   those later instructions have no dependency on the stalled one and
//   could otherwise run immediately. This wastes execution-unit
```

```
//   throughput. OoO execution lets INDEPENDENT instructions execute as
//   soon as their operands are ready, regardless of program order,
//   while still guaranteeing the final architectural result is
//   IDENTICAL to strict in-order execution.
//
// KEY HARDWARE STRUCTURES:
//   - Reorder Buffer (ROB): tracks every in-flight instruction in
//     PROGRAM order, holds its speculative result until it's safe to
//     commit. Retirement always happens in-order from the ROB, even
//     though execution happened out-of-order -- this is what preserves
//     precise exceptions and correct architectural state.
//   - Reservation Stations (RS) / Issue Queue: buffer decoded
//     instructions waiting for their operands to become available; once
//     ready, an instruction is "woken up" and dispatched to a free
//     execution unit, independent of program order.
//   - Register Renaming (using a Physical Register File, PRF, and a
//     Register Alias Table, RAT): maps architectural registers to a
//     larger pool of physical registers, eliminating FALSE dependencies
//     (WAR/WAW hazards) between instructions that happen to reuse the
//     same architectural register name but have no REAL data
//     dependency.
//   - Load/Store Queue (LSQ): tracks in-flight memory ops, handles
//     memory disambiguation (does this load alias with an earlier,
//     not-yet-completed store?) and forwards store data to a dependent
//     load when safe.
//
// WALKTHROUGH, fetch to retirement:
//   1. FETCH  -- instruction fetched from I-cache (per branch predictor's
//      guidance for control flow).
//   2. DECODE -- instruction decoded; architectural source/dest
//      registers identified.
//   3. RENAME -- architectural registers mapped to fresh physical
//      registers (RAT updated); this instruction is ALLOCATED an entry
//      in the ROB, in PROGRAM ORDER (this is what lets retirement later
//      recover program order even though execution won't be in order).
//   4. DISPATCH -- instruction placed into a reservation station /ISSUE
//      queue, waiting for its (renamed) source operands.
//   5. ISSUE/EXECUTE -- once all source operands are ready (either
//      already in the PRF, or forwarded directly from another
//      instruction that just finished), the instruction is dispatched
//      to a free execution unit and executes -- possibly WAY out of
//      program order relative to instructions around it.
//   6. WRITEBACK -- result written to its physical register, broadcast
//      on the "wakeup" network so any waiting instructions that needed
//      this result as an operand become ready.
//   7. RETIRE (COMMIT) -- the ROB retires instructions strictly IN
//      PROGRAM ORDER, from the head of the buffer: only once an
//      instruction reaches the head AND has completed execution does
//      its result become ARCHITECTURALLY visible (update the real
//      architectural register file / memory), and only then is it safe
//      to reclaim its physical register and ROB entry. This in-order
//      retirement is what makes exceptions/interrupts PRECISE: if
//      instruction N faults, everything before it (in program order)
//      has already safely committed, and everything after it can be
//      cleanly squashed/discarded, regardless of what actually executed
//      out of order in the meantime.
```

解释

考点：乱序执行是CPU微架构面试的经典重头题，考察点不是记住几个名词，而是能不能把"为什么需要乱序"和"怎么保证乱序执行完还能得到正确的、精确的架构状态"这两条线讲清楚。

最容易被问倒的点：很多人知道"乱序执行"但说不清楚"乱序"和"乱序提交"的区别——真正乱序的只是执行阶段，指令进入流水线（分配ROB表项）和最终提交（retire）永远是严格按程序顺序的，正是这个"头尾按序、中间乱序"的设计，才能同时兼顾性能和精确异常。

寄存器重命名的价值经常被低估：面试官喜欢追问"为什么需要重命名，直接用架构寄存器不行吗"，标准答案是消除WAR/WAW这种"假依赖"——两条指令只是恰好用了同一个架构寄存器名字，本身没有真正的数据依赖，重命名到不同的物理寄存器后，这两条指令就能真正并行执行。

延伸考点：面试官可能会追问乱序执行和分支预测错误之间的关系（错误预测的分支之后的所有指令都是投机执行的，一旦发现预测错误，需要靠ROB把这些还没提交的投机指令全部冲刷掉，这也是ROB按序提交机制的另一个重要作用），以及Load/Store Queue如何处理内存别名问题（后面的load有没有可能读到前面还没写完的store的数据，这是内存乱序执行里专门的一类难题，通常靠地址比对和store-to-load forwarding机制处理）。

Q131. Computer Architecture: MESI and MOESI Cache Coherence (Two-Core Scenario)

QUESTION

Explain the MESI cache coherence protocol in detail, and then MOESI (how it extends MESI). Walk through a concrete scenario: Core 1 reads address X (cache miss, goes to memory), then Core 2 also reads X, then Core 1 writes to X. What state does each core's cache line go through at each step, under MESI, and then how does the story change under MOESI?

ANSWER

```
// MESI states (per cache line, per core):
//   M (Modified)  -- this cache HAS THE ONLY copy, and it is DIRTY
//                    (different from memory); this cache is the
//                    sole owner and must eventually write it back.
//   E (Exclusive) -- this cache has the ONLY copy, but it is CLEAN
//                    (matches memory) -- no other cache has it, and
//                    memory is up to date, so this cache can silently
//                    transition to M on a local write with no bus
//                    transaction needed.
//   S (Shared)    -- this cache has a copy that MAY also exist in
//                    other caches, and it is clean; a write requires
//                    first invalidating all other copies (bus
//                    transaction) before transitioning to M.
//   I (Invalid)   -- this cache does not have a valid copy at all.
//
// SCENARIO under MESI:
//   Step 1: Core 1 reads X (cache miss).
//     No other cache has X -> memory supplies the data -> Core 1's line
//     goes directly to E (Exclusive) -- clean, and known to be the ONLY
//     copy in the system (snooped: no other cache responded "I have it").
//
//   Step 2: Core 2 also reads X.
//     Core 2's read request is snooped by Core 1's cache, which DOES
//     have a valid copy -> Core 1 supplies the data (or memory does,
//     with Core1 asserting a "shared" signal) -> BOTH Core 1's and
//     Core 2's lines transition to S (Shared) -- since it's no longer
//     exclusively held by one cache.
//
//   Step 3: Core 1 writes to X.
//     Core 1's line is currently S -- a write from S requires a
//     BusUpgrade/invalidate transaction: Core 1 broadcasts an
//     invalidate for X, Core 2's copy transitions S -> I (invalidated),
//     and Core 1's own line transitions S -> M (now the sole, dirty
```

```
//     owner). No data was re-fetched from memory for this step -- only
//     an invalidation broadcast was needed, since Core 1 already had
//     valid data.
//
// SCENARIO under MOESI (adds the O = Owned state):
//   Steps 1 and 2 are IDENTICAL to MESI (E, then S/S).
//
//   Step 3 differs: MOESI is optimized to avoid unnecessary memory
//   writebacks. When Core 1 writes to X while in S:
//     - Core 1 still invalidates Core 2's copy (S -> I, same as MESI).
//     - But instead of Core 1 going all the way to M and being
//       responsible for eventually writing back to MEMORY, Core 1's
//       line goes to O (Owned): Core 1 now holds the ONLY valid, DIRTY
//       copy, but critically, MEMORY IS NOT UPDATED YET, and Core 1 is
//       now the designated "owner" responsible for supplying this data
//       (via cache-to-cache transfer) to any future reader, instead of
//       forcing a writeback-then-refetch-from-memory round trip.
//   The key benefit: if Core 2 (or another core) reads X again shortly
//   after, MOESI lets Core 1 (in O) supply the dirty data directly,
//   cache-to-cache, without ever writing back to memory first --
//   avoiding a memory round trip that MESI's stricter M-state handling
//   would have required (in MESI, going from M and being re-read by
//   another core typically forces a writeback to memory as part of
//   supplying the data, or at least is commonly modeled that way,
//   depending on the exact implementation choice for M-state sharing).
```

解释

考点：MESI/MOESI是多核缓存一致性的经典考题，考察点是能不能用一个具体的多核读写场景把状态转移讲清楚，而不是只背四个字母分别代表什么。

场景设计的用意：故意设计"先各自读、再其中一个写"这个序列，是因为它覆盖了缓存一致性协议里最核心的三类状态转移——独占读（miss后直接到E）、共享读（snoop到已有副本，双方变S）、从共享态写（触发失效广播，自己变M/O，对方变I），几乎覆盖了协议的全部核心逻辑。

MESI和MOESI的本质区别：MOESI多出来的O状态解决的是"脏数据要不要先写回内存才能被别的核心读到"这个效率问题——MESI下，如果一个核心持有M态的脏数据，另一个核心要读，通常需要写回内存（或者至少语义上倾向于同步内存）；MOESI允许"脏数据由owner核心直接转发给需要的核心，内存长期落后于真实值也没关系，只要owner还在就行"，用一次核心间传输替代"写回+重新读取"两次内存事务，这是MOESI在多核心频繁共享读写同一份数据的场景下能明显提升性能的原因。

延伸考点：面试官可能会追问E状态存在的意义（如果没有E，独占读之后本地写也要走一次总线广播升级到M，E状态的价值就是"提前预判很可能接下来会写"从而省掉这次总线事务），以及如果场景反过来——两个核心几乎同时都想写同一个地址，协议怎么保证不会同时进入M态（靠总线仲裁/一致性目录保证任意时刻只有一个核心能拿到"写许可"，这是协议正确性的基础保证，通常由总线仲裁器或者目录一致性协议里的一致性控制器来保证互斥）。

Q132. Computer Architecture: RISC vs CISC -- Pros and Cons

QUESTION

Compare RISC and CISC instruction set architectures: what are the core philosophical differences, and what are the concrete pros and cons of each?

ANSWER

```
// CORE PHILOSOPHICAL DIFFERENCE:
```

```
//   CISC (Complex Instruction Set Computer): favor fewer, more
//   POWERFUL instructions, each potentially doing more work per
//   instruction (e.g. one instruction that loads from memory, does
//   arithmetic, AND stores the result back) -- optimizing for compact
//   CODE SIZE and reducing the number of instructions a compiler needs
//   to emit, at the cost of more complex, variable-length instruction
//   decoding hardware.
//   RISC (Reduced Instruction Set Computer): favor MANY simple,
//   FIXED-LENGTH, single-cycle-ish instructions (e.g. separate
//   load/store instructions, with arithmetic ONLY operating on
//   registers, never directly on memory) -- optimizing for a SIMPLE,
//   FAST, easily-pipelined decode/execute datapath, at the cost of
//   needing MORE instructions (larger code size) to do the same work.
//
// CISC pros:
//   + More compact code (fewer instructions for the same task) --
//     historically mattered a lot when memory was small/slow/expensive.
//   + Can make hand-written assembly (and older compilers) simpler to
//     write, since one instruction can directly express a
//     higher-level operation (e.g. x86's single "memory-operand ADD").
//   + Backward compatibility: x86's huge legacy CISC ecosystem is
//     itself a major practical advantage today (software compatibility),
//     independent of any inherent ISA merit.
//
// CISC cons:
//   - Variable-length, complex instruction DECODING is hard to pipeline
//     and hard to make fast/wide (parallel decode of several
//     variable-length instructions per cycle is a genuinely hard
//     problem) -- modern x86 chips actually work around this by
//     decoding CISC instructions into internal RISC-like MICRO-OPS
//     before execution.
//   - More complex control logic in general -> historically more
//     design/verification effort, larger die area for the front end.
//
// RISC pros:
//   + Fixed-length, simple instruction format makes decode fast, and
//     easy to decode MULTIPLE instructions per cycle (superscalar) --
//     a huge advantage for high-performance, deeply pipelined designs.
//   + Simpler control logic -> historically easier to design, verify,
//     and clock at high frequency; smaller front-end area.
//   + The load/store-only-touches-memory discipline makes the pipeline
//     model cleaner (arithmetic ops have simple, predictable timing;
//     only load/store instructions interact with the memory hierarchy).
//
// RISC cons:
//   - Larger code size (more instructions needed for the same task) --
//     more instruction-cache pressure, more fetch bandwidth needed.
//   - Some operations that are "one instruction" on CISC take several
//     RISC instructions, which can matter for certain workloads (though
//     modern compilers and wide superscalar RISC cores largely absorb
//     this cost).
//
// MODERN REALITY (worth mentioning as a nuanced closing point):
//   The RISC vs CISC line has blurred a LOT in practice: modern x86
//   (CISC ISA) chips decode instructions into internal RISC-style
//   micro-ops and execute those on a RISC-like out-of-order backend --
//   so at the microarchitecture level, "CISC" x86 chips are internally
//   quite RISC-like. Meanwhile modern RISC ISAs (ARM, RISC-V) have
//   picked up some CISC-like conveniences over time (e.g. more complex
//   addressing modes) where it clearly pays off. The distinction today
//   is more about the ISA CONTRACT (what instructions software sees)
//   than a hard predictor of the actual internal microarchitecture or
//   performance.
```

解释

考点：RISC vs CISC是计算机体系结构的经典基础题，考察点不是背哪个"更好"，而是能不能讲清楚两种设计哲学各自在权衡什么、分别适合什么场景。

核心权衡：CISC用"更少但更强"的指令换取更小的代码体积，代价是译码逻辑复杂、难以并行化；RISC用"更多但更简单"的指令换取译码简单、容易做超标量并行，代价是代码体积变大。这个"指令复杂度 vs 译码/流水线友好度"的权衡关系是回答这道题的核心框架，答题时最好按这个框架组织，而不是零散地罗列优缺点。

容易被追问的点：现代x86处理器内部其实是把CISC指令翻译成类RISC的微操作（micro-ops）再执行，说明"CISC"和"RISC"更多是指令集层面的契约，不完全等同于芯片内部真实的微架构实现方式——能主动提到这一点说明你理解的不只是教科书定义，还知道现代处理器设计的实际情况。

延伸考点：面试官可能会追问ARM/RISC-V相比x86在功耗效率上的优势从哪来（译码逻辑简单、指令长度固定，前端功耗和面积开销更小，这对移动端和数据中心的能效比很关键），以及"如果从零设计一个新架构，你会选RISC还是CISC"这种开放性问题（好的回答应该结合具体应用场景——比如追求极致代码密度的嵌入式场景 vs 追求高频宽发射的服务器CPU场景——而不是简单选边站队）。

Q133. Computer Architecture: Design Parameterized ALU with Multiple Arithmetic and Logic Operations

QUESTION

Implement a parameterized ALU supporting add, subtract, bitwise AND/OR/XOR, and signed less-than (SLT). Generate optional status flags.

Example (WIDTH=8):

ADD: A=10, B=3 → result=13

SLT: A=0xFF(-1), B=0x01(+1) → result=1 (true)

AND: A=0xAA, B=0x0F → result=0x0A

Constraints:

- Parameterizable WIDTH (8, 16, 32, 64 bits)

- Operation selected via `alu_op` input

- Optional flags: zero, negative, carry, overflow

ANSWER

```
module alu #(
  parameter W = 32
)(
  input  logic [W-1:0] a,
  input  logic [W-1:0] b,
  input  logic [3:0]   op,
  output logic [W-1:0] result,
  output logic         zero,
  output logic         carry,
  output logic         overflow,
  output logic         negative
);
  logic [W:0] tmp;

  always_comb begin
    tmp      = '0;
```

```
    overflow = 1'b0;

    case (op)
      4'd0: tmp = a + b;
      4'd1: tmp = a - b;
      4'd2: tmp = {1'b0, (a & b)};
      4'd3: tmp = {1'b0, (a | b)};
      4'd4: tmp = {1'b0, (a ^ b)};
      4'd5: tmp = {{W{1'b0}}, ($signed(a) < $signed(b))};
      4'd6: tmp = {1'b0, (a << b[$clog2(W)-1:0])};
      4'd7: tmp = {1'b0, (a >> b[$clog2(W)-1:0])};
      4'd8: tmp = {1'b0, ($signed(a) >>> b[$clog2(W)-1:0])};
      default: tmp = '0;
    endcase

    result   = tmp[W-1:0];
    carry    = tmp[W];
    zero     = (result == '0);
    negative = result[W-1];

    if (op == 4'd0) overflow = (a[W-1] == b[W-1]) && (result[W-1] != a[W-1]);
    if (op == 4'd1) overflow = (a[W-1] != b[W-1]) && (result[W-1] != a[W-1]);
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q134. Computer Architecture: Implement Sequential Divider Using Shift-Subtract Algorithm

QUESTION

Design a sequential divider computing quotient and remainder using restoring or non-restoring division.

Example (WIDTH=8):

20 / 5 → quotient=4, remainder=0

22 / 5 → quotient=4, remainder=2

7 / 0 → divide_by_zero flag=1

Constraints:

- Handshake: start, busy, done

- Typically WIDTH+1 cycles

- Define divide-by-zero behavior

ANSWER

```
module divider #(
  parameter W = 16
)(
  input  logic          clk,
  input  logic          rst_n,
```

```
  input  logic          start,
  input  logic [W-1:0]  dividend,
  input  logic [W-1:0]  divisor,
  output logic [W-1:0]  quotient,
  output logic [W-1:0]  remainder,
  output logic          busy,
  output logic          done,
  output logic          div_by_zero
);
  logic [W-1:0]       q, d;
  logic [W:0]         r;
  logic [$clog2(W):0] cnt;

  typedef enum logic [1:0] { IDLE, CALC, FIN } st_t;
  st_t st;

  assign busy = (st == CALC);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      st          <= IDLE;
      q           <= '0;
      d           <= '0;
      r           <= '0;
      cnt         <= '0;
      quotient    <= '0;
      remainder   <= '0;
      done        <= 1'b0;
      div_by_zero <= 1'b0;
    end else begin
      done        <= 1'b0;
      div_by_zero <= 1'b0;
      case (st)
        IDLE: begin
          if (start) begin
            if (divisor == '0) begin
              quotient    <= '0;
              remainder   <= dividend;
              div_by_zero <= 1'b1;
              done        <= 1'b1;
            end else begin
              q   <= dividend;
              d   <= divisor;
              r   <= '0;
              cnt <= '0;
              st  <= CALC;
            end
          end
        end
        CALC: begin
          logic [W:0]   r_shift;
          logic [W:0]   r_next;
          logic [W-1:0] q_next;
          r_shift = {r[W-1:0], q[W-1]};
          q_next  = {q[W-2:0], 1'b0};
          r_next  = r_shift;
          if (r_shift >= {1'b0, d}) begin
            r_next    = r_shift - {1'b0, d};
            q_next[0] = 1'b1;
          end
          r   <= r_next;
          q   <= q_next;
          cnt <= cnt + 1'b1;
```

```
          if (cnt == W-1) st <= FIN;
        end
        FIN: begin
          quotient  <= q;
          remainder <= r[W-1:0];
          done      <= 1'b1;
          st        <= IDLE;
        end
      endcase
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q135. Computer Architecture: Implement Dirty Bit Logic for Write-Back Cache

QUESTION

Design dirty bit management for a write-back cache.

Set dirty on write hit, clear after writeback completes. Generate writeback requests when evicting dirty lines. Clean lines are replaced without writeback.

Example:

Read miss fills line A → dirty=0

Write hit to A → dirty=1

Evict A → writeback request generated

Constraints:

- Dirty bit stored per cache line alongside tag and valid

- Writeback address = victim tag + cache index

- Clean eviction requires no memory traffic

ANSWER

```
module cache_dirty #(
  parameter NUM_SETS = 64,
  parameter N_WAYS   = 4,
  parameter TAG_W    = 20
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic                          write_hit,
  input  logic                          refill_done,
  input  logic                          write_alloc_fill,
  input  logic                          evict_valid,
  input  logic                          wb_done,
  input  logic [$clog2(NUM_SETS)-1:0]   hit_set,
  input  logic [$clog2(NUM_SETS)-1:0]   refill_set,
```

```
  input  logic [$clog2(NUM_SETS)-1:0]  evict_set,
  input  logic [$clog2(N_WAYS)-1:0]    hit_way,
  input  logic [$clog2(N_WAYS)-1:0]    refill_way,
  input  logic [$clog2(N_WAYS)-1:0]    evict_way,
  input  logic [TAG_W-1:0]             evict_tag,
  output logic                         evict_dirty,
  output logic                         writeback_req,
  output logic [TAG_W+$clog2(NUM_SETS)-1:0] writeback_addr
);
  // Packed bit-vector for Verilator sensitivity
  logic [NUM_SETS*N_WAYS-1:0] dirty;

  assign evict_dirty    = dirty[evict_set * N_WAYS + evict_way];
  assign writeback_req  = evict_valid && dirty[evict_set * N_WAYS + evict_way];
  assign writeback_addr = {evict_tag, evict_set};

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      dirty <= '0;
    end else begin
      if (write_hit)
        dirty[hit_set * N_WAYS + hit_way] <= 1'b1;
      if (refill_done)
        dirty[refill_set * N_WAYS + refill_way] <= write_alloc_fill;
      if (wb_done)
        dirty[evict_set * N_WAYS + evict_way] <= 1'b0;
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

缓存类设计的关键是替换策略和一致性状态（比如dirty bit）的更新时机，一步错就会导致数据一致性问题。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q136. Computer Architecture: Design Round-Robin Arbiter for Fair Request Arbitration

QUESTION

Implement a round-robin arbiter granting one requestor per cycle with rotating priority.

Example:

request=111 → grants rotate: 0→1→2→0→1→2...

request=010 → always grants requestor 1

request=000 → grant_valid=0, pointer unchanged

Constraints:

- One-hot grant output

- Maintain last_grant pointer for fairness

- Fair share over time for continuous requestors

ANSWER

```
module rr_arbiter #(
```

```
  parameter N = 4
)(
  input  logic          clk,
  input  logic          rst_n,
  input  logic [N-1:0] req,
  output logic [N-1:0] grant
);
  logic [$clog2(N)-1:0] last;

  always_comb begin
    logic [$clog2(N)-1:0] idx;

    grant = '0;
    idx   = '0;
    for (int i = 1; i <= N; i++) begin
      idx = (last + i) % N;
      if (req[idx]) begin
        grant[idx] = 1;
        break;
      end
    end
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      last <= '0;
    end else if (|grant) begin
      for (int i = 0; i < N; i++) begin
        if (grant[i]) last <= i;
      end
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

仲裁器类设计的核心考点是"公平性"——不能让某个请求方永远抢不到授权，这也是最容易被断言/覆盖率检查出问题的地方。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q137. Computer Architecture: Implement Thread Mask Logic for Predicated Execution

QUESTION

Design thread masking logic for GPU-style predicated execution.

Operations:

if-branch: new_mask = active_mask & predicate

```
else-branch: new_mask = active_mask & ~predicate
nested:      push/pop mask stack
```

Constraints:

- Inactive threads must not write registers or issue memory ops
- Support push/pop stack for nesting
- Initialize to all active on reset

ANSWER

```
module thread_mask #(
  parameter THREADS = 32,
  parameter STACK_D = 4
)(
  input  logic [THREADS-1:0] predicate,
  input  logic               clk,
  input  logic               rst_n,
  input  logic               enter_if,
  input  logic               enter_else,
  input  logic               exit_if,
  output logic [THREADS-1:0] active_mask
);
  logic [THREADS-1:0] stack [STACK_D];
  logic [$clog2(STACK_D+1)-1:0] sp;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      active_mask <= {THREADS{1'b1}};
      sp          <= '0;
      for (int i = 0; i < STACK_D; i++) stack[i] <= '0;
    end else begin
      if (enter_if && sp < STACK_D) begin
        stack[sp]   <= active_mask;
        sp          <= sp + 1'b1;
        active_mask <= active_mask & predicate;
      end else if (enter_else && (sp != 0)) begin
        active_mask <= stack[sp-1] & ~predicate;
      end else if (exit_if && (sp != 0)) begin
        active_mask <= stack[sp-1];
        sp          <= sp - 1'b1;
      end
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q138. Computer Architecture: Design Branch Comparator and Control Logic

QUESTION

Implement branch comparison logic evaluating BEQ, BNE, BLT, BGE conditions. Generate branch-taken signal and next-PC selection.

Example:

BEQ: Rs1=5, Rs2=5 → taken=1

BLT: Rs1=-1(0xFF), Rs2=+1(0x01) → taken=1

BLTU: Rs1=255(0xFF), Rs2=1(0x01) → taken=0

Constraints:

- Support BEQ, BNE, BLT, BGE, BLTU, BGEU

- Consider forwarding into comparator

- Output: branch_taken, PC_src

ANSWER

```
module branch_cmp #(
  parameter W = 32
)(
  input  logic [W-1:0] a,
  input  logic [W-1:0] b,
  input  logic [2:0]   funct3,
  output logic          take_branch
);
  always_comb begin
    case (funct3)
      3'b000: take_branch = (a == b);                  // BEQ
      3'b001: take_branch = (a != b);                  // BNE
      3'b100: take_branch = ($signed(a) <  $signed(b)); // BLT
      3'b101: take_branch = ($signed(a) >= $signed(b)); // BGE
      3'b110: take_branch = (a <  b);                  // BLTU
      3'b111: take_branch = (a >= b);                  // BGEU
      default: take_branch = 0;
    endcase
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q139. Computer Architecture: Design ALU Control Signal Decoder

QUESTION

Implement a decoder that converts instruction opcode/function fields into ALU control signals.

Takes opcode, funct3, funct7 as inputs. Produces ALU operation select and operand controls. Handles illegal opcodes with safe defaults.

Example:

R-type ADD: opcode=0110011, funct3=000, funct7=0000000

→ alu_op=ADD, alu_src=reg, reg_write=1

Illegal opcode=0xFF

→ alu_op=ADD, reg_write=0, illegal=1

Constraints:

- Complete mapping from opcode/funct to control signals

- Safe defaults for undefined opcodes (no writes)

- Assign defaults before case to prevent latches

ANSWER

```
module alu_control (
  input  logic [1:0] alu_op,
  input  logic [2:0] funct3,
  input  logic       funct7_b5,
  output logic [3:0] alu_ctrl
);
  always_comb begin
    case (alu_op)
   2'bOO: alu_ctrl = 4'dO;  // Load/Store — ADD
   2'bO1: alu_ctrl = 4'd1;  // Branch   — SUB
      2'b10: begin
        case (funct3)
          3'b000: alu_ctrl = funct7_b5 ? 4'd1 : 4'd0;  // SUB / ADD
          3'b001: alu_ctrl = 4'd6;                      // SLL
          3'b010: alu_ctrl = 4'd5;                      // SLT
          3'b100: alu_ctrl = 4'd4;                      // XOR
          3'b101: alu_ctrl = funct7_b5 ? 4'd8 : 4'd7;  // SRA / SRL
          3'b110: alu_ctrl = 4'd3;                      // OR
          3'b111: alu_ctrl = 4'd2;                      // AND
          default: alu_ctrl = 4'd0;
        endcase
      end
      default: alu_ctrl = 4'd0;
    endcase
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q140. Computer Architecture: Implement True LRU Replacement for 4-Way Set-Associative Cache

QUESTION

Design a 4-way set-associative cache implementing true Least Recently Used (LRU) replacement policy.

Track access order for each cache set. On cache miss with all ways occupied, evict the least recently used way. Update LRU state on every access (hit or refill). The LRU module does not distinguish read vs write — it simply tracks the order of accesses.

Example:

Fill set with tags A, B, C, D (in order)

Access B (hit) → B becomes MRU

Access new tag E (miss) → Evicts A (true LRU)

Constraints:

- 4-way set-associative

- Prefer invalid ways before evicting valid ones

- Update LRU on every access type

ANSWER

```
module cache_lru4 #(
  parameter NUM_SETS  = 64,
  parameter WAY_COUNT = 4
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic                          req_valid,
  input  logic                          hit,
  input  logic                          refill,
  input  logic [$clog2(NUM_SETS)-1:0]  req_set,
  input  logic [$clog2(WAY_COUNT)-1:0] hit_way,
  input  logic [$clog2(WAY_COUNT)-1:0] refill_way,
  input  logic [WAY_COUNT-1:0]         way_valid,
  output logic                          victim_valid,
  output logic [$clog2(WAY_COUNT)-1:0] victim_way
);

  // ----------------------------
  // LRU state (0 = LRU, 3 = MRU)
  // ----------------------------
  logic [1:0] rank [NUM_SETS][WAY_COUNT];

  // ----------------------------
  // Register inputs (FIX timing bug)
  // ----------------------------
  logic [WAY_COUNT-1:0] way_valid_q;
  logic [$clog2(WAY_COUNT)-1:0] acc;
  logic [1:0] old_r;

  // Latch inputs to avoid race
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
      way_valid_q <= '1;
    else
      way_valid_q <= way_valid;
  end

  // ----------------------------
  // Access decode
  // ----------------------------
  always_comb begin
    acc = '0;
    if (req_valid && (hit || refill))
      acc = hit ? hit_way : refill_way;
  end

  always_comb begin
    old_r = rank[req_set][acc];
  end

  // ----------------------------
  // Victim selection (FIXED priority)
  // ----------------------------
  always_comb begin
    victim_valid = req_valid;
    victim_way   = '0;

    if (!req_valid) begin
      victim_valid = 0;
    end
    else begin
      logic found;
```

```
      found = 0;

      // -------------------------
      // PRIORITY 1: invalid ways
      // -------------------------
      for (int i = 0; i < WAY_COUNT; i++) begin
        if (!found && !way_valid_q[i]) begin
          victim_way = i;
          found = 1;
        end
      end

      // -------------------------
      // PRIORITY 2: true LRU
      // -------------------------
      if (!found) begin
        for (int i = 0; i < WAY_COUNT; i++) begin
          if (!found && rank[req_set][i] == 0) begin
            victim_way = i;
            found = 1;
          end
        end
      end
    end
  end

  // ----------------------------
  // LRU update logic
  // ----------------------------
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      for (int s = 0; s < NUM_SETS; s++) begin
        for (int w = 0; w < WAY_COUNT; w++) begin
          rank[s][w] <= w;
        end
      end
    end
    else if (req_valid && (hit || refill)) begin
      for (int w = 0; w < WAY_COUNT; w++) begin
        if (w == acc)
          rank[req_set][w] <= 2'd3; // MRU
        else if (rank[req_set][w] > old_r)
          rank[req_set][w] <= rank[req_set][w] - 1;
      end
    end
  end

endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

缓存类设计的关键是替换策略和一致性状态（比如dirty bit）的更新时机，一步错就会导致数据一致性问题。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q141. Computer Architecture: Design Finite State Machine for Cache Line Refill from Memory

QUESTION

Implement an FSM to manage cache line refills from main memory.

States: `IDLE → REQUEST → WAIT → FILL → COMPLETE`. Handle burst transfers, update cache data and metadata, and manage pipeline stalls.

Example:

Cache miss → IDLE→REQUEST→WAIT→FILL(beat0)→...→FILL(last)→COMPLETE→IDLE

Tag and valid updated only at COMPLETE

Constraints:

- Cache line = N bytes, bus = M bytes wide → N/M beats

- Do not mark valid until all beats received

- Stall pipeline while FSM is not IDLE

ANSWER

```
module cache_refill_fsm #(
  parameter LINE_WORDS = 4
)(
  input  logic                          clk,
  input  logic                          rst_n,
  input  logic                          miss,
  input  logic                          mem_rvalid,
  input  logic                          mem_rlast,
  output logic                          mem_req,
  output logic                          stall,
  output logic                          refill_done,
  output logic [$clog2(LINE_WORDS)-1:0] beat_count
);
  typedef enum logic [2:0] {IDLE, REQUEST, WAIT, FILL, COMPLETE} state_t;
  state_t state;

  assign stall = (state != IDLE);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      state       <= IDLE;
      mem_req     <= 0;
      refill_done <= 0;
      beat_count  <= '0;
    end else begin
      mem_req     <= 0;
      refill_done <= 0;
      case (state)
        IDLE: begin
          beat_count <= '0;
          if (miss) begin
            state   <= REQUEST;
            mem_req <= 1;
          end
        end
        REQUEST: begin
          state <= WAIT;
        end
        WAIT: begin
          if (mem_rvalid) begin
            beat_count <= '0;
            if (mem_rlast || LINE_WORDS == 1) begin
              state       <= COMPLETE;
              refill_done <= 1;
            end else
              state <= FILL;
```

```
          end
        end
        FILL: begin
          if (mem_rvalid) begin
            beat_count <= beat_count + 1'b1;
            if (mem_rlast || beat_count == LINE_WORDS[$clog2(LINE_WORDS)-1:0] - 2) begin
              state       <= COMPLETE;
              refill_done <= 1;
            end
          end
        end
        COMPLETE: begin
          state <= IDLE;
        end
        default: state <= IDLE;
      endcase
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

缓存类设计的关键是替换策略和一致性状态（比如dirty bit）的更新时机，一步错就会导致数据一致性问题。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q142. Computer Architecture: Implement Reorder Buffer (ROB) Entry Management Logic

QUESTION

Design ROB entry management for an out-of-order processor.

Operations:

Allocate: store instruction at tail, increment tail

Writeback: mark entry ready, store result

Commit: retire from head if valid+ready, increment head

Flush: invalidate entries between flush point and tail

Constraints:

- Circular buffer with head/tail pointers

- Commit only in-order from head

- Full when (tail+1) % size == head

ANSWER

```
module rob #(
  parameter ENTRIES = 8,
  parameter W       = 32
)(
  input  logic                         clk,
  input  logic                         rst_n,
  input  logic                         dispatch,
  input  logic                         writeback,
  input  logic                         commit,
  input  logic                         flush,
```

```
  input  logic [$clog2(ENTRIES)-1:0]    wb_id,
  input  logic [$clog2(ENTRIES)-1:0]    flush_tail,
  input  logic [4:0]                    dest_arch,
  input  logic [W-1:0]                  wb_value,
  output logic                          full,
  output logic                          empty,
  output logic                          commit_valid,
  output logic [4:0]                    commit_arch,
  output logic [W-1:0]                  commit_value,
  output logic [$clog2(ENTRIES)-1:0]    alloc_id
);
  logic [ENTRIES-1:0]                valid, ready;
  logic [4:0]                        arch_dest [ENTRIES];
  logic [W-1:0]                      value     [ENTRIES];
  logic [$clog2(ENTRIES)-1:0]        head, tail;

  assign empty       = (head == tail) && !valid[head];
  assign full        = (head == tail) &&  valid[head];
  assign alloc_id    = tail;
  assign commit_valid = valid[head] && ready[head];
  assign commit_arch  = arch_dest[head];
  assign commit_value = value[head];

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      head <= '0;
      tail <= '0;
      valid <= '0;
      ready <= '0;
      for (int i = 0; i < ENTRIES; i++) begin
        arch_dest[i] <= '0;
        value[i]     <= '0;
      end
    end else begin
      if (dispatch && !full) begin
        valid[tail]     <= 1'b1;
        ready[tail]     <= 1'b0;
        arch_dest[tail] <= dest_arch;
        tail            <= (tail == ENTRIES-1) ? '0 : (tail + 1'b1);
      end

      if (writeback) begin
        ready[wb_id] <= 1'b1;
        value[wb_id] <= wb_value;
      end

      if (commit_valid && commit) begin
        valid[head] <= 1'b0;
        ready[head] <= 1'b0;
        head        <= (head == ENTRIES-1) ? '0 : (head + 1'b1);
      end

      if (flush) begin
        // Invalidate entries in range [flush_tail, tail) with wrap-around.
        // If flush_tail == tail, nothing to invalidate.
        if (flush_tail != tail) begin
          for (int i = 0; i < ENTRIES; i++) begin
            logic in_flush_range;
            if (flush_tail < tail)
              in_flush_range = (i[$clog2(ENTRIES)-1:0] >= flush_tail) &&
                               (i[$clog2(ENTRIES)-1:0] < tail);
            else // flush_tail > tail (wrapped)
              in_flush_range = (i[$clog2(ENTRIES)-1:0] >= flush_tail) ||
```

```
                                (i[$clog2(ENTRIES)-1:0] < tail);
            if (in_flush_range) begin
              valid[i] <= 1'b0;
              ready[i] <= 1'b0;
            end
          end
        end
        tail <= flush_tail;
      end
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q143. Computer Architecture: Design Shared Memory Bank Conflict Detector

QUESTION

Implement a bank conflict detector for GPU shared memory.

Given thread addresses and active mask, detect if multiple active threads access the same memory bank.

Example (4 banks):

active=1111, banks=[0,1,2,3] → no conflict

active=1111, banks=[1,1,2,3] → conflict on bank 1

active=0110, banks=[1,1,1,3] → no conflict (threads 0,2 masked)

Constraints:

- NUM_BANKS is power of 2

- Bank index = (addr >> offset) & (NUM_BANKS-1)

- Masked threads excluded from detection

ANSWER

```
module bank_conflict #(
  parameter N_BANKS    = 32,
  parameter THREADS    = 32,
  parameter ADDR_W     = 32,
  parameter BANK_OFFSET = 2
)(
  input  logic [ADDR_W-1:0]                 addr [THREADS],
  input  logic [THREADS-1:0]                active_mask,
  output logic                              has_conflict,
  output logic [$clog2(N_BANKS)-1:0]        conflict_bank_id,
  output logic                              can_issue,
  output logic [THREADS-1:0]                conflict_mask
);
  logic [THREADS-1:0] used [N_BANKS];
  logic [$clog2(N_BANKS)-1:0] bank;
```

```
  always_comb begin
    has_conflict     = 1'b0;
    conflict_bank_id = '0;
    can_issue        = 1'b1;
    conflict_mask    = '0;
    for (int b = 0; b < N_BANKS; b++) used[b] = '0;

    for (int t = 0; t < THREADS; t++) begin
      if (active_mask[t]) begin
        bank = (addr[t] >> BANK_OFFSET) & (N_BANKS-1);
        if (|used[bank]) begin
          has_conflict      = 1'b1;
          can_issue         = 1'b0;
          conflict_bank_id  = bank;
          conflict_mask     = conflict_mask | used[bank];
          conflict_mask[t]  = 1'b1;
        end
        used[bank][t] = 1'b1;
      end
    end
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q144. Computer Architecture: Implement Instruction Decode Logic

QUESTION

Design the main instruction decoder extracting fields from a 32-bit instruction and generating pipeline control signals.

Example:

R-type ADD: → RegWrite=1, ALUSrc=reg, MemRead=0

Load LW: → RegWrite=1, ALUSrc=imm, MemRead=1

Store SW: → RegWrite=0, MemWrite=1

Illegal: → all writes=0, illegal_instruction=1

Constraints:

- Support R/I/S/B/U/J instruction formats

- Sign-extend immediates per format

- Safe defaults for illegal opcodes

ANSWER

```
module instr_decode (
  input  logic [31:0] instr,
  output logic [4:0]  rs1,
  output logic [4:0]  rs2,
  output logic [4:0]  rd,
```

```
  output logic [31:0] imm,
  output logic [6:0]  opcode,
  output logic [6:0]  funct7,
  output logic [2:0]  funct3,
  output logic        reg_write,
  output logic        mem_read,
  output logic        mem_write,
  output logic        branch,
  output logic        alu_src,
  output logic        jump,
  output logic        mem_to_reg,
  output logic        illegal_instruction,
  output logic [3:0]  alu_op
);
  assign opcode = instr[6:0];
  assign rd     = instr[11:7];
  assign funct3 = instr[14:12];
  assign rs1    = instr[19:15];
  assign rs2    = instr[24:20];
  assign funct7 = instr[31:25];

  always_comb begin
    case (opcode)
      7'b0010011,
      7'b0000011,
      7'b1100111: imm = {{20{instr[31]}}, instr[31:20]};
      7'b0100011: imm = {{20{instr[31]}}, instr[31:25], instr[11:7]};
      7'b1100011: imm = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0
};
      7'b0110111,
      7'b0010111: imm = {instr[31:12], 12'b0};
      7'b1101111: imm = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:21], 1'
b0};
      default:    imm = 32'b0;
    endcase
  end

  always_comb begin
    reg_write           = 1'b0;
    mem_read            = 1'b0;
    mem_write           = 1'b0;
    branch              = 1'b0;
    alu_src             = 1'b0;
    jump                = 1'b0;
    mem_to_reg          = 1'b0;
    illegal_instruction = 1'b0;
    alu_op              = 4'd0;

    case (opcode)
      7'b0110011: begin // R-type
        reg_write = 1'b1;
        alu_src   = 1'b0;
        case (funct3)
          3'b000: alu_op = funct7[5] ? 4'd1 : 4'd0;
          3'b111: alu_op = 4'd2;
          3'b110: alu_op = 4'd3;
          default: alu_op = 4'd0;
        endcase
      end
      7'b0010011: begin // I-type ALU
        reg_write = 1'b1;
        alu_src   = 1'b1;
        alu_op    = 4'd0;
```

```
      end
      7'b0000011: begin // Load
        reg_write  = 1'b1;
        mem_read   = 1'b1;
        alu_src    = 1'b1;
        mem_to_reg = 1'b1;
        alu_op     = 4'd0;
      end
      7'b0100011: begin // Store
        mem_write = 1'b1;
        alu_src   = 1'b1;
        alu_op    = 4'd0;
      end
      7'b1100011: begin // Branch
        branch = 1'b1;
        alu_op = 4'd1;
      end
      7'b1101111,
      7'b1100111: begin // JAL/JALR
        jump      = 1'b1;
        reg_write = 1'b1;
        alu_src   = 1'b1;
      end
      7'b0110111,
      7'b0010111: begin // LUI/AUIPC
        reg_write = 1'b1;
        alu_src   = 1'b1;
      end
      default: begin
        illegal_instruction = 1'b1;
      end
    endcase
  end
endmodule
```

解释

考点：这是计算机体系结构里的经典模块设计题，重点考察对微架构概念的理解能否落地成可综合的RTL。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Low-Power Design

Q145. Low-Power Design: Latch-based Integrated Clock Gating Cell

QUESTION

Implement a glitch-free integrated clock gating (ICG) cell.

Interface:

Inputs: clk, en

Outputs: gclk

Concept: A naive `gclk = clk & en` can glitch if `en` toggles while `clk` is high. Use a level-sensitive latch on `en` (transparent when `clk == 0`) so `en` is held stable during the `clk == 1` window, then AND with the

raw clock.

ANSWER

```
// Latch-based Integrated Clock Gating (ICG) cell
module clock_gate (
  input  logic clk,
  input  logic en,
  output logic gclk
);

  logic latched_en;

 // Latch transparent when clk == 0 — holds en stable while clk == 1.
  always_latch
    if (!clk) latched_en = en;

  assign gclk = clk & latched_en;

endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

毛刺问题的根源通常是组合逻辑路径延迟不一致，解决思路要么是同步化输出，要么是用专门的无毛刺电路结构（比如格雷码译码、锁存器型时钟门控）。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q146. Low-Power Design: Rewrite a 4:1 MUX Tree to Suppress Internal Glitches

QUESTION

Given a naive 4:1 MUX built as a tree of 2:1 MUXes, identify the internal nets that glitch when the select signals transition together, and rewrite the design to suppress those glitches. Output behavior must match a clean 4:1 MUX exactly.

Interface: shown in the port map.

ANSWER

```
// Glitch-free 4:1 MUX — one-hot decode + AND-OR
module glitchfree_mux4 (
  input  logic [1:0] sel,
  input  logic [7:0] in0,
  input  logic [7:0] in1,
  input  logic [7:0] in2,
  input  logic [7:0] in3,
  output logic [7:0] out
);

  logic [3:0] oh;

 // One-hot decode of sel — only one bit high at any time.
  always_comb begin
    oh = 4'b0000;
    oh[sel] = 1'b1;
  end
```

```
  // AND-OR MUX — no intermediate 2:1 stages to glitch through.
  assign out = ({8{oh[0]}} & in0)
             | ({8{oh[1]}} & in1)
             | ({8{oh[2]}} & in2)
             | ({8{oh[3]}} & in3);

endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

毛刺问题的根源通常是组合逻辑路径延迟不一致，解决思路要么是同步化输出，要么是用专门的无毛刺电路结构（比如格雷码译码、锁存器型时钟门控）。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q147. Low-Power Design: Always-On to Power-Gated Domain Handshake

QUESTION

Design the interface between an always-on (AON) controller and a power-gated peripheral. The AON sends requests. If the peripheral is asleep, the AON must wake it, wait for the peripheral's ready signal (synchronized), then deliver the data. Mid-transaction sleep events must trigger replay on the next wake.

Interface:

Inputs: clk_aon, req, data_in[7:0], pg_ready

Outputs: wake, data_out[7:0], data_valid

ANSWER

```
// Always-on to power-gated domain handshake
module aon_pg_bridge (
  input  logic       clk_aon,
  input  logic       req,
  input  logic [7:0] data_in,
  output logic       wake,
  input  logic       pg_ready,
  output logic [7:0] data_out,
  output logic       data_valid
);

 // Pending bit — only clears on completion
  logic pending;
  logic [7:0] payload;

  // 2-flop synchronizer on pg_ready
  logic pg_ready_s1, pg_ready_s2, pg_ready_s3;
  always_ff @(posedge clk_aon) {pg_ready_s3, pg_ready_s2, pg_ready_s1} <= {pg_ready_s2, pg_rea
dy_s1, pg_ready};

  always_ff @(posedge clk_aon) begin
    if (req && !pending) begin
      pending <= 1'b1;
      payload <= data_in;
    end else if (pending && pg_ready_s2 && pg_ready_s3) begin
      pending <= 1'b0;
    end
  end
```

```
  assign wake       = pending;
  assign data_out   = payload;
  assign data_valid = pending && pg_ready_s2 && pg_ready_s3;

endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

涉及跨时钟域的部分要用两级同步器处理，格雷码保证多比特信号跨域时最多只有一位在变化。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q148. Low-Power Design: Glitch-Free Frequency Divider (1 / 2 / 4)

QUESTION

Implement a clock divider selectable between f/1, f/2, and f/4 via a runtime `sel[1:0]` input. Transitions between divider ratios must be glitch-free — no short pulses, no missed edges. Duty cycle ≈ 50% in steady state.

Interface:

Inputs: clk, sel[1:0]

Outputs: clk_out

ANSWER

```
// Glitch-free divide-by-1/2/4 clock divider
module freq_divider (
  input  logic       clk,
  input  logic [1:0] sel,
  output logic       clk_out
);

  logic       clk_d2;
  logic [1:0] cnt;

  always_ff @(posedge clk) begin
    cnt <= cnt + 1;
    if (cnt[0] == 0) clk_d2 <= ~clk_d2;
  end

  logic clk_d4;
  always_ff @(posedge clk) if (cnt == 2'b11) clk_d4 <= ~clk_d4;

  // Sync select through 2 flops to remove metastability on runtime switch
  logic [1:0] sel_q1, sel_q2;
  always_ff @(negedge clk) {sel_q2, sel_q1} <= {sel_q1, sel};

  always_comb begin
    case (sel_q2)
      2'b00:   clk_out = clk;
      2'b01:   clk_out = clk_d2;
      2'b10:   clk_out = clk_d4;
      default: clk_out = 1'b0;
    endcase
  end
```

```
endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

毛刺问题的根源通常是组合逻辑路径延迟不一致，解决思路要么是同步化输出，要么是用专门的无毛刺电路结构（比如格雷码译码、锁存器型时钟门控）。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q149. Low-Power Design: Find the Glitch in a Naive Clock Gate

QUESTION

A junior engineer wrote `assign gclk = clk & en;` and is seeing intermittent write-through failures on downstream flops. Given the buggy design and a waveform showing the glitch, diagnose the failure and submit a corrected ICG.

Interface: same as `lp-cg-1`.

Skill tested: waveform reading + combinational hazard identification.

ANSWER

```
// Corrected ICG replacing the buggy naive AND
module clock_gate (
  input  logic clk,
  input  logic en,
  output logic gclk
);

  logic latched_en;

  always_latch
    if (!clk) latched_en = en;

  assign gclk = clk & latched_en;

endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

毛刺问题的根源通常是组合逻辑路径延迟不一致，解决思路要么是同步化输出，要么是用专门的无毛刺电路结构（比如格雷码译码、锁存器型时钟门控）。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q150. Low-Power Design: Bus-Invert Coding for Off-Chip Toggle Reduction

QUESTION

Implement an encoder/decoder pair. On each cycle, the encoder computes the Hamming distance between the previous encoded word and the next raw data word; if more than half the bits would flip, send the

inverted word and assert an `inv` bit. The decoder reconstructs the original data from `inv` and the encoded word.

Bus width: 8 bits + 1 inv bit.

ANSWER

```
// Bus-invert coding (8-bit)
module bus_invert (
  input  logic       clk,
  input  logic [7:0] din,
  output logic [7:0] encoded,
  output logic       inv,
  output logic [7:0] decoded
);

  logic [7:0] prev_encoded;
  logic [7:0] xor_bits;
  int         flips;

  // Count how many bits differ between (din) and (previous encoded), and
  // between (~din) and (previous encoded).
  assign xor_bits = din ^ prev_encoded;
  always_comb begin
    flips = 0;
    for (int i = 0; i < 8; i++) if (xor_bits[i]) flips++;
  end

  assign inv     = (flips > 4);
  assign encoded = inv ? ~din : din;

  always_ff @(posedge clk) prev_encoded <= encoded;

 // Decoder — self-check output
  assign decoded = inv ? ~encoded : encoded;

endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

题目要求参数化（可配置位宽/深度），写代码时应该用parameter/localparam而不是硬编码常量，保证模块可复用。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q151. Low-Power Design: SystemVerilog Assertions for Power Sequencing

QUESTION

Given a bound-in power controller interface, write SVA properties that capture the legal power sequence: `ISO` must assert at least one cycle before `RET_SAVE`; `RET_SAVE` must complete before `VDD_OFF`; on wake, `RESTORE` must follow `VDD_STABLE` by at least 2 cycles; and `ISO` must release last. Provide both `assert property` and `cover property` for each.

ANSWER

```
// SVA for power sequencing — bind this to a controller testbench
```

```
module power_sequencing_sva (
  input logic clk,
  input logic iso,
  input logic ret_save,
  input logic ret_restore,
  input logic vdd_off,
  input logic vdd_stable
);

  // P1: ISO must assert at least one cycle before RET_SAVE
  property p_iso_before_save;
    @(posedge clk) $rose(ret_save) |-> iso;
  endproperty
  assert property (p_iso_before_save);
  cover  property (p_iso_before_save);

  // P2: RET_SAVE must complete before VDD_OFF
  property p_save_before_vddoff;
    @(posedge clk) $rose(vdd_off) |-> $past(ret_save,1) || $past(ret_save,2);
  endproperty
  assert property (p_save_before_vddoff);

  // P3: RESTORE must follow VDD_STABLE by at least 2 cycles
  property p_restore_after_stable;
    @(posedge clk) $rose(ret_restore) |-> $past(vdd_stable, 2);
  endproperty
  assert property (p_restore_after_stable);
  cover  property (p_restore_after_stable);

  // P4: ISO must release last on wake-up (after RESTORE)
  property p_iso_last;
    @(posedge clk) $fell(iso) |-> $past(ret_restore, 1);
  endproperty
  assert property (p_iso_last);

  // P5: RESTORE never fires before VDD_STABLE
  property p_no_early_restore;
    @(posedge clk) ret_restore |-> vdd_stable;
  endproperty
  assert property (p_no_early_restore);

endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

实现时要特别注意题目给出的具体数值示例，写完代码后最好口头过一遍这些例子验证逻辑是否自洽。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。

Q152. Low-Power Design: Async FIFO with Level-Shifted Handshakes

QUESTION

Design an 8-entry async FIFO between two domains running at different clocks AND different voltages. Cross-domain signals pass through 2-flop synchronizers and behavioral level shifters. Pointers are Gray-coded. `wfull` and `rempty` are computed in their respective domains.

Interface: shown in the port map.

ANSWER

```
// Async FIFO across differing clock (and modeled voltage) domains
module async_fifo_vf #(
  parameter int DEPTH = 8,
  parameter int WIDTH = 8
) (
  input  logic             wclk,
  input  logic             wreset,
  input  logic             winc,
  input  logic [WIDTH-1:0] wdata,
  output logic             wfull,

  input  logic             rclk,
  input  logic             rreset,
  input  logic             rinc,
  output logic [WIDTH-1:0] rdata,
  output logic             rempty
);

  localparam int AW = $clog2(DEPTH);

  // Memory
  logic [WIDTH-1:0] mem [0:DEPTH-1];

  // Binary and Gray pointers per domain
  logic [AW:0] wbin, wgray;
  logic [AW:0] rbin, rgray;

  // Write side
  logic [AW:0] wbin_next, wgray_next;
  assign wbin_next  = wbin + (winc && !wfull);
  assign wgray_next = (wbin_next >> 1) ^ wbin_next;

  always_ff @(posedge wclk or posedge wreset) begin
    if (wreset) begin wbin <= 0; wgray <= 0; end
    else        begin wbin <= wbin_next; wgray <= wgray_next; end
  end

  always_ff @(posedge wclk) if (winc && !wfull) mem[wbin[AW-1:0]] <= wdata;

  // Read side
  logic [AW:0] rbin_next, rgray_next;
  assign rbin_next  = rbin + (rinc && !rempty);
  assign rgray_next = (rbin_next >> 1) ^ rbin_next;

  always_ff @(posedge rclk or posedge rreset) begin
    if (rreset) begin rbin <= 0; rgray <= 0; end
    else        begin rbin <= rbin_next; rgray <= rgray_next; end
  end

  assign rdata = mem[rbin[AW-1:0]];

  // 2-flop synchronizers (behavioral level-shift = passthrough here)
  logic [AW:0] rgray_s1, rgray_s2;
  always_ff @(posedge wclk) {rgray_s2, rgray_s1} <= {rgray_s1, rgray};

  logic [AW:0] wgray_s1, wgray_s2;
  always_ff @(posedge rclk) {wgray_s2, wgray_s1} <= {wgray_s1, wgray};

  // Flags
  assign wfull  = (wgray_next == {~rgray_s2[AW:AW-1], rgray_s2[AW-2:0]});
  assign rempty = (rgray_next == wgray_s2);
```

```
endmodule
```

解释

考点：这是低功耗设计题，重点考察对毛刺、亚稳态、跨电压域这类低功耗设计里常见隐患的理解。

FIFO类设计的核心永远是满/空判断和读写指针的正确同步，是这道题的关键分数点。

延伸考点：面试官通常会在这道题基础上继续追问边界条件（比如参数取极值时是否还正确）或者要求扩展功能，建议实现时就考虑清楚这些点，回答时主动提及。